
Příprava na bakalářské státnice

Informatika - Umělá inteligence

zaměření **Strojové učení**

MFF UK • studijní plán 2025/2026

Kompletní studijní text postavený na poznámkách *notes-ipp* (Vít Kološ),
strukturovaný a auditovaný proti oficiálním požadavkům SZZ (verze 28. 7. 2025).

Jak tento dokument číst

- Obsah jednotlivých okruhů je **věrně převzatý z poznámek notes-ipp** (submodul repozitáře) a odborně nezměněný.
- Pořadí a členění kopíruje **oficiální požadavky** (požadavky/detailni-pozadavky.pdf, závazný zdroj). Rozsah je přesně zaměření Strojové učení, nic navíc - viz [3.1](#).
- Sekce [3 \(Audit pokrytí požadavků\)](#) ověřuje bod po bodu, že je pokryté *každé* požadované téma, a vypisuje místa, která poznámky jen naznačují.
- **Žluté rámečky** = poznámky odkazují na vlastní procvičení (výpočetní dovednost, ne definiční mezera).
- **Červené rámečky** = téma v poznámkách chybí nebo je jen alternativou; ověř/doplň.

Obsah

1	Struktura a hodnocení zkoušky	2
2	Jak se efektivně učit	2
3	Audit pokrytí požadavků	3
3.1	Rozsah: co je a není zahrnuto	3
3.2	Metodika	3
3.3	Souhrn	4
3.4	Detailní kontrola po tématech	4
3.5	Místa, která poznámky nechávají na vlastní procvičení	5
3.6	Jediná formální výjimka: Hilbertovský kalkul	6
4	Část I: Společná matematika	7
4.1	1. Základy diferenciálního a integrálního počtu	7
4.2	2. Algebra a lineární algebra	11
4.3	3. Diskrétní matematika	20
4.4	4. Teorie grafů	26
4.5	5. Pravděpodobnost a statistika	32
4.6	6. Logika	37
5	Část II: Společná informatika	44
5.1	1. Automaty a jazyky	44
5.2	2. Algoritmy a datové struktury	51
5.3	3. Programovací jazyky	59
5.4	4. Architektura počítačů a operačních systémů	68
6	Část III: Specializace - Základy umělé inteligence	76
6.1	Řešení úloh prohledáváním	76
6.2	Splňování omezujících podmínek	78
6.3	Logické uvažování	79
6.4	Pravděpodobnostní uvažování	81
6.5	Reprezentace znalostí	83
6.6	Automatické plánování	85
6.7	Markovské rozhodovací procesy (MDP)	86
6.8	Hry a teorie her	87
6.9	Strojové učení	89
7	Část IV: Specializace - Strojové učení	93
7.1	Učení s učitelem	93
7.2	Učení založené na příkladech	96
7.3	Lineární regrese	96
7.4	Logistická regrese	97
7.5	Rozhodovací stromy	98
7.6	Metoda podpurných vektorů	99
7.7	Kombinace více modelů	101
7.8	Statistické testy	102
7.9	Učení bez učitele	103

1 Struktura a hodnocení zkoušky

Formát odborné části SZZ (písemná forma)

- Zadáno je typicky **8 otázek**: 2 ze společné matematiky, 2 ze společné informatiky, 4 z okruhů specializace (Základy UI + Strojové učení).
- Za každou odpověď **0-3 body**.
- K úspěchu je nutné získat **min. 5 bodů ze společných okruhů** a **min. 7 bodů ze specializace**.
- Znamka podle součtu (max 24): od **12 b** dobře, od **16 b** velmi dobře, od **20 b** výborně.
- Zkouška má **přehledový charakter**: důraz na pochopení principů, přesné definice a terminologii, *ne* na složité důkazy.

Co to znamená pro přípravu na známku „výborně“ (A): potřebuješ stabilně dávat blízko plný počet bodů, tj. u každé otázky umět *přesnou definici, klíčové tvrzení/větu a hlavní myšlenku* (proč to platí, k čemu to je). Tomu odpovídá i forma tohoto textu - hutné definice a tvrzení, ne dlouhé důkazy.

Okruh	Počet témat	Část zkoušky
Společná matematika	6	společný okruh - 2 otázky
Společná informatika	4	společný okruh - 2 otázky
Základy umělé inteligence	1 (rozsáhlé)	specializace - povinné
Strojové učení	1 (rozsáhlé)	specializace - vlastní zaměření

2 Jak se efektivně učit

Praktický režim přípravy

1. Nejdřív si otevři oficiální checklist a ke každému bodu najdi odpovídající nadpis v tomto PDF. Cílem prvního průchodu není pamatovat si details, ale vědět, kde co leží.
2. U každého tématu si napiš vlastní třířádkovou odpověď: definice, hlavní věta nebo algoritmus, typický příklad použití.
3. Žluté rámečky řeš odděleně jako příklady na papír. Nejsou to mezery v textu, ale bez vlastního výpočtu z nich body snadno utečou.
4. Před zkouškou trénuj odpovědi v režimu 10 minut na otázku. Nejprve kostra odpovědi, potom přesné pojmy a až nakonec důkazy nebo odvození.
5. U specializace propojuj témata: pravděpodobnost, lineární algebra a optimalizace se vrací v Bayesovských sítích, SVM, regresi, PCA i reinforcement learningu.

Doporučené pořadí: matematika kvůli definicím a značení, společná informatika kvůli algoritmům a praktickým úlohám, potom Základy UI a nakonec Strojové učení. U každého okruhu si nech poslední průchod jen na slabá místa z auditu a na vlastní krátké odpovědi.

3 Audit pokrytí požadavků

Závěr auditu

Závazný zdroj pravdy je oficiální dokument `pozadavky/detailni-pozadavky.pdf` („Témata bakalářské SZZ“, verze 28. 7. 2025). Prošel jsem proti němu **všechny** body relevantní pro zaměření Strojové učení a porovnal je s poznámkami `notes-ipp`. **Všechny požadované okruhy a podtémata jsou v poznámkách obsažené** a zároveň **nic mimo požadavky se neučí** (viz 3.1). Jediné formálně sporné místo je *Hilbertovský kalkul*: požadavky ho uvádějí jen jako *jednu z alternativ* formálního dokazovacího systému. Tablo i rezoluce pokryté jsou, Hilbertovský kalkul sám o sobě ne - viz 3.6. Sedm míst poznámky vědomě nechávají na vlastní procvičení (výpočetní dovednosti) - viz 3.5.

3.1 Rozsah: co je a není zahrnuto

Podle oficiálního dokumentu se pro zaměření **Strojové učení** zkouší právě tyto okruhy: *společná matematika* (6 témat), *společná informatika* (4 témata) a ze specializace Umělá inteligence *téma 1 = Základy UI* (společné všem zaměřením) a *téma 3 = Strojové učení*. Tento dokument obsahuje přesně tyto čtyři okruhy a nic víc.

Mezera v poznámkách - ověř / doplň

Záměrně NEzahrnuto (pro toto zaměření se to nezkouší, takže se to neuč): Robotika (téma 2) a Zpracování přirozeného jazyka / NLP (téma 4) ze specializace UI; a celé jiné specializace (Web a datově orientované programování, Obecná informatika, Počítačová grafika a vidění, Systémové programování, Programování a vývoj software). Z toho plyne, že do přípravy **nepatří** např. neuronové sítě a hluboké učení (perceptron, MLP, CNN, word2vec patří k NLP), robotika, počítačové sítě, databáze a SQL, RSA / FFT a pokročilé algoritmy, výpočetní geometrie, softwarové inženýrství. Ověřeno: obsah níže žádné takové mimo-rozsahové téma neobsahuje.

3.2 Metodika

Pro každý okruh jsem vzal seznam podtémat z `detailni-pozadavky.pdf` a ověřil, že odpovídající látka v poznámkách existuje (strukturálně i obsahově - členění poznámek kopíruje oficiální osnovu). U níže uvedených seznamů znamená ✓, že téma je v textu zpracované.

3.3 Souhrn

Okruh	Témat v požadavcích	Pokryto
I/1 Diferenciální a integrální počet	8	✓ vše
I/2 Algebra a lineární algebra	9	✓ vše
I/3 Diskrétní matematika	7	✓ vše
I/4 Teorie grafů	9	✓ vše
I/5 Pravděpodobnost a statistika	8	✓ vše
I/6 Logika	6	✓ vše*
II/1 Automaty a jazyky	4	✓ vše
II/2 Algoritmy a datové struktury	6	✓ vše
II/3 Programovací jazyky	8	✓ vše
II/4 Architektura počítačů a OS	7	✓ vše
III Základy umělé inteligence	9	✓ vše
IV Strojové učení	9	✓ vše

*U logiky chybí jen Hilbertovský kalkul. Jedno PDF ho uvádí alternativně, druhé méně jednoznačně; tablo a rezoluce pokryté jsou.

3.4 Detailní kontrola po tématech

Část I - Společná matematika

1. Diferenciální a integrální počet

- ✓ posloupnosti a limity, aritmetika limit
- ✓ věta o dvou polícijských
- ✓ řady; geometrická a harmonická řada
- ✓ limita funkce, spojitost (mezihodnoty, maximum)
- ✓ derivace, l'Hospital, průběh funkce
- ✓ Taylorův polynom (limitní forma)
- ✓ primitivní funkce, Riemann/Newton
- ✓ aplikace integrálu (obsah, objem, povrch, délka)

2. Algebra a lineární algebra

- ✓ grupy, permutace, tělesa (konečná)
- ✓ soustavy rovnic, Gauss(-Jordan)
- ✓ matice, hodnost, inverze
- ✓ vektorové prostory, Steinitz, báze, dimenze
- ✓ lineární zobrazení, obraz/jádro, isomorfismus
- ✓ skalární součin, Gram-Schmidt, projekce
- ✓ determinanty (Laplace, geometrie)
- ✓ vlastní čísla, diagonalizace, spektrální rozklad
- ✓ pozitivně (semi)definitní, Cholesky

3. Diskrétní matematika

- ✓ vlastnosti relací
- ✓ ekvivalence a rozklady
- ✓ částečná uspořádání, výška/šířka
- ✓ funkce a jejich počty

- ✓ kombinační čísla, binomická věta
- ✓ inkluze a exkluze (i aplikace)
- ✓ Hallova věta a SRR

4. Teorie grafů

- ✓ základní pojmy a příklady grafů
- ✓ souvislost, komponenty, vzdálenost
- ✓ stromy a charakteristiky
- ✓ rovinné grafy, Eulerova formule
- ✓ barevnost a klikovost
- ✓ Mengerova věta (obě verze)
- ✓ orientované grafy, silná/slábá souvislost
- ✓ toky, Ford-Fulkerson

5. Pravděpodobnost a statistika

- ✓ prostor, podmíněná pst, Bayes
- ✓ náhodné veličiny, distribuce, hustota
- ✓ střední hodnota, Markovova nerovnost
- ✓ rozptyl (i pro součet)
- ✓ konkrétní rozdělení
- ✓ LLN a centrální limitní věta
- ✓ bodové a intervalové odhady
- ✓ testování hypotéz, chyby I/II, hladina

6. Logika

- ✓ syntaxe, CNF/DNF/PNF, SAT/rezoluce

- ✓ sémantika, splnitelnost, důsledek
- ✓ extenze, konzervativnost, skolemizace
- ✓ dokazatelnost: tablo, rezoluce

- ✓ kompaktnost a úplnost
- ✓ rozhodnutelnost

Část II - Společná informatika

1. Automaty a jazyky

- ✓ regulární jazyky (gramatiky, DFA/NFA, regexy)
- ✓ bezkontextové (gramatiky, zásobníkový automat)
- ✓ rekurzivně spočetné (typ 0, Turing, nerozhodnutelnost)
- ✓ Chomského hierarchie

2. Algoritmy a datové struktury

- ✓ složitost, asymptotická notace
- ✓ P, NP, převoditelnost, NP-úplnost
- ✓ rozděl a panuj, Master theorem
- ✓ vyhledávací stromy, AVL
- ✓ třídění a dolní odhad
- ✓ grafové algoritmy (BFS/DFS, Dijkstra, MST, toky)

3. Programovací jazyky

- ✓ abstrakce, dědičnost, polymorfismus
- ✓ typy, reference, boxing
- ✓ generika a funkcionální prvky
- ✓ zdroje a výjimky
- ✓ životní cyklus, GC, reference counting
- ✓ vlákna a synchronizace
- ✓ vtable a implementace OO
- ✓ bytecode, JIT/AOT, linkování, knihovny

4. Architektura počítačů a OS

- ✓ architektura, reprezentace čísel, adresování
- ✓ instrukční sada a vazba na vyšší jazyk
- ✓ privilegovaný režim, jádro
- ✓ periférie: PIO, přerušení
- ✓ procesy/vlákna, kontext, plánování
- ✓ virtuální paměť, stránkování, ochrana
- ✓ soubory, race conditions, zámky

Část III - Základy UI / Část IV - Strojové učení

III. Základy umělé inteligence

- ✓ prohledávání (DFS/BFS/UCS, A*, heuristiky)
- ✓ CSP, AC-3, MAC
- ✓ logické uvažování (DPLL, řetězení, rezoluce)
- ✓ pravděpodobnostní uvažování, Bayesovy sítě
- ✓ reprezentace znalostí, HMM vs. DBN
- ✓ automatické plánování
- ✓ MDP, Bellman, value/policy iteration, POMDP
- ✓ hry (minimax, alfa-beta), Nash, mechanism design
- ✓ ML přehledově (učení, stromy, SVM, EM, RL)

IV. Strojové učení

- ✓ učení s učitelem, metriky, regularizace
- ✓ k-nejbližších sousedů
- ✓ lineární regrese (LSQ, SGD)
- ✓ logistická regrese (sigmoid, softmax)
- ✓ rozhodovací stromy, prořezávání
- ✓ SVM (separabilní/neseperabilní, jádra, multiclass)
- ✓ kombinace modelů (bagging, boosting, random forest)
- ✓ statistické testy (t-test, chí-kvadrát)
- ✓ učení bez učitele (k-means, hierarchické, PCA)

3.5 Místa, která poznámky nechávají na vlastní procvičení

Nejde o chybějící definice, ale o **výpočetní dovednosti**, které text vědomě nerozepisuje. V příslušných okruzích jsou navíc označené žlutým rámečkem přímo u tématu.

1. Derivační pravidla (součin, podíl, složená funkce) - I/1.
2. Integrační techniky: substitute a per partes - I/1.
3. Odhad součtu řady přes integrál (konkrétní postup) - I/1.
4. Vlastní výpočet determinantu - I/2.
5. Překlad konstrukcí vyššího jazyka do instrukcí procesoru - II/4.
6. Opačný směr: poznat konstrukci z dané sekvence instrukcí - II/4.
7. Programem řízená obsluha zařízení (PIO) pro zadané adresy a porty - II/4.

3.6 Jediná formální výjimka: Hilbertovský kalkul

Mezera v poznámkách - ověř / doplň

Požadavky u okruhu *Logika* zmiňují dokazovací systémy. V jednom PDF je formulace „tablo, rezoluce **nebo** Hilbertovský kalkul“, ve druhém jen „tablo, rezoluce, Hilbertovský kalkul“. Poznámky podrobně rozebírají **tablo metodu i rezoluci. Hilbertovský (axiomatický) kalkul** v poznámkách **není**. Pokud bys ho chtěl umět jako pojistku, doplň si: jazyk s několika *axiomovými schématy* (implikační fragment) a jediným odvozovacím pravidlem *modus ponens*; důkaz je posloupnost formulí, kde každá je buď axiom, předpoklad, nebo plyne z předchozích pravidlem; věta o dedukci. Toto si **ověř** v oficiálních materiálech NAIL062, pokud bys na něj chtěl spoléhat.

4 Část I: Společná matematika

4.1 1. Základy diferenciálního a integrálního počtu

- Posloupnosti reálných čísel a jejich limity (definice)
 - posloupnost s hodnotami v množině M je zobrazení z $\mathbb{N}$ do M
 - nechť (a_n) je reálná posloupnost a $L \in \mathbb{R}^*$
 - pokud $(\forall \varepsilon > 0)(\exists n_0 \in \mathbb{N}) : (n \geq n_0 \implies |a_n - L| < \varepsilon)$, píšeme, že $\lim a_n = L$ nebo $\lim_{n \rightarrow \infty} a_n = L$ nebo $a_n \rightarrow L$, a řekneme, že posloupnost (a_n) má limitu L
 - pro $L \in \mathbb{R}$ mluvíme o vlastní limitě a pro $L = \pm\infty$ o limitě nevlastní
 - posloupnost, která má vlastní limitu, konverguje (pokud ji nemá, tak diverguje)
 - každá posloupnost reálných čísel má nejvýše jednu limitu
- Aritmetika limit
 - nechť $(a_n), (b_n)$ jsou posloupnosti reálných čísel takové, že $\lim a_n = a$ a $\lim b_n = b$
 - pak...
 - $\lim (a_n + b_n) = a + b$
 - $\lim (a_n b_n) = ab$
 - pokud $b_n \neq 0$ pro každé $n > 0$, pak $\lim (a_n/b_n) = a/b$
 - to vše platí za předpokladu, že je výraz na pravé straně definován
 - neurčitě výrazy (nejsou definované): součet nekonečen s různými znaménky, součin nekonečna s nulou, podíl nekonečen, dělení reálného čísla nulou
 - pokud je (a_n) omezená a (b_n) konverguje k nule, pak $\lim (a_n b_n) = 0$
 - tzn. limita (a_n) nemusí existovat
- Limity a uspořádání
 - nechť $(a_n), (b_n)$ jsou posloupnosti reálných čísel takové, že mají limity $\lim a_n = a$ a $\lim b_n = b$
 - když $a < b$, tak existuje n_0 , že $n > n_0 \implies a_n < b_n$
 - a naopak, když $a_n \leq b_n$ pro každé $n > n_0$, pak $a \leq b$
 - poznámka: může se stát, že $\forall n : a_n < b_n$, ale $\lim a_n = \lim b_n$
 - např. uvažujme $a_n = 1 - \frac{1}{n} < b_n = 1$, přičemž $\lim a_n = \lim b_n = 1$
- Věta o dvou policajtech
 - nechť posloupnosti $(a_n), (b_n), (c_n) \subseteq \mathbb{R}$ splňují
 - $\lim a_n = \lim b_n = a \in \mathbb{R}$
 - $\forall n > n_0 : a_n \leq c_n \leq b_n$
 - pak (c_n) konverguje a $\lim c_n = a$
 - platí to i pro nevlastní limity - tam nám stačí jeden policajt
- Součet řady a částečný součet řady
 - nekonečná řada (reálných čísel) je výraz

$$\sum_{n=1}^{\infty} a_n = a_1 + a_2 + a_3 + \dots$$

- kde $(a_n)_{n \geq 1}$ je posloupnost reálných čísel
- $(n$ -tý) částečný součet řady se značí s_n a je to součet jejích prvních n členů ($s_n = a_1 + a_2 + \dots + a_n$)
- pokud existuje vlastní limita posloupnosti (s_n) částečných součtů dané řady, mluvíme o *konvergentní* řadě a limita $\lim s_n$ je jejím *součtem*
- pokud $\lim s_n$ neexistuje nebo je nevlastní, je daná řada *divergentní*
- Geometrická řada, harmonická řada

- geometrická řada
 - $\sum_{n=0}^{\infty} q^n = 1 + q + q^2 + q^3 + \dots$
 - $q \in \mathbb{R}$ je parametr zvaný *kvocient*
 - pro součet geometrické řady platí

$$\sum_{n=0}^{\infty} q^n = \begin{cases} \frac{1}{1-q} & -1 < q < 1 \\ +\infty & q \geq 1 \\ \text{neexistuje} & q \leq -1 \end{cases}$$

- vyplývá to ze vzorce pro částečný součet $s_n = \frac{q^{n+1}-1}{q-1}$
- harmonická řada
 - $\sum_{n=1}^{\infty} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \dots$
 - diverguje, přestože její prvky konvergují k nule
- pro řady tvaru $\sum_{n=1}^{\infty} \frac{1}{n^s}$ obecně platí, že konvergují pro $s > 1$ a divergují pro $s \leq 1$
- Limita funkce v bodě: definice, aritmetika limit
 - okolí bodu je interval $U(a, \delta) = (a - \delta, a + \delta)$
 - okolí nekonečen definujeme jako $U(+\infty, \delta) = (1/\delta, +\infty)$, podobně pro $-\infty$
 - definujeme také pravé okolí bodu $U^+(a, \delta) = [a, a + \delta)$, podobně levé okolí U^-
 - prstencová okolí bodu $a \in \mathbb{R}$ definujeme jako obyčejná okolí s vyjmutým bodem a , značí se jako $P(a, \delta)$ apod.
 - řekneme, že funkce f má v bodě $a \in \mathbb{R}^*$ limitu $A \in \mathbb{R}$, když $(\forall \varepsilon > 0)(\exists \delta > 0) : x \in P(a, \delta) \implies f(x) \in U(A, \varepsilon)$
 - zapisujeme $\lim_{x \rightarrow a} f(x) = A$
 - poznámka: limita funkce f v bodě a nezávisí na její hodnotě v a , funkce f dokonce ani nemusí být v a definovaná
 - pokud místo prstencového okolí P použijeme pravé prstencové okolí P^+ , dostaneme definici limity zprava, značí se $\lim_{x \rightarrow a^+} f(x)$
 - * obdobně lze definovat limitu zleva
 - * pokud $\lim_{x \rightarrow a^+} f(x) = \lim_{x \rightarrow a^-} f(x) = A$, pak $\lim_{x \rightarrow a} f(x) = A$
 - * naopak pokud jsou limity zprava a zleva různé, limita neexistuje
 - podobně jako u posloupnosti - pokud limita funkce existuje, je jednoznačně určena
 - limitu funkce lze rovněž definovat pomocí posloupnosti
 - aritmetika limit
 - necht $a, A, B \in \mathbb{R}^*$, necht f, g jsou funkce definované na nějakém prstencovém okolí $P(a, \delta)$ bodu a , necht platí $\lim_{x \rightarrow a} f(x) = A$ a $\lim_{x \rightarrow a} g(x) = B$
 - pak platí $\lim_{x \rightarrow a} f(x) + g(x) = A + B$, je-li tento součet definován
 - $\lim_{x \rightarrow a} f(x)g(x) = AB$, je-li tento součin definován
 - pokud je $g(x)$ nenulová na nějakém prstencovém okolí bodu a , pak $\lim_{x \rightarrow a} f(x)/g(x) = A/B$, je-li tento podíl definován
- Limita funkce v bodě: vztah s uspořádáním
 - necht $c \in \mathbb{R}^*$ a funkce f, g, h jsou definované na prstencovém okolí bodu c
 - mají-li f, g v bodě c limitu a $\lim_{x \rightarrow c} f(x) > \lim_{x \rightarrow c} g(x)$, pak existuje $\delta > 0$ takové, že $f(x) > g(x)$ pro každé $x \in P(c, \delta)$
 - existuje-li $\delta > 0$ takové, že $f(x) \geq g(x)$ pro každé $x \in P(c, \delta)$, a mají-li funkce f, g limitu v bodě c , potom $\lim_{x \rightarrow c} f(x) \geq \lim_{x \rightarrow c} g(x)$
 - existuje-li $\delta > 0$ takové, že $f(x) \leq h(x) \leq g(x)$ pro každé $x \in P(c, \delta)$ a $\lim_{x \rightarrow c} f(x) = \lim_{x \rightarrow c} g(x) = A \in \mathbb{R}^*$ potom i $\lim_{x \rightarrow c} h(x) = A$
 - v podstatě dva policajti

- Limita funkce v bodě: limita složené funkce
 - necht $A, B, C \in \mathbb{R}^*$, necht $g(x)$ je funkce splňující $\lim_{x \rightarrow A} g(x) = B$ a necht $f(x)$ je funkce splňující $\lim_{x \rightarrow B} f(x) = C$
 - navíc necht je splněna aspoň jedna z následujících podmínek
 - $f(x)$ je spojitá v B
 - na nějakém prstencovém okolí bodu A funkce $g(x)$ nikde nenabývá hodnotu B
 - potom $\lim_{x \rightarrow A} f(g(x)) = C$
- Spojitost a spojitost na intervalu
 - řekneme, že funkce f je v bodě $a \in \mathbb{R}$ spojitá, pokud $\lim_{x \rightarrow a} f(x) = f(a)$
 - spojitost zprava/zleva definujeme pomocí jednostranných limit
 - funkce je spojitá na intervalu, pokud je spojitá v každém vnitřním bodu a pokud je (odpovídajícím způsobem) jednostranně spojitá v každém krajním bodu
 - funkce může být v daném bodě buď spojitá, nebo nespojitá, nebo nedefinovaná
- Funkce spojité na intervalu: nabývání mezíhodnot
 - necht $a < b$ jsou reálná čísla
 - necht $f : [a, b] \rightarrow \mathbb{R}$ je na intervalu $[a, b]$ spojitá
 - označme $m = \min\{f(a), f(b)\}$ a $M = \max\{f(a), f(b)\}$
 - pak každé reálné číslo z intervalu $[m, M]$ je hodnotou funkce f (pro každé $y \in [m, M]$ existuje $x \in [a, b]$ takové, že $f(x) = y$)
- Funkce spojité na intervalu: nabývání maxima
 - necht $a \leq b$ jsou reálná čísla
 - necht $f : [a, b] \rightarrow \mathbb{R}$ je spojitá funkce
 - potom f nabývá na intervalu $[a, b]$ svého maxima (i minima)
- Derivace - definice a základní pravidla pro výpočet
 - necht $f : M \rightarrow \mathbb{R}$, $b \in M$ a $U(b, \delta) \subseteq M$ pro nějaké $\delta > 0$
 - derivace funkce f v bodě b je limita

$$f'(b) := \lim_{h \rightarrow 0} \frac{f(b+h) - f(b)}{h} = \lim_{x \rightarrow b} \frac{f(x) - f(b)}{x - b}$$

- také můžeme mít derivaci zprava/zleva, pak je limita jednostranná
- derivace buď existuje vlastní, nebo existuje nevlastní, nebo neexistuje
- pokud má f v b vlastní derivaci, říkáme, že f je v b diferencovatelná
- pro pravidla derivací viz libovolnou vhodnou tabulku, některá složitější jsou v přípravě na zápočtový test

Procvičit (poznámky odkazují na vlastní trénink)

Poznámky zde derivační pravidla nerozepisují. Natrénuj derivace součinu, podílu a složené funkce na konkrétních příkladech (viz tabulka derivací).

- L'Hospitalovo pravidlo
 - necht $a \in \mathbb{R}^*$, necht funkce $f, g : P(a, \delta) \rightarrow \mathbb{R}$ mají na $P(a, \delta)$ vlastní derivaci a necht $g'(x) \neq 0$ na $P(a, \delta)$
 - pokud $\lim_{x \rightarrow a} f(x) = \lim_{x \rightarrow a} g(x) = 0$ a $\lim_{x \rightarrow a} \frac{f'(x)}{g'(x)} = A \in \mathbb{R}^*$, pak i $\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = A$
 - pokud $\lim_{x \rightarrow a} |g(x)| = +\infty$ a $\lim_{x \rightarrow a} \frac{f'(x)}{g'(x)} = A \in \mathbb{R}^*$, pak i $\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = A$
- Vyšetření průběhu funkcí: extrémy, monotonie a konvexita/konkavita
 - extrémy
 - pokud má funkce f v bodě a nenulovou derivaci $f'(a) \neq 0$, pak f v a nenabývá lokální extrém

- tedy pokud má f lokální extrém bodě a , tak buď $f'(a) = 0$, nebo $f'(a)$ neexistuje
- pozor, funkce může mít globální extrém v krajním bodě uzavřeného intervalu
- monotonie
 - uvažujeme interval s kladnou délkou a spojitou funkci f s derivací v každém vnitřním bodě intervalu
 - pokud na vnitřku intervalu platí $f' > 0$, je f na daném intervalu rostoucí
 - pokud $f' \geq 0$, pak je neklesající
 - pokud $f' < 0$, pak je klesající
 - pokud $f' \leq 0$, pak je nerostoucí
 - pokud $f' = 0$, pak je konstantní
- konvexita/konkavita
 - zjednodušená definice: funkce f je na intervalu (a, b) konvexní, pokud graf funkce na intervalu (a, b) leží pod úsečkou spojující body $(a, f(a))$ a $(b, f(b))$
 - * respektive pokud leží pod nebo na úsečce, tak je konvexní, pokud leží ostře pod úsečkou, tak je ryze konvexní
 - * podobně konkávnost
 - pokud je druhá derivace funkce kladná, je ryze konvexní (pokud je nezáporná, je konvexní)
 - pokud je druhá derivace funkce záporná, je ryze konkávní (pokud je nekladná, je konkávní)
- Taylorův polynom (limitní forma)
 - Taylorův polynom řádu n funkce f v bodě a :

$$T_n^{f,a}(x) = \sum_{i=0}^n \frac{f^{(i)}(a)}{i!} (x-a)^i$$

- poznámka: nultý člen součtu bude vždy $f(a)$
- má-li funkce f v bodě $a \in \mathbb{R}$ derivace všech řádů, rozumíme pro $x \in \mathbb{R}$ její taylorovou řadou se středem v a řadu

$$T^{f,a}(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$$

- součet Taylorovy řady je $f(x)$
- Primitivní funkce: definice a metody výpočtu (substituce, per-partes)
 - nechť $-\infty \leq a < b \leq +\infty$ a $f : (a, b) \rightarrow \mathbb{R}$ je daná funkce
 - pokud má funkce $F : (a, b) \rightarrow \mathbb{R}$ na (a, b) derivaci a ta se rovná $f(x)$, řekneme, že F je na intervalu (a, b) primitivní funkcí k funkci f
 - primitivní funkce je jednoznačná až na konstantu
 - Newtonův integrál funkce f na intervalu (a, b) definujeme jako

$$(N) \int_a^b f(x) dx := [F]_a^b = F(b^-) - F(a^+) = \lim_{x \rightarrow b^-} F(x) - \lim_{x \rightarrow a^+} F(x)$$

- poznámka: konstanta c se odečte
- substituci a per-partes je nutné nastudovat (i pro určitý integrál)

Procvičit (poznámky odkazují na vlastní trénink)

Výpočetní techniky (substituce, per partes) si projdi na příkladech - v poznámkách je jen vzorec, ne postup.

- vzorec pro per-partes: $\int u'v = uv - \int uv'$
- Riemannův integrál: definice, souvislost s primitivní funkcí (Newtonovým integrálem)
 - uvažujeme dělení $D = (a_0, a_1, \dots, a_k)$ intervalu $[a, b]$

- tzn. $a = a_0 < a_1 < a_2 < \dots < a_k = b$
- definujeme dolní Riemannovu sumu $s(f, D) = \sum_{i=0}^{k-1} |I_i| m_i$
 - $I_i = [a_i, a_{i+1}]$
 - m_i je infimum funkce f v intervalu I_i
- také definujeme horní Riemannovu sumu $S(f, D) = \sum_{i=0}^{k-1} |I_i| M_i$
 - $M_i \dots$ supremum
- dolní Riemannův integrál na intervalu $[a, b]$ definujeme jako supremum z $s(f, D)$ pro všechna dělení D intervalu $[a, b]$
 - značí se $\int_a^b f$
- horní Riemannův integrál je infimum z $S(f, D)$
 - $\overline{\int_a^b f}$
- funkce má Riemannův integrál, pokud se horní a dolní R. integrály rovnají (pak tuhle společnou hodnotu nazveme Riemannovým integrálem)
 - $\int_a^b f$
- 2\.. základní věta analýzy: pokud Riemannův a Newtonův integrál existují, pak jsou si rovny
- Aplikace integrálů: odhady součtu řad (konečných i nekonečných)
 - pro přirozené n a neklesající funkci f na intervalu $[1, n]$ platí

$$\sum_{k=1}^{n-1} f(k) \leq \int_1^n f \leq \sum_{k=2}^n f(k)$$

- integrální kritérium konvergence
 - uvažujme nerostoucí nezápornou funkci f
 - řada $\sum_{k=1}^{\infty} f(k)$ konverguje $\iff \lim_{b \rightarrow +\infty} \int_1^b f < +\infty$
- přesný postup pro odhad součtu řady je potřeba nacvičit

Procvičit (poznámky odkazují na vlastní trénink)

Odhad součtu řady přes integrál je výpočetní dovednost - vyřeš pár konkrétních příkladů.

- Aplikace integrálů: obsahy rovinných útvarů
 - plocha rovinného útvaru $U(a, b, f)$ pod grafem funkce $f \dots \int_a^b f$
- Aplikace integrálů: objemy a povrchy rotačních útvarů v prostoru
 - $V \dots$ rotační těleso vzniklé rotací rovinného útvaru $U(a, b, f)$ pod grafem funkce f kolem osy x
 - $\text{objem}(V) = \pi \int_a^b f(t)^2 dt$
 - $\text{povrch}(V) = 2\pi \int_a^b f(t) \sqrt{1 + (f'(t))^2} dt$
- Aplikace integrálů: délka křivky
 - uvažujeme křivku z bodu a do bodu b popsanou funkcí f
 - $\int_a^b \sqrt{1 + (f'(t))^2} dt$

4.2 2. Algebra a lineární algebra

- Grupy a podgrupy, permutace
 - binární operace na množině X je zobrazení $X \times X \rightarrow X$
 - neutrální prvek: $(\exists e \in G)(\forall a \in G) : a \circ e = e \circ a = a$
 - inverzní prvek: $(\forall a \in G)(\exists b \in G) : a \circ b = b \circ a = e$
 - grupa $(G, \circ)$ je množina G spolu s binární operací $\circ$ na G splňující asociativitu operace $\circ$, existenci neutrálního prvku a existenci inverzních prvků

- pokud je navíc operace $\circ$ komutativní, pak se jedná o abelovskou grupu
- grupa $(H, \bullet)$ je podgrupou grupy $(G, \circ)$, pokud $H \subseteq G$ a $\forall a, b \in H : a \bullet b = a \circ b$
 - píšeme $(H, \bullet) \leq (G, \circ)$
- permutace na množině $\{1, 2, \dots, n\}$ je bijektivní zobrazení $p : \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$
 - skládání permutací ... $(q \circ p)(i) = q(p(i))$
 - permutační matice
 - * $(P)_{i,j} = \begin{cases} 1 & \text{pokud } p(i) = j \\ 0 & \text{jinak} \end{cases}$
 - cyklus ... např. $(1, 2, 3, 4)$
 - * odpovídá zobrazení, které mapuje $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 4, 4 \rightarrow 1$
 - transpozice ... permutace, která má pouze jeden netriviální cyklus o délce 2, tedy např. $(1, 3)$
 - jakoukoliv permutaci lze rozložit na transpozice
 - * cyklus $(1, 2, 3, 4)$ lze rozložit na $(1, 4) \circ (1, 3) \circ (1, 2)$ nebo na $(1, 2) \circ (2, 3) \circ (3, 4)$
 - znaménko permutace p je $\text{sgn}(p) = (-1)^{\text{počet inverzí v } p}$
 - * inverze v p je dvojice prvků $(i, j) : i < j \wedge p(i) > p(j)$
 - * nebo také $\text{sgn}(p) = (-1)^{\text{počet transpozic v } p}$
- Tělesa a speciálně konečná tělesa
 - těleso je množina $\mathbb{K}$ spolu se dvěma komutativními binárními operacemi $+$ a $\cdot$, kde $(\mathbb{K}, +)$ a $(\mathbb{K} \setminus \{0\}, \cdot)$ jsou abelovské grupy a navíc platí distributivita $\forall a, b, c \in \mathbb{K} : a \cdot (b + c) = (a \cdot b) + (a \cdot c)$
 - charakteristika tělesa
 - v tělese $\mathbb{K}$, pokud $\exists n \in \mathbb{N} : \underbrace{1 + 1 + \dots + 1}_n = 0$, pak nejmenší takové n je charakteristika tělesa $\mathbb{K}$
 - jinak má těleso $\mathbb{K}$ charakteristiku 0
 - značí se $\text{char}(\mathbb{K})$
 - lze dokázat, že u konečných těles musí být prvočíselná
 - $\mathbb{Z}_p$ je těleso, právě když je p prvočíslo
 - ale existují i konečná tělesa s neprvočíselným řádem (počtem prvků) - např. Galois field $GF(4)$
 - řád musí být kladná celá mocnina prvočísla
- Soustavy lineárních rovnic - maticový zápis, elementární řádkové úpravy, odstupňovaný tvar matice
 - maticový zápis soustavy rovnic
 - soustava m lineárních rovnic o n neznámých ... $Ax = b$
 - rozšířená matice soustavy ... $(A|b) \in \mathbb{R}^{m \times (n+1)}$
 - matice soustavy ... $A \in \mathbb{R}^{m \times n}$
 - vektor pravých stran ... $b \in \mathbb{R}^m$
 - vektor neznámých ... $x = (x_1, \dots, x_n)^T$
 - vektor $x \in \mathbb{R}^n$ je řešení soustavy $Ax = b$, pokud splňuje všechny její rovnice
 - soustavy $Ax = 0$ se nazývají homogenní a vždy umožňují $x = 0$
 - elementární řádkové operace
 - definujeme základní dvě elementární řádkové úpravy
 - * vynásobení i -tého řádku nenulovým $t \in \mathbb{R} \setminus \{0\}$
 - * přičtení j -tého řádku k i -tému řádku
 - z těch lze odvodit další dvě úpravy
 - * přičtení t -násobku j -tého řádku k i -tému řádku (t může být i nulové)

- * záměna dvou řádků
- provedení jedné elementární úpravy značíme $A \sim A'$
- provedení posloupnosti úprav značíme $A \sim\sim A'$
- elementárními úpravami soustavy $(A|b)$ se nemění množina řešení
- odstupňovaný tvar matice
 - matice je v řádkově odstupňovaném tvaru (REF = row echelon form), pokud jsou nenulové řádky (ostře) seřazeny podle počtu počátečních nul a nulové řádky jsou pod nenulovými
 - * k formální definici bychom zavedli notaci $j(i)$ pro pozici prvního nenulového prvku v i -tém řádku
 - první nenulový prvek nenulového řádku se nazývá pivot, pod pivotem jsou v REF všechny prvky nulové
 - pro soustavu $A'x = b'$ s A' v REF jsou proměnné odpovídající sloupcům s pivoty báze, ostatní jsou volné
- redukovaný odstupňovaný tvar (RREF)
 - pivoty jsou jednotkové
 - nad i pod pivoty jsou nuly
- Gaussova a Gaussova-Jordanova eliminace, popis množiny řešení
 - Gaussova eliminace = převod do odstupňovaného tvaru pomocí elementárních úprav, to se dá popsat algoritmicky:
 - seřadíme řádky podle počtu počátečních nul
 - najdeme první dva řádky, co mají stejně počátečních nul
 - od toho druhého (a všech dalších, co mají stejně nul) odečteme vhodný násobek toho prvního, abychom mu první nenulu vynulovali
 - tohle opakujeme, dokud se nedostaneme do REF
 - Gaussova-Jordanova eliminace = převod do RREF
 - z pivotů se jedničky dělají snadno - stačí řádek vydělit pivotem
 - nad pivoty se jedničky také dostávají snadno - stačí od řádku odečíst vhodný násobek řádku s pivotem v daném sloupci (dává smysl postupovat odspodu)
 - množina řešení
 - pokud je pivot v posledním sloupci rozšířené matice soustavy, soustava rovnic nemá řešení
 - existuje bijekce $\bar{x} \mapsto \bar{x} + x^0$ mezi množinami $\{\bar{x} : A\bar{x} = 0\}$ a $\{x : Ax = b\}$
 - * pokud x^0 splňuje $Ax^0 = b$
 - množinu řešení soustavy $Ax = b$ popíšeme jako $x = y + p_1\bar{x}_1 + p_2\bar{x}_2 + \dots + p_{n-r}\bar{x}_{n-r}$
 - $y \dots$ libovolné řešení soustavy $Ax = b$
 - * u homogenní soustavy to typicky bude nulový vektor
 - * je to člen, který odpovídá tomu x^0 výše
 - $p_i\bar{x}_i$
 - * takový člen tam bude za každou volnou proměnnou (bude jich $n - r$, tedy počet řádků - hodnost matice)
 - * p_i je libovolný reálný parametr
 - * tyto členy můžeme získat tak, že řešíme soustavu $A\bar{x} = 0$ a řešení vyjádříme pomocí parametrů (a pak je vytkneme)
 - * nebo je lze vyčíst z RREF matice
 - * ve vektoru $\bar{x}_i$ bude typicky jednička na místě odpovídající volné proměnné a nuly u všech ostatních volných proměnných (protože je to vlastně složka, která se stará o konkrétní volnou proměnnou)
- Operace s maticemi a základní typy matic, hodnost matice

- hodnost matice
 - hodnost matice A , značená jako $\text{rank}(A)$, je počet pivotů v libovolné A' v REF takové, že $A \sim A'$
- jednotková matice
 - pro $n \in \mathbb{N}$ je jednotková matice $I_n \in \mathbb{R}^{n \times n}$ definovaná tak, že $(I_n)_{i,j} = 1 \iff i = j$, ostatní prvky jsou nulové
- transponovaná matice
 - transponovaná matice k matici $A \in \mathbb{R}^{m \times n}$ je matice $A^T \in \mathbb{R}^{n \times m}$ splňující $(A^T)_{i,j} = a_{j,i}$
- symetrická matice
 - čtvercová matice A je symetrická, pokud $A^T = A$, tedy $a_{i,j} = a_{j,i}$
- maticový součin
 - pro $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$ je součin $(AB) \in \mathbb{R}^{m \times p}$ definován $(AB)_{i,j} = \sum_{k=1}^n a_{i,k} b_{k,j}$
- Regulární a inverzní matice
 - inverzní matice
 - pokud pro čtvercovou matici $A \in \mathbb{R}^{n \times n}$ existuje $B \in \mathbb{R}^{n \times n}$ taková, že $AB = I_n$, pak se B nazývá inverzní matice a značí se A^{-1}
 - výpočet: $(A|I_n) \sim (I_n|A^{-1})$
 - regulární/singulární matice
 - pokud má matice A inverzi, pak se nazývá regulární, jinak je singulární
 - věta: pro čtvercovou matici $A \in \mathbb{R}^{n \times n}$ jsou následující podmínky ekvivalentní
 1. matice A je regulární, tedy k ní existuje inverzní matice $\dots \exists B : AB = I_n$
 2. $\text{rank}(A) = n$
 3. $A \sim I_n$
 4. systém $Ax = 0$ má pouze triviální řešení $x = 0$
 - inverzní matici k matici A lze najít pomocí Gaussovy-Jordanovy eliminace $(A|I_n) \sim (I_n|A^{-1})$
 - pokud tento proces selže, tak je A singulární
- Vektorový prostor, lineární kombinace, lineární závislost a nezávislost, lineární obal, systém generátorů
 - vektorový prostor
 - vektorový prostor $(V, +, \cdot)$ nad tělesem $(\mathbb{K}, +, \cdot)$ je množina spolu s binární operací $+$ na V a binární operací skalárního násobku $\cdot : \mathbb{K} \times V \rightarrow V$
 - $(V, +)$ je abelovská grupa
 - $\forall \alpha, \beta \in \mathbb{K}, \forall u, v \in V$
 - * asociativita $\dots (\alpha \cdot \beta) \cdot u = \alpha \cdot (\beta \cdot u)$
 - * neutrální prvek (skalár) vůči násobení skalárem $\dots 1 \cdot u = u$
 - * distributivita $\dots (\alpha + \beta) \cdot u = (\alpha \cdot u) + (\beta \cdot u)$
 - * distributivita $\dots \alpha \cdot (u + v) = (\alpha \cdot u) + (\alpha \cdot v)$
 - prvky $\mathbb{K}$ se nazývají skaláry, prvky V vektory
 - rozlišujeme nulový skalár 0 a nulový vektor o
 - podprostor vektorového prostoru
 - nechť V je vektorový prostor na $\mathbb{K}$, potom podprostor U je neprázdná podmnožina V splňující uzavřenost na součet vektorů a uzavřenost na násobení skalárem (z $\mathbb{K}$) - z toho nutně vyplývá $o \in U$
 - lineární kombinace
 - lineární kombinace vektorů $v_1, \dots, v_k \in V$ nad $\mathbb{K}$ je libovolný vektor $u = \alpha_1 v_1 + \dots + \alpha_k v_k$, kde $\alpha_1, \dots, \alpha_k \in \mathbb{K}$

- lineárně nezávislé vektory
 - množina vektorů X je lineárně nezávislá, pokud nulový vektor nelze získat netriviální lineární kombinací vektorů z X ; v ostatních případech je množina X lineárně závislá
 - vektory $v_1, \dots, v_n$ jsou lineárně nezávislé $\equiv \sum_{i=1}^n \alpha_i v_i = 0 \iff \alpha_1 = \dots = \alpha_n = 0$
- lineární obal (podprostor generovaný množinou)
 - lineární obal $\mathcal{L}(X)$ podmnožiny X vektorového prostoru V je průnik všech podprostorů U z V , které obsahují X
 - alternativní značení: $\text{span}(X)$
 - pro $X \subseteq V$ platí $\text{span}(X) = \bigcap U : U \in \mathcal{U}, X \subseteq U$
 - jde o podprostor generovaný X , vektory v množině X se označují jako generátory podprostoru
- věta: průnik podprostorů je taky podprostor
- věta: $\mathcal{L}(X)$ je množina všech lineárních kombinací vektorů z X
- Steinitzova věta o výměně, báze, dimenze, souřadnice
 - lemma o výměně
 - buď $y_1, \dots, y_n$ systém generátorů vektorového prostoru V
 - nechť vektor $x \in V$ má vyjádření $x = \sum_{i=1}^n \alpha_i y_i$
 - pak pro libovolné k takové, že $\alpha_k \neq 0$, je $y_1, \dots, y_{k-1}, x, y_{k+1}, \dots, y_n$ systém generátorů prostoru V
 - Steinitzova věta o výměně
 - buď V vektorový prostor
 - buď $x_1, \dots, x_m$ lineárně nezávislý systém ve V
 - nechť $y_1, \dots, y_n$ je systém generátorů V
 - pak platí $m \leq n$ a existují navzájem různé indexy $k_1, \dots, k_{n-m}$ takové, že $x_1, \dots, x_m, y_{k_1}, \dots, y_{k_{n-m}}$ tvoří systém generátorů V
 - báze vektorového prostoru
 - báze vektorového prostoru V je lineárně nezávislá množina X , která generuje V (tedy $\text{span}(X) = V$)
 - dimenze vektorového prostoru
 - dimenze konečně generovaného vektorového prostoru V je mohutnost (kardinalita / počet prvků) kterékoli z jeho bází; značí se $\dim(V)$
 - vektor souřadnic
 - nechť $X = (v_1, \dots, v_n)$ je uspořádaná báze vektorového prostoru V nad $\mathbb{K}$, potom vektor souřadnic $u \in V$ vzhledem k bázi X je $[u]_X = (\alpha_1, \dots, \alpha_n)^T \in \mathbb{K}^n$, kde $u = \sum_{i=1}^n \alpha_i v_i$
- Vektorové podprostory, zejména maticové (řádkový, sloupcový, jádro) a jejich dimenze
 - řádkový a sloupcový prostor matice $A \in \mathbb{K}^{m \times n}$
 - sloupcový prostor $\mathcal{S}(A) \subseteq \mathbb{K}^m$ je lineární obal sloupců A
 - řádkový prostor $\mathcal{R}(A) \subseteq \mathbb{K}^n$ je lineární obal řádků A
 - $\mathcal{S}(A) = \{u \in \mathbb{K}^m : u = Ax, x \in \mathbb{K}^n\}$
 - $\mathcal{R}(A) = \{v \in \mathbb{K}^n : v = A^T y, y \in \mathbb{K}^m\}$
 - jádro matice $A \in \mathbb{K}^{m \times n}$
 - $\ker(A) = \{x \in \mathbb{K}^n : Ax = 0\}$
 - věta: $\forall A \in \mathbb{K}^{m \times n} : \dim(\ker(A)) + \text{rank}(A) = n$
 - pozorování: dimenze řádkového a sloupcového prostoru se shodují (a rovnají se hodnoti)
- Lineární zobrazení - definice, maticová reprezentace lineárního zobrazení, matice složeného zobrazení
 - lineární zobrazení

- nechť U a V jsou vektorové prostory nad stejným tělesem $\mathbb{K}$
- zobrazení $f : U \rightarrow V$ nazveme lineární, pokud splňuje $\forall u, v \in U, \forall \alpha \in \mathbb{K} :$
 - * $f(u + v) = f(u) + f(v)$
 - * $f(\alpha \cdot u) = \alpha \cdot f(u)$
 - z toho vyplývá, že pro lineární zobrazení obecně platí $f(o) = o$
- věta: lineární zobrazení je jednoznačně definováno tím, kam zobrazí vektory báze (proto můžeme definovat matici lineárního zobrazení)
- matice lineárního zobrazení
 - nechť U a V jsou vektorové prostory nad stejným tělesem $\mathbb{K}$ s bázemi $X = (u_1, \dots, u_n)$ a $Y = (v_1, \dots, v_m)$
 - matice lineárního zobrazení $f : U \rightarrow V$ vzhledem k bázím X a Y je $[f]_{X,Y} \in \mathbb{K}^{m \times n}$, jejíž sloupce jsou vektory souřadnic obrazů vektorů báze X vzhledem k bázi Y , tedy $[f(u_1)]_Y, \dots, [f(u_n)]_Y$
 - pro $w \in U$ tedy platí, že $[f(w)]_Y = [f]_{X,Y}[w]_X$
- matice složeného zobrazení
 - mějme vektorové prostory U, V, W s konečnými uspořádanými bázemi B, C, D
 - pro matice lineárních zobrazení $f : U \rightarrow V$ a $g : V \rightarrow W$ platí vztah $[g \circ f]_{B,D} = [g]_{C,D}[f]_{B,C}$
 - je to hezčí, když použijeme Hladíkovovo značení: $D[g \circ f]_B = D[g]_C \cdot C[f]_B$
- matice přechodu
 - nechť X a Y jsou dvě konečné báze vektorového prostoru U
 - matice přechodu od X k Y je $[id]_{X,Y}$
 - pro $u \in U$ tedy platí, že $[u]_Y = [id(u)]_Y = [id]_{X,Y}[u]_X$
 - matice přechodu je regulární, platí $[id]_{Y,X} = ([id]_{X,Y})^{-1}$
 - výpočet: $[id]_{X,Y} = Y^{-1}X$ nebo také $(Y|X) \rightsquigarrow (I_n|[id]_{X,Y})$
- Obraz a jádro lineárních zobrazení
 - nechť U a V jsou vektorové prostory nad stejným tělesem $\mathbb{K}$
 - jádro lineárního zobrazení
 - $\ker(f) = \{w \in U : f(w) = o\}$
 - vektory v prostoru U , které se zobrazí na nulový vektor v prostoru V
 - obraz $f(U)$ podprostoru U je podprostorem V
 - pokud máme matici zobrazení, tak obraz získáme jako lineární obal těch sloupců, které odpovídají bázičným proměnným (při Gaussovcé nám tam vyjdou pivoty)
- Isomorfismus prostorů
 - vektorové prostory jsou isomorfní, pokud mezi nimi existuje isomorfismus, tedy bijektivní (vzájemně jednoznačné) lineární zobrazení
 - pro isomorfismus f platí, že existuje f^{-1} a je také isomorfismem
 - isomorfní prostory mají shodné dimenze
 - věta: f je isomorfismus, právě když $[f]_{X,Y}$ je regulární
- Skalární součin, norma indukovaná skalárním součinem
 - skalární součin pro vektorové prostory nad komplexními čísly
 - skalární součin na vektorovém prostoru V nad $\mathbb{C}$ je zobrazení, které přiřadí každé dvojici vektorů $u, v \in V$ skalár $\langle u|v \rangle \in \mathbb{C}$ tak, že jsou splněny následující axiomy:
 - * $\forall u \in V : \langle u|u \rangle \in \mathbb{R}_0^+$ (reálné číslo ≥ 0)
 - * $\forall u \in V : \langle u|u \rangle = 0 \iff u = 0$
 - * $\forall u, v \in V : \langle v|u \rangle = \overline{\langle u|v \rangle}$ (komplexně sdružené)
 - * $\forall u, v, w \in V : \langle u + v|w \rangle = \langle u|w \rangle + \langle v|w \rangle$
 - * $\forall u, v \in V, \forall \alpha \in \mathbb{C} : \langle \alpha u|v \rangle = \alpha \langle u|v \rangle$

- formálně je každý skalární součin zobrazení $V \times V \rightarrow \mathbb{C}$
- skalární součin na V nad $\mathbb{R}$ je zobrazení $V \times V \rightarrow \mathbb{R}$, přičemž $\langle v|u \rangle = \langle u|v \rangle$ (ve třetím axiomu), $\alpha \in \mathbb{R}$ (v posledním axiomu)
- standardní skalární součin na $\mathbb{R}^n$: $\langle u|v \rangle = v^T u$
- standardní skalární součin na $\mathbb{C}^n$: $\langle u|v \rangle = v^H u$
- norma spojená se skalárním součinem
 - je-li V prostor se skalárním součinem, pak norma odvozená ze skalárního součinu je zobrazení $V \rightarrow \mathbb{R}_0^+$ přiřazující vektoru u jeho normu $\|u\| = \sqrt{\langle u|u \rangle}$
 - geometrická interpretace v euklidovském prostoru $\mathbb{R}^n$
 - * $\|u\|$... délka (velikost) u
 - * $\|u - v\|$... vzdálenost bodů u, v
 - * $\langle u|v \rangle$ souvisí s „úhlem“ φ mezi u, v a délkami u, v
 - $\langle u|v \rangle = \|u\| \cdot \|v\| \cos \varphi$
 - to vyplývá z kosinové věty, kde $a = \|u\|$, $b = \|v\|$, $c = \|u - v\|$
- kolmé vektory
 - vektory u, v z prostoru se skalárním součinem jsou kolmé (ortogonální), pokud $\langle u|v \rangle = 0$
 - kolmé vektory značíme $u \perp v$
- Pythagorova věta, Cauchyho-Schwarzova nerovnost, trojúhelníková nerovnost
 - Pythagoras: $\|a\|^2 + \|b\|^2 = \|c\|^2$
 - pokud a, b jsou na sebe kolmé a $c = b - a$
 - ovšem taky platí $\|a + b\|^2 = \|a\|^2 + \|b\|^2$, rovněž pro kolmé a, b
 - Cauchy-Schwarz: $|\langle u|v \rangle| \leq \sqrt{\langle u|u \rangle \langle v|v \rangle}$
 - trojúhelníková nerovnost: $\|u + v\| \leq \|u\| + \|v\|$
- Ortonormální systémy vektorů, Fourierovy koeficienty, Gramova-Schmidtova ortogonalizace
 - ortonormální báze
 - bázi $Z = \{v_1, \dots, v_n\}$ prostoru V se skalárním součinem nazveme ortonormální, pokud platí $v_i \perp v_j$ pro každé $i \neq j$ a také $\|v_i\| = 1$ pro každé $v_i \in Z$
 - pozorování: matice, jejichž sloupce tvoří vektory ortonormální báze $\mathbb{C}^n$ vzhledem ke std. skal. součinu, splňují $A^H A = I_n \implies$ jsou unitární
 - Fourierovy koeficienty
 - necht $Z = \{v_1, \dots, v_n\}$ je ortonormální báze prostoru V
 - tvrzení: pro každé $u \in V$ platí $u = \langle u|v_1 \rangle v_1 + \dots + \langle u|v_n \rangle v_n$
 - $\langle u|v_i \rangle$... Fourierovy koeficienty
 - algoritmus GS ortogonalizace
 - převede libovolnou bázi $(u_1, \dots, u_n)$ prostoru V se skalárním součinem na ortonormální bázi $(v_1, \dots, v_n)$
 - for $i = 1, \dots, n$ do
 - * $w_i \leftarrow u_i - \sum_{j=1}^{i-1} \langle u_i|v_j \rangle v_j$
 - * $v_i \leftarrow \frac{1}{\|w_i\|} w_i$
 - end
- Ortogonální doplněk, ortogonální projekce, projekce jako lineární zobrazení
 - ortogonální doplněk
 - ortogonální doplněk podmnožiny V prostoru se skalárním součinem W je $V^\perp = \{u \in W : (\forall v \in V)(u \perp v)\}$
 - ortogonální projekce
 - necht W je prostor se skalárním součinem a V je jeho podprostor s ortonormální báží

$$Z = (v_1, \dots, v_n)$$

· zobrazení $p_Z : W \rightarrow V$ definované $p_Z(u) = \sum_{i=1}^n \langle u | v_i \rangle v_i$ je ortogonální (kolmá) projekce W na V

- pozorování: ortogonální projekce je lineární zobrazení

• Ortogonální matice a jejich vlastnosti

- matice $Q \in \mathbb{R}^{n \times n}$ je ortogonální, pokud $Q^T Q = I_n$

· pro ortogonální matici Q platí $Q^T = Q^{-1}$

- věta: Q je ortogonální, právě když sloupce tvoří ortonormální bázi $\mathbb{R}^n$

- zjevně: je-li Q ortogonální, pak...

· Q^T je rovněž ortogonální

· Q^{-1} existuje a je ortogonální

- součin ortogonálních matic je ortogonální matice

- další vlastnosti

· $\langle Qx | Qy \rangle = \langle x | y \rangle$

· $\|Qx\| = \|x\|$

· zobrazení $x \mapsto Qx$ zachovává úhly a délky

· platí to i naopak - matice zobrazení zachovávajícího skalární součin je ortogonální

· $\forall i, j : |Q_{ij}| \leq 1$

· $\begin{pmatrix} 1 & o^T \\ o & Q \end{pmatrix}$ je ortogonální matice

• Definice a základní vlastnosti determinantu (multiplikativnost, determinant transponované matice, vztah s regularitou a vlastními čísly)

- determinant matice $A \in \mathbb{K}^{n \times n}$ je dán výrazem

$$\det A = \sum_{p \in S_n} \text{sgn}(p) \prod_{i=1}^n a_{i,p(i)}$$

- věta: $\forall A, B \in \mathbb{K}^{n \times n} : \det(AB) = \det A \cdot \det B$

- transpozice nemění determinant, $\det A = \det A^T$

- matice je regulární, právě když má nenulový determinant

- pro regulární matici A platí $\det(A^{-1}) = \det(A)^{-1}$

- výpočet determinantu je nutné nastudovat

Procvičit (poznámky odkazují na vlastní trénink)

Vlastní výpočet determinantu (rozvoj, úprava na trojúhelníkový tvar) si natrénuj na příkladech.

- $\det(A) = \lambda_1 \cdot \lambda_2 \cdot \dots \cdot \lambda_n$

- λ je vlastním číslem A , právě když $\det(A - \lambda I_n) = 0$

· tedy právě když je kořenem charakteristického polynomu $p_A(t) = \det(A - tI_n)$

• Laplaceův rozvoj determinantu

- notace: A^{ij} je podmatice získaná z A odstraněním i -tého řádku a j -tého sloupce

- věta: pro libovolné $A \in \mathbb{K}^{n \times n}$ a jakékoli $i \in \{1, \dots, n\}$ platí

$$\det A = \sum_{j=1}^n a_{ij} (-1)^{i+j} \det A^{ij}$$

• Geometrická interpretace determinantu

- objem rovnoběžnostěny určeného k vektory je roven absolutní hodnotě determinantu matice, jejíž sloupce tyto vektory tvoří

- po provedení lineárního zobrazení se objemy těles změny úměrně absolutní hodnotě determinantu matice zobrazení
- Definice, geometrický význam a základní vlastnosti vlastních čísel, charakteristický polynom, násobnost vlastních čísel
 - definice
 - mějme matici $A \in \mathbb{C}^{n \times n}$
 - vlastní číslo matice A je jakékoliv $\lambda \in \mathbb{C}$, pro které existuje vektor $x \in \mathbb{C}^n \setminus \{0\}$ takový, že $Ax = \lambda x$
 - x je pak vlastní vektor příslušný vlastnímu číslu λ
 - geometrie
 - vlastní vektor reprezentuje invariantní směr při zobrazení $x \mapsto Ax$, tedy směr, který se zobrazí opět na ten samý směr
 - vlastní číslo odpovídá škálování v tomto invariantním směru
 - základní vlastnosti
 - determinant matice je roven součinu vlastních čísel
 - stopa matice (součet diagonály) je rovna součtu vlastních čísel
 - matice je regulární, právě když nula není její vlastní číslo
 - A^T má stejná vlastní čísla jako A , ale vlastní vektory obecně jiné
 - uvažujeme matici $A \in \mathbb{C}^{n \times n}$ s vlastními čísly $\lambda_1, \dots, \lambda_n$ a jim odpovídajícími vlastními vektory $x_1, \dots, x_n$
 - * je-li A regulární, pak A^{-1} má vlastní čísla $\lambda_1^{-1}, \dots, \lambda_n^{-1}$ a vlastní vektory $x_1, \dots, x_n$
 - * A^2 má vlastní čísla $\lambda_1^2, \dots, \lambda_n^2$ a vlastní vektory $x_1, \dots, x_n$
 - * podobně pro αA a taky pro $A + \alpha I_n$
 - charakteristický polynom ... $p_A(t) = \det(A - tI_n)$
 - vlastní čísla matice A jsou právě kořeny jejího charakteristického polynomu a je jich n včetně násobností
 - algebraická násobnost λ je rovna násobnosti λ jakožto kořene p_A
 - geometrická násobnost λ je rovna $n - \text{rank}(A - \lambda I_n)$, tj. počtu lineárně nezávislých vlastních vektorů, které odpovídají λ
 - algebraická násobnost $\geq$ geometrická násobnost
 - obvykle se násobností myslí ta algebraická
- Podobnost a diagonalizovatelnost matic, spektrální rozklad
 - matice $A, B \in \mathbb{C}^{n \times n}$ jsou podobné, pokud existuje regulární $S \in \mathbb{C}^{n \times n}$ tak, že $A = SBS^{-1}$
 - věta: podobné matice mají stejná vlastní čísla
 - počet vlastních vektorů pro konkrétní vlastní číslo je stejný u obou matic
 - matice $A \in \mathbb{C}^{n \times n}$ je diagonalizovatelná, je-li podobná nějaké diagonální matici
 - diagonalizovatelná matice A jde tedy vyjádřit ve tvaru $A = S\Lambda S^{-1}$, kde S je regulární a Λ diagonální
 - tomuto se říká spektrální rozklad
 - sloupce matice S odpovídají vlastním vektorům (v pořadí, v němž jsou vlastní čísla na diagonále Λ)
 - věta: matice $A \in \mathbb{C}^{n \times n}$ je diagonalizovatelná, právě když má n lineárně nezávislých vlastních vektorů
 - věta: pokud má matice n navzájem různých vlastních čísel, pak je diagonalizovatelná
 - $A^2 = S\Lambda^2 S^{-1}$
- Symetrické matice, jejich vlastní čísla a spektrální rozklad
 - matice je symetrická, pokud $A = A^T$

- vlastní čísla reálných symetrických matic jsou reálná
- věta: pro každou symetrickou matici $A \in \mathbb{R}^{n \times n}$ existuje ortogonální $Q \in \mathbb{R}^{n \times n}$ a diagonální $\Lambda \in \mathbb{R}^{n \times n}$ takové, že $A = Q\Lambda Q^T$
- Positivně semidefinitní a pozitivně definitní matice - charakterizace a vlastnosti, vztah se skalárním součinem, vlastními čísly
 - definice
 - buď $A \in \mathbb{R}^{n \times n}$ symetrická
 - pak A je pozitivně semidefinitní, pokud $x^T A x \geq 0$ pro všechna $x \in \mathbb{R}^n$
 - A je pozitivně definitní, pokud $x^T A x > 0$ pro všechna $x \neq 0$
 - poznámka: nesymetrické matice můžeme symetrizovat úpravou $\frac{1}{2}(A + A^T)$
 - následující tři vlastnosti jsou ekvivalentní
 - A je pozitivně semidefinitní (nebo definitní)
 - vlastní čísla A jsou nezáporná
 - * pro definitní musí být kladná
 - existuje matice $U \in \mathbb{R}^{m \times n}$ taková, že $A = U^T U$
 - * pro definitní musí mít matice U hodnotu n
 - vlastnosti
 - jsou-li A, B pozitivně definitní, pak i $A + B$ je p. d.
 - je-li A pozitivně definitní a $\alpha > 0$, pak i αA je p. d.
 - je-li A pozitivně definitní, pak je regulární a A^{-1} je p. d.
 - operace $\langle x|y \rangle$ je skalárním součinem v $\mathbb{R}^n$, právě když má tvar $\langle x|y \rangle = x^T A y$ pro nějakou pozitivně definitní matici $A \in \mathbb{R}^{n \times n}$
 - testování p. d. pomocí Gaussovy eliminace
 - používáme pouze úpravu přičtení násobku řádku s pivotem k jinému řádku pod ním
 - pokud nám vyjde odstupňovaný tvar s kladnou diagonálou, byla původní matice pozitivně definitní
- Choleského rozklad (znění věty a praktické použití)
 - pro každou pozitivně definitní matici $A \in \mathbb{R}^{n \times n}$ existuje jediná dolní trojúhelníková matice $L \in \mathbb{R}^{n \times n}$ s kladnou diagonálou taková, že $A = LL^T$
 - algoritmus vypadá složitě, ale dá se to vymyslet intuitivně (je vhodné začít tím, že první prvek matice A je druhou mocninou čísla, které je v obou trojúhelníkových maticích vlevo nahoře)
 - praktické použití
 - řešíme soustavu $Ax = b$
 - máme Choleského rozklad $A = LL^T$
 - snadno najdeme řešení soustavy $Ly = b$ (díky nulám v trojúhelníkové matici stačí substitovat)
 - pak najdeme řešení soustavy $L^T x = y$
 - proč to funguje?
 - * $L(\underbrace{L^T x}_{=y}) = b$

4.3 3. Diskrétní matematika

- Relace, vlastnosti binárních relací (reflexivita, symetrie, antisymetrie, tranzitivita)
 - (binární) relace mezi množinami X, Y je podmnožina $X \times Y$
 - relace na množině X je podmnožina X^2

- značení - pro relaci R mezi $X, Y : xRy \equiv (x, y) \in R$
- příklady relací
 - prázdná $\emptyset$
 - univerzální $X \times Y$
 - diagonální $\Delta_X := \{(x, x) \mid x \in X\}$, např. rovnost $x = y$
- operace s relacemi
 - inverze
 - * k relaci R mezi X, Y lze definovat inverzní relaci R^{-1} mezi Y, X , přičemž $R^{-1} := \{(y, x) \mid (x, y) \in R\}$
 - skládání
 - * pro relaci R mezi X, Y a relaci S mezi Y, Z lze definovat složenou relaci $T = R \circ S$ mezi X, Z
 - * $xTz \equiv \exists y \in Y : xRy \wedge ySz$
 - * $R \circ \Delta_Y = R, \quad \Delta_X \circ R = R$
 - * značení skládání funkcí: $(f \circ g)(x) = g(f(x))$
- relace R na X je...
 - reflexivní $\equiv \forall x \in X : xRx$
 - * $\Delta_X \subseteq R$
 - symetrická $\equiv \forall x, y \in X : xRy \implies yRx$
 - * $R = R^{-1}$
 - antisymetrická $\equiv \forall x, y \in X : xRy \wedge yRx \implies x = y$
 - * $R \cap R^{-1} \subseteq \Delta_X$
 - tranzitivní $\equiv \forall x, y, z \in X : xRy \wedge yRz \implies xRz$
 - * $R \circ R \subseteq R$
- Ekvivalence a rozkladové třídy
 - relace R na X je ekvivalence $\equiv R$ je reflexivní & symetrická & tranzitivní
 - např. rovnost čísel, rovnost mod K , geometrická podobnost
 - ekvivalenční třída prvku $x \in X : R[x] = \{y \in X : xRy\}$
 - množinový systém $\mathcal{S} \subseteq 2^X$ je rozklad množiny $X \equiv$
 - $\forall A \in \mathcal{S} : A \neq \emptyset$
 - $\forall A, B \in \mathcal{S} : A \neq B \implies A \cap B = \emptyset$
 - $\bigcup_{A \in \mathcal{S}} A = X$
 - věta (vztah mezi ekvivalencemi a rozklady)
 1. $\forall x \in X : R[x] \neq \emptyset$
 2. $\forall x, y \in X : \text{buď } R[x] = R[y], \text{ nebo } R[x] \cap R[y] = \emptyset$
 3. $\{R[x] \mid x \in X\}$ (množina všech ekvivalenčních tříd) určuje ekvivalenci R jednoznačně
- Částečná uspořádání - základní pojmy (minimální a maximální prvky, nejmenší a největší prvky, řetězec, antiřetězec)
 - relace R na množině X je (částečné) uspořádání $\equiv R$ je reflexivní & antisymetrická & tranzitivní
 - (částečně) uspořádaná množina (X, R)
 - zkráceně ČUM
 - R je (částečné) uspořádání na X
 - prvky $x, y \in X$ jsou porovnatelné $\equiv xRy \vee yRx$
 - uspořádání je lineární $\equiv \forall x, y \in X$ porovnatelné
 - (všechny prvky množiny jsou navzájem porovnatelné)

- částečné uspořádání (nebo pouze uspořádání) je obecný pojem, některá taková uspořádání jsou navíc lineární
- ostré uspořádání - každému uspořádání $\leq$ na X přiřadíme relaci $<$ na X : $a < b \equiv a \leq b \wedge a \neq b$
 - pozor - ostré uspořádání není speciálním případem uspořádání (protože není reflexivní)
 - vlastnosti ostrého uspořádání - ireflexivní, antisymetrické, tranzitivní
- Hasseův diagram graficky zachycuje vztahy mezi prvky ČUM (porovnatelné prvky jsou spojeny, větší prvky jsou výše)
- prvek $x \in X$ je nejmenší $\equiv \forall y \in X : x \leq y$
- prvek $x \in X$ je minimální $\equiv \nexists y \in X : y < x$
- x je nejmenší $\implies x$ je minimální
- v Hassově diagramu
 - z minimálního prvku dolů nevede žádná spojnice
 - nejmenší prvek je nejnižší v diagramu, existuje do něj cesta z libovolného jiného prvku
- věta: konečná neprázdná uspořádaná množina má minimální a maximální prvek
- pro $(X, \leq)$ ČUM:
 - $A \subseteq X$ je řetězec $\equiv \forall a, b \in A : a, b$ jsou porovnatelné
 - $A \subseteq X$ je antiřetězec (nezávislá množina) $\equiv \nexists a, b$ různé & porovnatelné
- Výška a šířka částečně uspořádané množiny a věta o jejich vztahu (o dlouhém a širokém)
 - $\omega(X, \leq)$ je délka nejdelšího řetězce = maximum z délek řetězců (výška uspořádání)
 - $\alpha(X, \leq)$ je délka nejdelšího antiřetězce (šířka uspořádání)
 - věta: pro každou konečnou ČUM $(X, \leq)$ platí $\alpha(X, \leq) \cdot \omega(X, \leq) \geq |X|$
 - důsledek: $\max(\alpha, \omega) \geq \sqrt{|X|}$
- Funkce, typy funkcí (prostá, na, bijekce)
 - funkce z množiny X do množiny Y je relace A mezi X a Y t. ž. $(\forall x \in X)(\exists! y \in Y) : xAy$
 - funkce $f : X \rightarrow Y$ je...
 - prostá (injektivní) $\equiv \nexists x, x' \in X : x \neq x' \wedge f(x) = f(x')$
 - na Y (surjektivní) $\equiv (\forall y \in Y)(\exists x \in X) : f(x) = y$
 - vzájemně jednoznačná (bijektivní) $\equiv (\forall y \in Y)(\exists! x \in X) : f(x) = y$
 - * taková funkce je tedy prostá i „na“
 - * k takové funkci existuje inverzní funkce f^{-1} z Y do X
- Počty různých typů funkcí mezi dvěma konečnými množinami
 - věta: počet $f : N \rightarrow M = m^n$
 - pro $|N| = n, |M| = m; \quad m, n > 0$
 - definice: klesající mocnina
 - $m^{\underline{n}} = \underbrace{m \cdot (m-1) \cdot (m-2) \cdot \dots \cdot (m-n+1)}_n$
 - tedy $m^{\underline{n}} = \frac{m!}{(m-n)!}$
 - věta: počet prostých $f : N \rightarrow M = m^{\underline{n}}$
 - počet surjektivních zobrazení lze určit pomocí principu inkluze a exkluze
 - bijekcí je $n!$
 - počet uspořádaných k -tic $|X^k| = |X|^k$, lze jej totiž vyjádřit jako počet funkcí $f : [k] \rightarrow X$
- Permutace a jejich základní vlastnosti (počet a pevný bod)
 - definice: $[n] = \{1, 2, \dots, n\}$
 - věta: na množině $[n]$ existuje $n!$ permutací (podobně na každé n -prvkové množině)
 - pevný bod permutace je prvek, který se zobrazí sám na sebe
- Kombinační čísla a vztahy mezi nimi, binomická věta a její aplikace

- kombinační číslo (binomický koeficient) $\binom{n}{k} := \frac{n!}{k!(n-k)!}$
- Pascalův trojúhelník - n roste shora dolů, k zleva doprava
- $\binom{X}{k}$... množina všech k -prvkových podmnožin množiny X
- $\binom{n}{k} = \binom{n}{n-k}$... každé k -prvkové podmnožině přiřadíme její doplněk
 - Pascalův trojúhelník je symetrický
- $\binom{n-1}{k-1} + \binom{n-1}{k} = \binom{n}{k}$
 - zvolíme jeden prvek a a rozdělíme všechny k -prvkové podmnožiny podle toho, zda obsahují a , nebo ne
 - buňka v Pascalově trojúhelníku je součtem dvou buněk nad ní (pokud není na kraji)
- Binomická věta: $(x+y)^n = \sum_{i=0}^n \binom{n}{i} x^{n-i} y^i$
- aplikace
 - řádek Pascalova trojúhelníku se sečte na $2^n = (1+1)^n = \sum_{k=0}^n \binom{n}{k}$
 - důkaz $(x^n)' = nx^{n-1}$
 - * použijeme $f(x+h) = (x+h)^n = x^n + nx^{n-1}h + \binom{n}{2}x^{n-2}h^2 + \dots + h^n$
 - * dosadíme do $\lim_{h \rightarrow 0} \frac{f(x+h)-f(x)}{h}$
 - $e = \lim (1 + \frac{1}{n})^n$
- Princip inkluze a exkluze: obecná formulace (a důkaz)
 - konkrétní ukázky principu inkluze a exkluze (PIE)
 - $|A \cup B| = |A| + |B| - |A \cap B|$
 - $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$
 - věta: pro konečné množiny $A_1, \dots, A_n$ platí

$$|\bigcup_{i=1}^n A_i| = \sum_{k=1}^n (-1)^{k+1} \sum_{I \in \binom{[n]}{k}} |\bigcap_{i \in I} A_i|$$

- důkaz
 - pro prvek x ve sjednocení spočítáme příspěvky k levé (vždy 1) a pravé straně
 - nechť x patří do právě t množin
 - průniky k -tic
 - * $k > t$... přispěje 0
 - * $k \leq t$... přispěje $(-1)^{k+1} \binom{t}{k}$
 - vybíráme k -tice množin z t -množin, do kterých prvek patří
 - minus jednička vychází ze vzorce
 - * chceme $\sum_{k=1}^t (-1)^{k+1} \binom{t}{k} = 1$
 - * lze upravit na $\sum_{k=1}^t (-1)^k \binom{t}{k} = -1$
 - * z binomické věty $0 = (1-1)^t = \sum_{k=0}^t \binom{t}{k} (-1)^k$
 - * tedy bez prvního členu se součet rovná -1 $\square$
- Princip inkluze a exkluze: použití (problém šatnářky, Eulerova funkce pro počet dělitelů, počet surjekcí)
 - problém šatnářky
 - Šatnářka n pánům vydá náhodně n klobouků (které si předtím odložili v šatně). Jaká je pravděpodobnost, že žádný pán nedostane od šatnářky zpět svůj klobouk?
 - jaká je pravděpodobnost, že náhodně zvolená permutace nebude mít žádný pevný bod
 - * každá z $n!$ permutací je stejně pravděpodobná
 - * $\check{s}(n)$... počet permutací bez pevného bodu
 - * pravděpodobnost je rovna $\check{s}(n)/n!$
 - S_n ... množina všech permutací

- $A_i = \{\pi \in S_n : \pi(i) = i\}$
 - * množina permutací s pevným bodem v i -tém prvku (jedna permutace může patřit do více takových množin)
- $A = \bigcup_{i=1}^n A_i \dots$ (množina všech „špatných“ permutací)
- musíme vyjádřit velikosti průniků
 - * permutací s (konkrétními) k pevnými body je $(n-k)!$, protože permutují všechny prvky kromě těch pevných
- dosazením do principu inkluze a exkluze vyjde $|A| = \sum_{k=1}^n (-1)^{k+1} \binom{n}{k} (n-k)!$
 - * $\binom{n}{k}$ vyplývá z počtu prvků druhé sumy
 - * $\binom{n}{k} (n-k)! = \frac{n!}{k!}$
- $|A| = \sum_{k=1}^n (-1)^{k+1} \frac{n!}{k!} = n! \cdot \sum_{k=1}^n \frac{(-1)^{k+1}}{k!}$
- $|A| = n! \left(\frac{1}{1!} - \frac{1}{2!} + \frac{1}{3!} - \dots + \frac{(-1)^{n+1}}{n!} \right)$
- $\check{s}(n) = n! - |A| = n! \cdot \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^n}{n!} \right)$
 - * závorka konverguje k e^{-1}
 - * závorka odpovídá pravděpodobnosti v problému šatnářky
- $\check{s}(n) = n! \cdot \sum_{k=0}^n \frac{(-1)^k}{k!}$
- pravděpodobnost $\dots \sum_{k=0}^n \frac{(-1)^k}{k!} \approx e^{-1}$
- Eulerova funkce $\varphi(n)$
 - je to počet čísel z $[n]$ nesoudělných s n
 - * pro prvočíslo p platí $\varphi(p) = p - 1$, protože je soudělné samo se sebou a protože podle definice nesoudělnosti ($\text{NSD} = 1$) není soudělné s jedničkou
 - tvrzení: necht $n = p_1^{e_1} \cdot p_2^{e_2} \cdot \dots \cdot p_k^{e_k}$ je prvočíselný rozklad, pak $\varphi(n) = n \left(1 - \frac{1}{p_1} \right) \left(1 - \frac{1}{p_2} \right) \dots \left(1 - \frac{1}{p_k} \right)$
 - důkaz
 - * jako A_i označím množinu čísel z $[n]$ soudělných i -tým prvočíslem p_i
 - * zjevně $\varphi(n) = n - |\bigcup_{i=1}^k A_i|$
 - * $|A_i| = \frac{n}{p_i}$
 - * pro různé $i, j \in [k]$ platí $|A_i \cap A_j| = \frac{n}{p_i p_j}$
 - * to se dá zobecnit na průnik více množin
 - * dle PIE $|\bigcup_{i=1}^k A_i| = \sum_{I \in 2^{[k]} \setminus \{\emptyset\}} (-1)^{|I|+1} \frac{n}{\prod_{i \in I} p_i}$
 - * $= n \left(\frac{1}{p_1} + \dots + \frac{1}{p_k} - \frac{1}{p_1 p_2} - \dots + \frac{1}{p_1 p_2 p_3} + \dots + (-1)^{k+1} \frac{1}{p_1 \dots p_k} \right)$
 - * proto $\varphi(n) = n \left(1 - \frac{1}{p_1} - \dots - \frac{1}{p_k} + \frac{1}{p_1 p_2} + \dots + (-1)^k \frac{1}{p_1 \dots p_k} \right)$
 - * tedy $\varphi(n) = n \left(1 - \frac{1}{p_1} \right) \left(1 - \frac{1}{p_2} \right) \dots \left(1 - \frac{1}{p_k} \right)$ $\square$
 - z každé závorky vždy vybíráme levý nebo pravý člen
 - počet surjekcí
 - věta
 - * necht X, Y jsou konečné množiny, $|X| = n$, $|Y| = m$, $n \geq m > 0$
 - * potom počet zobrazení f množiny X na množinu Y je $\sum_{k=0}^m (-1)^k \binom{m}{k} (m-k)^n$
 - důkaz
 - * počet všech funkcí z X do Y je m^n
 - * definujeme F_i jako množinu zobrazení, že i -tý prvek z množiny Y není pokrytý (neexistuje $x \in X$, aby $f(x) = y_i$, kde y_i je i -tý prvek z Y)
 - * takže počet surjekcí bude $m^n - |\bigcup_{i=1}^m F_i|$
 - * použijeme PIE: $|\bigcup_{i=1}^m F_i| = \sum_{k=1}^m (-1)^{k+1} \sum_{I \in \binom{[m]}{k}} |\bigcap_{i \in I} F_i|$
 - * $= \sum_{k=1}^m (-1)^{k+1} \binom{m}{k} (m-k)^n$

- $(m - k)^n$ dostaneme tak, že uvažujeme funkce z n -prvkové množiny do množiny s $m - k$ prvky (na vybraných k bodů naše funkce nesmí směřovat)
- * počet surjekcí $m^n - |\bigcup_{i=1}^m F_i| = m^n - \sum_{k=1}^m (-1)^{k+1} \binom{m}{k} (m - k)^n$ pak ještě upravíme do finálního tvaru $\square$
- Hallova věta o systému různých reprezentantů a její vztah k párování v bipartitním grafu, princip důkazu a algoritmické aspekty (polynomiální algoritmus pro nalezení SRR)
 - párování je taková množina hran, kde žádné dvě hrany nemají společný vrchol
 - maximální párování je takové párování, které má nejvíce hran
 - perfektní párování je takové párování, které pokrývá všechny vrcholy grafu
 - systém různých reprezentantů v hypergrafu
 - systém různých reprezentantů (SRR) v hypergrafu $H = (V, E)$ je funkce $r : E \rightarrow V$ taková, že
 - * $\forall e \in E : r(e) \in e$
 - * $\forall e, f \in E : e \neq f \implies r(e) \neq r(f) \dots$ tj. r je prostá
 - $r(e) \dots$ „reprezentant hyperhrany e “
 - analogie s předsedy spolků
 - pozorování: v bipartitním grafu s partitami A, B má každé párování velikost nejvýše $|A|$ (i nejvýše $|B|$)
 - definice: pro graf $G = (V, E)$ a množinu $X \subseteq V$ označím $N(X) := \{y \in V \setminus X : (\exists x \in X)\{x, y\} \in E\}$
 - Hallova věta, bipartitní grafová verze
 - nechť G je bipartitní graf s partitami A, B , potom G má párování velikosti $|A| \iff \underbrace{\forall X \subseteq A : |N(X)| \geq |X|}_{\text{„Hallová podmínka“}}$
 - důkaz
 - $\implies$
 - * pokud existuje párování velikosti $|A|$, tak pro každou $X \subseteq A$ existuje $|X|$ vrcholů spárovaných s X , ty patří do $N(X)$, tedy $|N(X)| \geq |X|$
 - $\impliedby$
 - * nechť M je největší párování v G
 - * pro spor nechť $|M| < |A|$
 - * dle Königovy-Egerváryho věty existuje pokrytí C , kde $|C| = |M| < |A|$
 - * $C_A := C \cap A$
 - * $C_B := C \cap B$
 - * $X := A \setminus C_A$
 - * zjistím, že $N(X) \subseteq C_B$
 - protože nepokryté hrany musí pokrývat vrcholy v $N(X)$
 - * navíc $|X| = |A| - |C_A| > |C_B| \geq |N(X)|$
 - jelikož $|C| < |A| \implies |A| > |C_A| + |C_B|$
 - * to je spor s Hallovou podmínkou
 - idea $\impliedby$
 - * uvažujeme ostře menší párování, tedy podle Königovy-Egerváryho věty musí existovat i menší pokrytí
 - věta říká, že velikost maximálního párování se rovná velikosti minimálního pokrytí (jako u toku a řezu)
 - * prohlásíme, že vrcholy v X nejsou v pokrytí, tak ty hrany mezi nimi a $N(X)$ musí pokrývat vrcholy v $N(X)$, ale těch by pak bylo málo (méně než požaduje Hallova

podmínka)

* takže jsme dokázali obměnu implikace

- pozorování: jakmile uvnitř hypergrafu je množina čtyř hyperhran, které ve svém sjednocení mají tři vrcholy, tak nemůžu najít systém různých reprezentantů
 - platí to obecně pro množinu k hyperhran - když budou mít ve sjednocení méně než k vrcholů, tak nenajdu SRR
 - implikace platí i obráceně, lze vyslovit jako větu
- Hallova věta, hypergrafová verze
 - hypergraf $H = (V, E)$ má SRR, právě když $\forall F \subseteq E : |\bigcup_{e \in F} e| \geq |F|$
- důkaz
 - nechť $H = (V, E)$ je hypergraf, nechť I_H je jeho graf incidence
 - H má SRR $\iff I_H$ má párování velikosti $|E|$
 - Hallova podmínka pro H : $\forall F \subseteq E : |\bigcup_{e \in F} e| \geq |F| \iff$ bipartitní Hallova podmínka pro I_H a partitu E
- algoritmické aspekty
 - maximální párování lze najít v polynomiálním čase
 - proto i SRR lze najít v polynomiálním čase

4.4 4. Teorie grafů

- Základní pojmy teorie grafů - graf, vrcholy a hrany, izomorfismus grafů, podgraf, okolí vrcholu a stupeň vrcholu, doplněk grafu, bipartitní graf
 - graf je (V, E) , kde V je konečná neprázdná množina vrcholů a $E \subseteq \binom{V}{2}$ je množina hran
 - lze značit jako $G = (V, E)$, potom $V(G)$ je množina vrcholů a $E(G)$ je množina hran
 - izomorfismus
 - grafy jsou izomorfní $\equiv$ existuje bijekce, která zachovává vlastnost být spojen hranou
 - v podstatě stačí přejmenovat vrcholy a dostaneme dva stejné grafy
 - značení $\cong$
 - $\cong$ je ekvivalence na libovolné množině grafů
 - * neexistuje množina všech grafů (protože neexistuje množina všech množin)
 - podgraf
 - graf $G' = (V', E')$ je podgrafem grafu $G = (V, E)$ (značíme $G' \subseteq G$), když $V' \subseteq V \wedge E' \subseteq E$
 - graf $G' = (V', E')$ je indukovaným podgrafem grafu $G = (V, E)$, když $V' \subseteq V \wedge E' = E \cap \binom{V'}{2}$
 - * „podgraf indukovaný množinou vrcholů“
 - * $G[A] := (A, E(G) \cap \binom{A}{2})$, kde $A \subseteq V(G)$
 - okolí vrcholu
 - okolí vrcholu v jsou ty vrcholy, které s vrcholem v sousedí (sdílejí hranu)
 - $N_G(v) = \{u \in V(G) : \exists uv \in E(G)\}$
 - stupeň vrcholu
 - stupeň vrcholu - počet hran, kterých se účastní daný vrchol
 - graf je k -regulární, pokud je stupeň všech vrcholů grafu roven k
 - skóre grafu = posloupnost stupňů vrcholů (až na pořadí) $\rightarrow$ jakmile dvěma grafům vyjde jiné skóre, nemohou být izomorfní
 - doplněk grafu
 - doplněk/komplement grafu G je graf G_0 , který má stejné vrcholy jako G , ale má právě ty hrany, které v grafu G chybí

- bipartitní graf
 - graf (V, E) je bipartitní $\equiv \exists L, P \subseteq V$ t. ž.:
 - * $L \cup P = V$
 - * $L \cap P = \emptyset$
 - * $\forall e \in E : |e \cap L| = 1 \quad (\wedge |e \cap P| = 1)$
 - nebo $E(G) \subseteq \{\{x, y\} \mid x \in L, y \in P\}$
 - partity grafu - jednotlivé „strany“
- Základní příklady grafů - úplný graf a úplný bipartitní graf, cesty a kružnice
 - úplný graf K_n
 - $V(K_n) := [n]$
 - $E(K_n) := \binom{V(K_n)}{2}$
 - úplný bipartitní $K_{m,n}$
 - každý prvek nalevo je spojený s každým napravo
 - prvky na jedné straně mezi sebou nejsou spojeny
 - prázdný graf E_n
 - $V(E_n) := [n]$
 - $E(E_n) = \emptyset$
 - cesta P_n
 - $V(P_n) := \{0, \dots, n\}$
 - $E(P_n) := \{\{i, i+1\} \mid 0 \leq i < n\}$
 - délka cesty se měří v počtu hran
 - kružnice/cyklus C_n
 - $n \geq 3$
 - $V(C_n) := \{0, \dots, n-1\}$
 - $E(C_n) := \{\{i, (i+1) \bmod n\} \mid 0 \leq i < n\}$
- Souvislost grafů, komponenty souvislosti, vzdálenost v grafu
 - graf G je souvislý $\equiv \forall u, v \in V(G) : \text{existuje cesta v } G \text{ s krajními vrcholy } u, v$
 - dosažitelnost v G je relace $\sim$ na $V(G)$ t. ž. $u \sim v \equiv \text{existuje cesta v } G \text{ s krajními vrcholy } u, v$
 - relace $\sim$ je ekvivalence
 - tranzitivita se dokazuje pomocí dvou posloupností vrcholů $x \sim y$ a $y \sim z$ a následně zvolení nejzazšího vrcholu z posloupnosti $y \sim z$, který je obsažen v posloupnosti $x \sim y$ a v tomto vrcholu se posloupnosti slepí (přičemž části za ním v první posloupnosti a před ním v druhé posloupnosti se ustříhnou)
 - komponenty souvislosti jsou podgrafy indukované třídami ekvivalence $\sim$
 - komponenty jsou souvislé
 - graf je souvislý $\iff$ má 1 komponentu
 - vzdálenost (grafová metrika) v souvislém grafu G
 - $d_G : V^2 \rightarrow \mathbb{R}$
 - $d_G(u, v) := \min. \text{ z délek (počtu hran) všech cest mezi } u, v$
 - vlastnosti metriky (funkce je metrika = chová se jako vzdálenost)
 - $d_G(u, v) \geq 0$
 - $d_G(u, v) = 0 \iff u = v$
 - platí trojúhelníková nerovnost $d_G(u, w) \leq d_G(u, v) + d_G(v, w)$
 - $d_G(v, u) = d_G(u, v)$
 - grafové operace: přidání/odebrání vrcholu/hrany, dělení hrany, kontrakce hrany
 - sled v grafu (walk) - můžou se opakovat vrcholy i hrany

- $(v_0, e_1, v_1, e_2, \dots, e_n, v_n)$
- $\forall i : e_i = \{v_{i-1}, v_i\}$
- tah v grafu - můžou se opakovat vrcholy, hrany ne
 - tah může být uzavřený (končí, kde začal), nebo otevřený
- eulerovský tah
 - eulerovský tah obsahuje všechny hrany a vrcholy grafu
 - * někdy se eulerovský tah definuje bez nutnosti obsahovat všechny vrcholy
 - graf je eulerovský $\equiv$ existuje v něm uzavřený eulerovský tah
- věta: graf G je eulerovský $\iff G$ je souvislý a každý jeho vrchol má sudý stupeň
- Stromy - definice a základní vlastnosti (existence listů, počet hran stromu)
 - strom je souvislý graf bez kružnic (= acyklický)
 - les je acyklický graf
 - list je vrchol stupně 1
 - strom o jednom vrcholu („pařez“) nemá žádný list
 - lemma: každý strom s aspoň 2 vrcholy má aspoň 1 list (respektive aspoň dva listy)
 - lemma: je-li v list grafu G , pak G je strom, právě když $G - v$ je strom
 - strom G má $|V(G)| - 1$ hran (Eulerova formule)
- Ekvivalentní charakteristiky stromů
 1. G je souvislý a acyklický (strom)
 2. mezi vrcholy u, v existuje právě jedna cesta (jednoznačně souvislý)
 3. G je souvislý a po smazání libovolné jedné hrany už nebude souvislý (minimální souvislý)
 4. G je acyklický a po přidání libovolné jedné hrany vznikne cyklus (maximální acyklický)
 5. G je souvislý a platí pro něj Eulerova formule $|E(G)| = |V(G)| - 1$
- Rovinné grafy - definice a základní pojmy (rovinný graf a rovinné nakreslení grafu, stěny)
 - rovinné nakreslení grafu = nakreslení grafu, aby se hrany nekřížily
 - vrcholy = body v rovině (navzájem různé)
 - hrany = křivky, které se neprotínají a jejich společnými body jsou jejich společné vrcholy
 - stěny nakreslení
 - části, na které nakreslení grafu rozděluje rovinu
 - stěnou je i vnější stěna (zbytek roviny)
 - hranice stěny - skládá se z hran
 - hranice stěny je nakreslení uzavřeného sledu
 - graf je rovinný, pokud má alespoň jedno rovinné nakreslení
 - topologický graf je uspořádaná dvojice (graf, nakreslení)
 - K_5 a $K_{3,3}$ nejsou rovinné
 - graf je rovinný, právě když žádný jeho podgraf není izomorfní s nějakým dělením grafu K_5 nebo $K_{3,3}$ (tzn. podgraf může mít podrozdělené hrany - „ K_5 “ s libovolně podrozdělenými hranami pořád nebude rovinná)
- Eulerova formule a maximální počet hran rovinného grafu (důkaz a použití)
 - věta
 - nechť G je souvislý graf nakreslený do roviny, $v := |V(G)|$, $e := |E(G)|$, $f :=$ počet stěn nakreslení
 - potom $v + f = e + 2$ (Eulerova formule)
 - důkaz: indukci podle e
 - $e = v - 1$ (G je strom)
 - * $f = 1$

- * $v + 1 = v - 1 + 2 \quad \checkmark$
- $e - 1 \rightarrow e$
- * mějme graf G s e hranami
- * zvolím si libovolnou hranu x na kružnici
- * $G' := G - x$
- * $v' = v, \quad e' = e - 1, \quad f' = f - 1$
- * z IP: $v' + f' = e' + 2$
- * po dosazení: $v + f - 1 = e - 1 + 2$
- * k oběma stranám přičteme jedničku
- * $v + f = e + 2 \quad \square$
- věta: maximální rovinný graf je triangulace
 - dokud hranice stěny není trojúhelník, je tam nějaká dvojice nespojených vrcholů
- počet hran maximálního rovinného grafu
 - každá stěna přispěje třemi hranami
 - každá hrana patří ke dvěma stěnám
 - počítáme „strany hran“: $3f = 2e$
 - $f = \frac{2}{3}e$
 - $v + \frac{2}{3}e = e + 2$
 - $e = 3v - 6$
- věta: v každém rovinném grafu s aspoň 3 vrcholy je $|E| \leq 3|V| - 6$
- důkaz
 - doplníme do G hrany, až získáme maximální rovinný G'
 - $e' = 3v - 6$ (vrcholy nepřidáváme)
 - $e \leq e' = 3v - 6$
- důsledek
 - průměrný stupeň vrcholu v rovinném grafu je menší než 6
 - * $\sum \deg(\xi) = 2e \leq 6v - 12$
 - * průměrný stupeň $\leq \frac{6v-12}{v} < 6$
- počet hran a vrchol nízkého stupně v rovinných grafech bez trojúhelníků
 - maximální rovinné grafy bez trojúhelníků mají stěny čtvercové, pětiúhelníkové, nebo to může být strom ve tvaru hvězdy
 - pro čtvercově stěny platí $4f = 2e$, pro pětiúhelníkové $5f = 2e$
 - obecně $4f \leq 2e \rightarrow f \leq \frac{1}{2}e$
 - $v + \frac{1}{2}e \geq e + 2$
 - $e \leq 2v - 4$
 - průměrný stupeň $\leq \frac{4v-8}{v} < 4$
 - existuje vrchol stupně max. 3
- Barevnost grafů - definice dobrého obarvení, vztah barevnosti a klikovosti grafu
 - obarvení grafu G k barvami (k -obarvení) je $c : V(G) \rightarrow [k]$ t. ž. kdykoli $\{x, y\} \in E(G)$, pak $c(x) \neq c(y)$
 - barevnost $\chi(G)$ grafu $G := \min k : \exists k\text{-obarvení grafu } G$
 - pozorování: kdykoli $H \subseteq G$, pak $\chi(H) \leq \chi(G)$
 - barevnost některých grafů
 - úplné grafy ... $\chi(K_n) = n$
 - cesty ... $\chi(P_n) = 2$ pro $n \geq 1$
 - sudé kružnice ... $\chi(C_{2k}) = 2$
 - liché kružnice ... $\chi(C_{2k+1}) = 3$

- stromy jsou 2-obarvitelné
- pro rovinné grafy existuje věta o čtyřech barvách (je to těsný horní odhad - existují rovinné grafy, které nelze obarvit třemi barvami, např. K_4)
- věta: $\chi(G) \leq 2 \iff G$ je bipartitní $\iff G$ neobsahuje lichou kružnici
- definice: klikovost grafu $\kappa(G)$ je maximální k takové, že v grafu jako podgraf existuje úplný graf K_k
 - $\chi(G) \geq \kappa(G)$
- klika v grafu je množina vrcholů taková, že každé dva jsou spojené hranou
- nezávislá množina v grafu je množina vrcholů taková, že žádné dva vrcholy nejsou spojené hranou
- Hranová a vrcholová souvislost grafů
 - $F \subseteq E$ je hranový řez v G , pokud $G - F$ je nesouvislý
 - kde $G - F := (V, E \setminus F)$
 - graf je hranově k -souvislý, pokud neobsahuje žádný hranový řez velikosti menší než k
 - stupeň hranové souvislosti (nebo jen hranová souvislost) grafu G , značený $k_e(G)$, je největší k takové, že G je hranově k -souvislý
 - pozorování: $k_e(G) =$ velikost nejmenšího hranového řezu v G
 - $A \subseteq V$ je vrcholový řez, pokud $G - A$ je nesouvislý
 - kde $G - A = (V \setminus A, E \cap (V \setminus A)^2)$
 - graf je vrcholově k -souvislý, pokud má aspoň $k + 1$ vrcholů a neobsahuje žádný vrcholový řez velikosti menší než k
 - vrcholová souvislost grafu G , značená $k_v(G)$, je největší k takové, že G je vrcholově k -souvislý
 - pozorování: $k_v(K_n) = n - 1$
 - pozorování: G není úplný $\dots$ $k_v(G) =$ velikost nejmenšího vrcholového řezu
- Hranová a vrcholová verze Mengerovy věty
 - Mengerova věta, hranová xy -verze
 - pro dva různé vrcholy x, y grafu G platí, že G obsahuje k hranově disjunktních cest z x do $y \iff G$ neobsahuje hranový xy -řez velikosti menší než k
 - Mengerova věta, globální hranová verze
 - graf G je hranově k -souvislý $\iff$ mezi každými dvěma různými vrcholy existuje k hranově disjunktních cest
 - definice: dvě cesty v G z x do y jsou navzájem vnitřně vrcholově disjunktní (VVD), pokud nemají žádný společný vrchol kromě x, y
 - Mengerova věta, vrcholová xy -verze
 - nechť $G = (V, E)$ je graf
 - nechť x, y jsou různé **nesousední** vrcholy
 - nechť $k \in \mathbb{N}$
 - potom G obsahuje k navzájem VVD cest z x do $y \iff G$ neobsahuje vrcholový xy -řez velikosti $< k$
 - Mengerova věta, globální vrcholová verze
 - G je vrcholově k -souvislý $\iff$ mezi každými dvěma různými vrcholy x, y existuje k navzájem VVD cest
- Orientované grafy, silná a slabá souvislost
 - orientovaný graf $\dots (V, E) : E \subseteq V^2 \setminus \{(x, x) \mid x \in V\}$
 - nepovolíme smyčky (v podstatě zakážeme diagonálu na relaci)
 - podkladový graf je neorientovaný graf založený na tom původním orientovaném
 - pro orientovaný $G = (V, E)$ existuje podkladový $G^0 = (V, E^0)$, kde $\{u, v\} \in E^0 \equiv (u, v) \in E$

$$E \vee (v, u) \in E$$

- vstupní a výstupní stupeň $\deg^{\text{in}}, \deg^{\text{out}}$ (hrany vedoucí do vrcholu / z vrcholu)
- vrchol je vyvážený $\equiv \deg^{\text{in}}(v) = \deg^{\text{out}}(v)$
- graf je vyvážený $\equiv$ všechny vrcholy jsou vyvážené
- součet vstupních stupňů = součet výstupních stupňů = počet hran
- orientovaný graf je slabě souvislý $\equiv$ jeho podkladový graf je souvislý
- o. graf je silně souvislý $\equiv$ existuje orientovaná cesta mezi každými dvěma vrcholy
- silná souvislost $\implies$ slabá souvislost
- věta: pro orientovaný graf G platí: G je vyvážený a slabě souvislý $\iff G$ je eulerovský $\iff G$ je vyvážený a silně souvislý
- Toky v sítích: definice sítě a toku v ní
 - tokovou síť tvoří
 - $V \dots$ množina vrcholů
 - $E \dots$ množina orientovaných hran, $E \subseteq V \times V$
 - $z \in V \dots$ zdroj
 - $s \in V \setminus \{z\} \dots$ spotřebič / stok
 - $c : E \rightarrow [0, +\infty) \dots c(e)$ je kapacita hrany e
 - definice umožňuje mít hrany oběma směry, připouští také smyčky, ty však z hlediska toků v sítích nejsou nijak užitečné
 - poznámka: ze stoku může vycházet orientovaná hrana, podobně do zdroje může vést hrana
 - tok v síti (V, E, z, s, c) je funkce $f : E \rightarrow [0, +\infty)$ splňující
 - $\forall e \in E : 0 \leq f(e) \leq c(e)$
 - $\forall x \in V \setminus \{z, s\} : \text{součet toků do vrcholu } x \text{ se rovná součtu toků z vrcholu (formálně pomocí sum) } \dots \text{ Kirchhoffův zákon}$
- Existence maximálního toku (bez důkazu)
 - velikost toku f v síti (V, E, z, s, c) je $w(f) := f[\text{Out}(z)] - f[\text{In}(z)]$
 - kde $\text{Out}(z)$ a $\text{In}(z)$ jsou množiny hran do, respektive ze zdroje a $f[A] := \sum_{e \in A} f(e)$ pro $A \subseteq E$
 - maximální tok je tok, který má největší velikost
 - fakt: v každé tokové síti existuje maximální tok
 - poznámka: obecně na přednášce uvažujeme konečné tokové sítě, grafy apod.
 - idea důkazu
 - mějme síť, očíslovme hrany (od 1 do m)
 - tok f reprezentujeme jako uspořádanou m -tici hodnot
 - množina všech toků je uzavřená a omezená podmnožina $\mathbb{R}^m$, je tedy kompaktní
 - funkce, která toku f přiřadí $w(f)$, je spojitá
 - víme z analýzy, že spojitá funkce na kompaktní množině nabývá maxima
 - řez v síti (V, E, z, s, c) je množina hran $R \subseteq E$ taková, že každá orientovaná cesta ze z do s obsahuje aspoň jednu hranu R
 - Minimaxová věta o toku a řezu: nechť $f_{\max}$ je maximální tok a $R_{\min}$ je minimální řez v (V, E, z, s, c) , potom $w(f_{\max}) = c(R_{\min})$
- Princip hledání maximálního toku v síti s celočíselnými kapacitami (například pomocí Ford-Fulkersonova algoritmu)
 - rezerva hrany uv
 - $r(uv) := c(uv) - f(uv) + f(vu)$
 - hrana je nasycená $\equiv$ má nulovou rezervu
 - hrana je nenasyčená $\equiv$ má kladnou rezervu
 - cesta je nenasyčená $\equiv$ žádná její hrana není nasycená / všechny mají kladné rezervy

- algoritmus
 - iterujeme, dokud existuje nenasycená cesta ze zdroje do stoku
 - spočítáme rezervu celé cesty (minimum přes rezervy hran cesty)
 - pro každou hranu upravíme tok - v protisměru odečteme co nejvíc, zbytek přičteme po směru
- pro celočíselné kapacity vrátí celočíselný tok
- racionální kapacity převedeme na celočíselné
- pro iracionální kapacity se může rozbít

4.5 5. Pravděpodobnost a statistika

- Pravděpodobnostní prostor, náhodné jevy, pravděpodobnost (definice, příklady)
 - Ω ... množina elementárních jevů
 - $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ je prostor jevů, pokud ...
 - $\emptyset, \Omega \in \mathcal{F}$
 - $A \in \mathcal{F} \implies A^c = \Omega \setminus A \in \mathcal{F}$
 - $A_1, A_2, \dots \in \mathcal{F} \implies \bigcup A_i \in \mathcal{F}$
 - $P : \mathcal{F} \rightarrow [0, 1]$ je pravděpodobnost, pokud ...
 - $P(\Omega) = 1$
 - $P(\bigcup A_i) = \sum P(A_i)$ pro $A_1, A_2, \dots \in \mathcal{F}$ po dvou disjunktní
 - příklady pravděpodobnostních prostorů
 - klasický - tedy konečný s uniformní pravděpodobností, např. $\Omega = [6]$ (hod kostkou)
 - * příklad elementárního jevu ... padla šestka
 - * příklad jevu ... padlo sudé číslo
 - diskrétní - dovolíme cinknoutou kostku (nevyžadujeme uniformní pravděpodobnost)
 - geometrický - např. „náhodně“ střílíme na terč, takže šance, že zasáhneme konkrétní oblast, je úměrná obsahu oblasti (ale může být i ve více než dvou dimenzích)
 - spojitý - schopný střelec se trefuje spíše do středu terče (opět dovolíme i jiné než uniformní rozdělení)
- Základní pravidla pro počítání s pravděpodobnostmi
 - $P(A) + P(A^c) = 1$
 - $A \subseteq B \implies P(B \setminus A) = P(B) - P(A) \implies P(A) \leq P(B)$
 - $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
 - $P(A_1 \cup A_2 \cup \dots) \leq \sum_i P(A_i)$... subaditivita, Booleova nerovnost
- Nezávislost náhodných jevů, podmíněná pravděpodobnost
 - jevy $A, B \in \mathcal{F}$ jsou nezávislé, pokud $P(A \cap B) = P(A) \cdot P(B)$
 - můžeme uvažovat i větší množiny jevů
 - jevy v množině jsou (vzájemně) nezávislé, pokud pro každou konečnou podmnožinu platí, že $P(\bigcap) = \prod P$
 - pokud podmínka platí jen pro dvouprvkové podmnožiny, jsou jevy *po dvou nezávislé*
 - pokud $A, B \in \mathcal{F}$ a $P(B) > 0$, definujeme podmíněnou pravděpodobnost A při B jako $P(A | B) = \frac{P(A \cap B)}{P(B)}$
 - zjevně $P(A \cap B) = P(A) \cdot P(B | A)$
 - $P(A_1 \cap A_2 \cap \dots \cap A_n) = P(A_1)P(A_2 | A_1)P(A_3 | A_1 \cap A_2) \dots P(A_n | A_1 \cap \dots \cap A_{n-1})$
 - lze ukázat indukcí (nebo neformálně rozepsáním členů vpravo a vykrácením)
 - věta o úplné pravděpodobnosti

- mějme $B_1, B_2, \dots$ rozklad Ω
- $P(A) = \sum_i P(B_i) \cdot P(A | B_i)$
 - * sčítance s $P(B_i) = 0$ považujeme za nulové
- Bayesův vzorec
 - mějme $B_1, B_2, \dots$ rozklad Ω
 - $P(B_j | A) = \frac{P(B_j) \cdot P(A|B_j)}{\sum_i P(B_i) \cdot P(A|B_i)}$
- Náhodné veličiny a jejich rozdělení
 - náhodná veličina je funkce, která přiřazuje každému elementárnímu náhodnému jevu nějakou (obvykle číselnou) hodnotu
 - pro diskrétní náhodnou veličinu X definujeme pravděpodobnostní funkci $p_X(x) = P(\{X = x\})$
 - spojitá náhodná veličina obvykle popisujeme pomocí distribuční funkce a hustoty
 - distribuční funkce náhodné veličiny X je funkce $F_X : \mathbb{R} \rightarrow [0, 1]$ definovaná předpisem $F_X(x) := P(X \leq x)$
 - zjevně $P(a < X \leq b) = F_X(b) - F_X(a)$
 - vlastnosti
 - * F_X je neklesající
 - * $\lim_{x \rightarrow +\infty} F_X(x) = 1$
 - * $\lim_{x \rightarrow -\infty} F_X(x) = 0$
 - * F_X je zprava spojitá
 - náhodná veličina X se nazývá spojitá, pokud existuje nezáporná reálná funkce f_X tak, že $F_X(x) = P(X \leq x) = \int_{-\infty}^x f_X(t) dt$
 - hustota $f_X \dots$ „limita histogramů“
 - zjevně $\int_{-\infty}^{\infty} f_X = 1$
- Střední hodnota - linearita střední hodnoty, střední hodnota součinu nezávislých veličin, Markovova nerovnost
 - $\mathbb{E}(X) = \sum_{x \in \text{Im}(X)} x \cdot P(X = x)$
 - nebo $\mathbb{E}(X) = \int_{-\infty}^{\infty} x f_X(x) dx$
 - pozorování: $\mathbb{E}(X) = \sum_{\omega \in \Omega} X(\omega) \cdot P(\{\omega\})$
 - pravidlo naivního statistika
 - $\mathbb{E}(g(X)) = \sum_{x \in \text{Im}(X)} g(x) P(X = x)$
 - nebo $\mathbb{E}(g(X)) = \int_{-\infty}^{\infty} g(x) f_X(x) dx$
 - linearita $\mathbb{E}(aX + b) = a\mathbb{E}(X) + b$ plyne z PNS pro funkci $ax + b$
 - podmíněná střední hodnota $\mathbb{E}(X | B) = \sum_{x \in \text{Im}(X)} x \cdot P(X = x | B)$
 - věta o celkové střední hodnotě: $\mathbb{E}(X) = \sum_i P(B_i) \cdot \mathbb{E}(X | B_i)$
 - $\mathbb{E}(g(X, Y)) = \sum_x \sum_y g(x, y) P(X = x \wedge Y = y)$
 - $\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$
 - pro nezávislé X, Y platí $\mathbb{E}(XY) = \mathbb{E}(X)\mathbb{E}(Y)$
 - Markovova nerovnost
 - pro $X \geq 0$ a $a > 0$ platí $P(X \geq a) \leq \frac{\mathbb{E}(X)}{a}$
- Rozptyl - definice, vzorec pro rozptyl součtu (závislých či nezávislých veličin)
 - rozptyl $\dots \text{var}(X) = \mathbb{E}((X - \mathbb{E}X)^2)$
 - směrodatná odchylka $\dots \sigma_X = \sqrt{\text{var}(X)}$
 - věta: $\text{var}(X) = \mathbb{E}(X^2) - \mathbb{E}(X)^2$
 - počítání s rozptylem
 - $\text{var}(aX + b) = a^2 \text{var}(X)$

- rozptyl součtu nezávislých veličin je součet rozptylů
 - $\text{var}(X + Y) = \text{var}(X) + \text{var}(Y)$
- kovariance a její vlastnosti
 - definice: $\text{cov}(X, Y) = \mathbb{E}((X - \mathbb{E}X)(Y - \mathbb{E}Y))$
 - věta: $\text{cov}(X, Y) = \mathbb{E}(XY) - \mathbb{E}(X)\mathbb{E}(Y)$
 - pro nezávislé X, Y platí $\text{cov}(X, Y) = 0$
 - zjevně $\text{cov}(X, X) = \text{var}(X)$
 - $\text{cov}(aX + bY, Z) = a\text{cov}(X, Z) + b\text{cov}(Y, Z)$
 - korelace
 - * $\rho(X, Y) = \frac{\text{cov}(X, Y)}{\sigma_X \cdot \sigma_Y}$
- rozptyl součtu náhodných veličin (závislých i nezávislých)
 - necht $X = \sum_{i=1}^n X_i$
 - pak $\text{var}(X) = \sum_{i=1}^n \sum_{j=1}^n \text{cov}(X_i, X_j)$
- Práce s konkrétními rozděleními: geometrické, binomické, Poissonovo, normální, exponenciální
 - Bernoulliho/alternativní rozdělení $\text{Ber}(p)$
 - počet úspěchů při jednom pokusu (kde p je pravděpodobnost úspěchu)
 - $p_X(1) = p$
 - $p_X(0) = 1 - p$
 - jinak $p_X(x) = 0$
 - $\mathbb{E}(X) = p$
 - $\text{var}(X) = p(1 - p)$
 - geometrické rozdělení $\text{Geom}(p)$
 - při kolikátém pokusu poprvé uspějeme
 - $p_X(k) = (1 - p)^{k-1} \cdot p$
 - $\mathbb{E}(X) = 1/p$
 - $\text{var}(X) = \frac{1-p}{p^2}$
 - binomické rozdělení $\text{Bin}(n, p)$
 - počet úspěchů při n nezávislých pokusech
 - $p_X(k) = \binom{n}{k} p^k (1 - p)^{n-k}$
 - $\mathbb{E}(X) = np$
 - $\text{var}(X) = np(1 - p)$
 - Poissonovo rozdělení $\text{Pois}(\lambda)$
 - počet doručených zpráv za časový úsek, λ je průměrná hodnota
 - $p_X(k) = \frac{\lambda^k}{k!} e^{-\lambda}$
 - $\text{Pois}(\lambda)$ je limitou $\text{Bin}(n, \lambda/n)$ pro $n \rightarrow \infty$
 - $\mathbb{E}(X) = \lambda$
 - $\text{var}(X) = \lambda$
 - uniformní $U(a, b)$
 - $f_X(x) = \frac{1}{b-a}$
 - $F_X(x) = \frac{x-a}{b-a}$
 - $\mathbb{E}(X) = \frac{a+b}{2}$
 - $\text{var}(X) = \frac{(b-a)^2}{12}$
 - exponenciální $\text{Exp}(\lambda)$
 - $f_X(x) = \lambda e^{-\lambda x}$ pro $x \geq 0$
 - $F_X(x) = 1 - e^{-\lambda x}$ pro $x \geq 0$
 - $\mathbb{E}(X) = 1/\lambda$

- $\text{var}(X) = 1/\lambda^2$
- standardní normální rozdělení
 - $N(0, 1)$
 - $f_X(x) = \varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$
- obecné normální rozdělení
 - $N(\mu, \sigma^2)$
 - $f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{x-\mu}{\sigma})^2}$
 - máme-li $X \sim N(\mu, \sigma^2)$, pak pro $Z = \frac{X-\mu}{\sigma}$ platí $Z \sim N(0, 1)$
 - pro normální nezávislé náhodné veličiny $X_i \sim N(\mu_i, \sigma_i^2)$ (necht' je jich konečně) je i součet normální náhodná veličina $\sum X_i \sim N(\sum \mu_i, \sum \sigma_i^2)$
 - pravidlo 3σ
 - * $P(\mu - \sigma < X < \mu + \sigma) \doteq 0.68$
 - * $P(\mu - 2\sigma < X < \mu + 2\sigma) \doteq 0.95$
 - * $P(\mu - 3\sigma < X < \mu + 3\sigma) \doteq 0.997$
- Limitní věty: zákon velkých čísel
 - mějme $X_1, X_2, \dots$ stejně rozdělené nezávislé náhodné veličiny
 - $\bar{X}_n = (X_1 + \dots + X_n)/n \dots$ výběrový průměr
 - silný zákon velkých čísel
 - $\lim_{n \rightarrow \infty} \bar{X}_n = \mu$ skoro jistě (s pravděpodobností 1)
 - použití: Monte Carlo integrování kruhu
 - slabý zákon velkých čísel (zlepšení přesnosti opakovaným měřením)
 - $\forall \varepsilon > 0 : \lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| > \varepsilon) = 0$
 - říkáme, že $\bar{X}_n$ konverguje k μ v pravděpodobnosti, píšeme $\bar{X}_n \xrightarrow{P} \mu$
- Limitní věty: centrální limitní věta
 - necht' $X_1, X_2, \dots$ jsou stejně rozdělené se střední hodnotou μ a rozptylem σ^2
 - označme $Y_n = \frac{(X_1 + \dots + X_n) - n\mu}{\sigma\sqrt{n}}$
 - pak $Y_n \xrightarrow{d} N(0, 1)$
 - Y_n konverguje v distribuci k $N(0, 1)$
 - tzn. $\lim_{n \rightarrow \infty} F_{Y_n}(x) = \Phi(x)$
 - CLV se hodí k aproximaci distribuce součtu nebo průměru velkého počtu náhodných veličin normálním rozdělením
 - takže můžeme provádět bodové a intervalové odhady i tam, kde data nejsou normálně rozdělená, ale známe rozptyl
- Bodové odhady - alespoň jedna metoda pro jejich tvorbu, vlastnosti
 - poznámka: neodhadujeme přímo θ , nýbrž $g(\theta)$, aby to bylo obecnější
 - pro náhodný výběr $X_1, \dots, X_n \sim F_\theta$ a libovolnou funkci g nazveme bodový odhad $\hat{\theta}_n \dots$
 - nevychýlený/nestranný, pokud $\mathbb{E}(\hat{\theta}_n) = g(\theta)$
 - asymptoticky nevychýlený, pokud $\lim_{n \rightarrow \infty} \mathbb{E}(\hat{\theta}_n) = g(\theta)$
 - konzistentní, pokud $\hat{\theta}_n \xrightarrow{P} g(\theta)$
 - dále definujeme
 - vychýlení ... bias($\hat{\theta}_n$) = $\mathbb{E}(\hat{\theta}_n) - g(\theta)$
 - střední kvadratickou chybu ... MSE($\hat{\theta}_n$) = $\mathbb{E}((\hat{\theta}_n - g(\theta))^2)$
 - věta: MSE($\hat{\theta}_n$) = bias($\hat{\theta}_n$)² + var($\hat{\theta}_n$)
 - metoda momentů
 - r -tý moment $X \dots \mathbb{E}(X^r) = m_r(\theta)$

- r -tý výběrový moment ... $\frac{1}{n} \sum_{i=1}^n X_i^r = \widehat{m}_r$
 - * konzistentní nevychýlený odhad pro r -tý moment
- nalezneme θ takové, že $m_r(\theta) = \widehat{m}_r$
- typicky stačí použít první moment, dostaneme nějakou rovnici
- $m_1 = \mu$
- $m_2 = \mathbb{E}(X^2) = \text{var}(X) + (\mathbb{E}X)^2 = \sigma^2 + \mu^2$
- metoda maximální věrohodnosti
 - $\hat{\theta}_{ML} = \text{argmax}_{\theta} p(x; \theta)$
 - * $\text{argmax}_{\theta} f(x; \theta)$
 - * abych se nemusel rozhodovat mezi p a f , budu používat L
 - výpočetně jednodušší bude používat logaritmus L , který označíme ℓ
 - příklad
 - * ve vzorku k leváků z n lidí, hledáme pravděpodobnost θ , že je někdo levák
 - * $L(x; \theta) = \theta^k (1 - \theta)^{n-k}$
 - * $\ell(x; \theta) = k \log \theta + (n - k) \log(1 - \theta)$
 - * $\ell'(x; \theta) = \frac{k}{\theta} - \frac{n-k}{1-\theta}$
 - hledáme maximum, položíme derivaci rovnou nule (a zkontrolujeme krajní hodnoty)
 - podobně pro spojitý případ - „maximalizujeme“ rovnici pro pravděpodobnost konkrétního výběru
- Intervalové odhady: metoda založená na aproximaci normálním rozdělením
 - statistiky $D \leq H$ určují konfidenční interval o spolehlivosti $1 - \alpha$, pokud $P(D \leq \theta \leq H) = 1 - \alpha$
 - zkráceně $(1 - \alpha)$ -CI
 - interval budeme uvažovat ve tvaru $[x - \delta, x + \delta]$
 - postup
 - máme nestranný bodový odhad $\hat{\theta}$ pro parametr θ
 - $\hat{\theta}$ má normální rozdělení
 - $\hat{\theta} \pm z_{\alpha/2} \cdot \text{se}$ je $(1 - \alpha)$ -CI
 - * $z_{\alpha/2} := \Phi^{-1}(1 - \alpha/2)$
 - * $\text{se} := \sigma(\hat{\theta})$
- Testování hypotéz - základní přístup, chyby 1. a 2. druhu, hladina významnosti
 - zvolíme testovou statistiku T
 - nulová hypotéza ... defaultní, konzervativní model
 - alternativní hypotéza ... alternativní model, „zajímavost“
 - nulovou hypotézu buď zamítneme, nebo nezamítneme
 - chyba 1. druhu - chybné zamítnutí, „trapas“
 - chyba 2. druhu - chybné přijetí, „promarněná příležitost“
 - hladina významnosti α ... pravděpodobnost chyby 1. druhu
 - typicky se volí $\alpha = 0.05$
 - β ... pravděpodobnost chyby 2. druhu
 - kritický obor ... je to množina, kterou určíme před provedením testu; pokud se výsledek našeho testu bude nacházet v kritickém oboru, zamítneme nulovou hypotézu
 - tedy $\alpha = P(h(X) \in W; H_0)$
 - síla testu ... $1 - \beta$
 - chceme co největší
 - p -hodnota ... nejmenší α taková, že na hladině α zamítáme H_0
 - takže zamítáme, pokud p -hodnota vyjde menší rovna naší zvolené α

- p -hodnota je vlastně pravděpodobnost, že při platnosti H_0 nabývá zvolená testová statistika T naměřené hodnoty nebo hodnot ještě extrémnějších

4.6 6. Logika

- Znalost a práce se základními pojmy syntaxe výrokové a predikátové logiky (jazyk, otevřená a uzavřená formule, instance formule, apod.)
 - výroková logika
 - jazyk je určený neprázdnou množinou výrokových proměnných $\mathbb{P}$ (také jim říkáme prvovýroky nebo atomické výroky)
 - množina $\mathbb{P}$ může být konečná nebo i nekonečná (ale obvykle bude spočetná) a bude mít dané uspořádání
 - predikátová logika
 - signatura je dvojice $\langle \mathcal{R}, \mathcal{F} \rangle$, kde $\mathcal{R}, \mathcal{F}$ jsou disjunktní množiny symbolů (relační a funkční, ty zahrnují konstantní) spolu s danými aritami a neobsahují symbol $=$
 - * signaturu ale často uvádíme jako prostý výčet symbolů
 - do jazyka patří...
 - * spočetně mnoho proměnných
 - * relační, funkční a konstantní symboly ze signatury a symbol $=$, jde-li o jazyk s rovností
 - * univerzální a existenční kvantifikátory pro každou proměnnou
 - * symboly pro logické spojky a závorky
 - např. jazyk uspořádání zapíšeme jako „ $L = \langle \leq \rangle$ s rovností“
 - * tzn. napíšeme signaturu a také uvedeme, zda jde o jazyk s rovností
 - struktura v signatuře $\langle \mathcal{R}, \mathcal{F} \rangle$ je trojice $\mathcal{A} = \langle A, \mathcal{R}^{\mathcal{A}}, \mathcal{F}^{\mathcal{A}} \rangle$, kde...
 - * A je neprázdná množina, říkáme jí doména (také universum)
 - * $\mathcal{R}^{\mathcal{A}}$ je množina interpretací jednotlivých relačních symbolů (kde interpretace n -árního relačního symbolu je množina uspořádaných n -tic prvků z A)
 - * $\mathcal{F}^{\mathcal{A}}$ je množina interpretací jednotlivých funkčních symbolů (kde interpretace n -árního funkčního symbolu odpovídá funkci přiřazující n -tici prvků z A jeden prvek z A)
 - speciálně pro konstantní symbol $c \in \mathcal{F}$ je $c^{\mathcal{A}} \in A$
 - termy jazyka L jsou konečné nápisy definované induktivně
 - * proměnná je term
 - * konstantní symbol je term
 - * pro n -ární funkční symbol f a termy $t_1, \dots, t_n$ je nápis $f(t_1, \dots, t_n)$ také term
 - atomická formule jazyka L je nápis $R(t_1, \dots, t_n)$, kde R je n -ární relační symbol z L (v jazyce s rovností to může být i rovnost) a t_i je L -term
 - formule jazyka L jsou konečné nápisy definované induktivně
 - * atomická formule je formule
 - * negace formule je formule
 - * spojením dvou formulí binární logickou spojkou dostaneme formuli
 - * přidáním kvantifikátoru před formuli dostaneme formuli
 - výskyt proměnné je vázaný, pokud jí odpovídá nějaký kvantifikátor, jinak je výskyt volný
 - proměnná je volná, pokud má volný výskyt, je vázaná, pokud má vázaný výskyt
 - formule je otevřená, neobsahuje-li žádný kvantifikátor
 - formule je uzavřená (= sentence), pokud nemá žádnou volnou proměnnou
 - term t je substituovatelný za proměnnou x ve formuli φ , pokud po simultánním nahrazení všech volných výskytů x ve φ za t nevznikne ve φ žádný vázaný výskyt proměnné z t

- * v tom případě říkáme vzniklé formuli instance φ vzniklá substitucí t za x a označujeme ji $\varphi(x/t)$
- Prenexní tvary formulí predikátové logiky
 - PNF = kvantifikátory jsou před formulí, formule se dělí na kvantifikátorový prefix a otevřené jádro
 - převod na PNF - postupně kvantifikátory vytahuju (podle pravidel), v případě potřeby přejmenuju proměnnou, tím vznikne varianta (pokud je v druhé části formule volná proměnná se stejným názvem)
 - pravidlo převodu: pokud vytahujeme kvantifikátor z negace, musíme ho „přepnout“ (pozor: u implikace je levá strana jakoby negovaná)
- Znalost základních normálních tvarů (CNF, DNF, PNF)
 - PNF = kvantifikátory jsou před formulí, formule se dělí na kvantifikátorový prefix a otevřené jádro
 - tvary výroků
 - literál je prvovýrok nebo negace prvovýroku
 - * pro literál ℓ označuje $\bar{\ell}$ literál k němu opačný (tedy jeho negaci)
 - klauzule je disjunkce literálů
 - * jednotková klauzule je samotný literál
 - * prázdná klauzule je $\perp$
 - výrok je v konjunktivní normální formě (CNF), pokud je konjunkcí klauzulí
 - * prázdný výrok v CNF je $\top$
 - elementární konjunkce je konjunkce literálů
 - * jednotková elementární konjunkce je samotný literál
 - * prázdná elementární konjunkce je $\top$
 - výrok je v disjunktivní normální formě (DNF), pokud je disjunkcí elementárních konjunktí
 - * prázdný výrok v DNF je $\perp$
 - výrok je hornovský (v Hornově tvaru), pokud je konjunkcí hornovských klauzulí, tj. klauzulí obsahujících nejvýše jeden pozitivní literál
 - množinová reprezentace
 - klauzule je konečná množina literálů
 - * prázdnou klauzuli označíme $\square$, není nikdy splněna
 - CNF formule je (konečná nebo nekonečná) množina klauzulí
 - * prázdná formule $\emptyset$ je vždy splněna
- Převody na normální tvary
 - sémantický převod
 - najdeme modely výroku
 - pro DNF budeme uvažovat disjunkci elementárních konjunktí, přičemž každá elementární konjunkce popisuje jeden model
 - pro CNF musíme najít nemodely formule, každá klauzule zakáže jeden nemodel
 - * pokud je první nemodel $(0, 0, 0)$, pak první klauzule bude $(p \vee q \vee r)$
 - syntaktický převod
 - přepíšeme implikaci a ekvivalenci pomocí konjunkce, disjunkce a negace
 - přesuneme negace dovnitř (k literálům)
 - použijeme distributivitu konjunkce a disjunkce, abychom jednu přesunuli dovnitř (podle toho, jestli chceme CNF nebo DNF)
- Použití normálních tvarů pro algoritmy (SAT, rezoluce)

- problém Horn-SAT
 - výrok je hornovský, pokud je konjunkcí hornovských klauzulí, tj. klauzulí obsahujících nejvýše jeden pozitivní literál
 - algoritmus
 - * pokud φ obsahuje dvojici opačných jednotkových klauzulí, není splnitelný
 - * pokud φ neobsahuje žádnou jednotkovou klauzuli, je splnitelný, ohodnotíme všechny zbývající proměnné nulou
 - * pokud φ obsahuje jednotkovou klauzuli ℓ , ohodnotíme literál ℓ hodnotou 1, provedeme jednotkovou propagaci a postup opakujeme
 - jednotková propagace pro $\ell = 1$
 - * každou klauzuli obsahující ℓ odstraníme (protože je takto splněna)
 - * $\bar{\ell}$ odstraníme ze všech klauzulí, které ho obsahují (protože $\bar{\ell}$ nemůže zajistit splnění dané klauzule)
 - Horn-SAT lze řešit v lineárním čase
- algoritmus DPLL pro řešení SAT
 - literál ℓ má čistý výskyt ve φ , pokud se ℓ vyskytuje ve φ a opačný literál $\bar{\ell}$ se ve φ nevyskytuje
 - * takový literál můžeme nastavit na 1
 - * to neovlivní splnitelnost výroku, ale zmenší to množinu modelů, které jsem schopen nalézt
 - algoritmus
 - * dokud φ obsahuje jednotkovou klauzuli ℓ , ohodnot $\ell = 1$ a proved' jednotkovou propagaci
 - * dokud existuje literál ℓ , který má ve φ čistý výskyt, ohodnot $\ell = 1$ a odstraň klauzule obsahující ℓ
 - * pokud φ neobsahuje žádnou klauzuli, je splnitelný
 - * pokud φ obsahuje prázdnou klauzuli, není splnitelný
 - * jinak zvol dosud neohodnocenou výrokovou proměnnou p a zavolej algoritmus rekurzivně na $\varphi \wedge p$ a na $\varphi \wedge \neg p$
 - algoritmus běží v exponenciálním čase
- rezoluce
 - použitím rezolučního pravidla z klauzulí $\varphi \vee p$ a $\psi \vee \neg p$ dostaneme $\varphi \vee \psi$
 - takhle spolu můžeme postupně rezolvovat různé klauzule z teorie
 - pokud nám vyjde prázdná klauzule, je teorie sporná
 - k dokázání výroku φ v teorii T hledáme rezoluční zamítnutí $T \cup \neg\varphi$
 - * nebo přímo rezoluční důkaz výroku φ (tzn. na konci rezoluce nám vyjde φ)
- Pojem modelu teorie
 - model (jazyka) ve výrokové logice je pravdivostní ohodnocení proměnných
 - model v je modelem teorie T , pokud každý axiom z T v modelu v platí, píšeme $v \models T$
 - v predikátové logice: model jazyka L nebo také L -struktura je libovolná struktura v signatuře jazyka L
 - model teorie T je L -struktura, ve které platí všechny axiomy teorie T
- Pravdivost, lživost, nezávislost formule vzhledem k teorii, splnitelnost, tautologie, důsledek
 - pravdivý výrok platí v každém modelu (jazyka/teorie)
 - je pravdivý v logice = platí v logice = je tautologie
 - je pravdivý v T = platí v T = je důsledek T
 - lživý (sporný) výrok neplatí v žádném modelu
 - nezávislý výrok platí v nějakém modelu a neplatí v jiném modelu

- splnitelný výrok platí v nějakém modelu (tedy není lživý, je pravdivý nebo nezávislý)
- Analýza výrokových teorií nad konečně mnoha prvovýroky
 - nechť T je bezesporná teorie nad $\mathbb{P}$
 - $|\mathbb{P}| = n \in \mathbb{N}^+$... počet proměnných v jazyce
 - $m = |M_{\mathbb{P}}(T)|$... velikost množiny modelů teorie T
 - neekvivalentních výroků nad $\mathbb{P}$ je 2^{2^n}
 - každý výrok odpovídá jedné podmnožině množiny modelů (všech modelů je 2^n)
 - neekvivalentních výroků nad $\mathbb{P}$ pravdivých/lživých v T je $2^{2^n - m}$
 - u pravdivých musí množina modelů obsahovat všech m modelů, takže je zafixujeme (u lživých naopak nesmí obsahovat žádný z nich)
 - neekv. výroků nad $\mathbb{P}$ nezávislých v T je $2^{2^n} - 2 \cdot 2^{2^n - m}$
 - vezmeme všechny a odečteme pravdivé a lživé
 - neekv. jednoduchých extenzí teorie T je 2^m , z toho sporná je jedna
 - podle toho, které modely ubereme
 - neekv. kompletních jednoduchých extenzí teorie T je m
 - každá odpovídá jednomu modelu
 - T -neekvivalentních výroků nad $\mathbb{P}$ je 2^m
 - T -neekv. výroků nad $\mathbb{P}$ pravdivých/lživých (v T) je 1
 - T -neekv. výroků nad $\mathbb{P}$ nezávislých (v T) je $2^m - 2$
- Extenze teorií - schopnost porovnat sílu teorií, konzervativnost, skolemizace
 - extenze teorie T je libovolná teorie T' v jazyce $\mathbb{P}' \supseteq \mathbb{P}$ splňující $\text{Cs}_{\mathbb{P}}(T) \subseteq \text{Cs}_{\mathbb{P}'}(T')$
 - kde $\text{Cs}_{\mathbb{P}}(T)$ je množina všech důsledků teorie (výroků/sentencí pravdivých v teorii T v jazyce $\mathbb{P}$)
 - extenze je jednoduchá, pokud $\mathbb{P}' = \mathbb{P}$
 - extenze je konzervativní, pokud $\text{Cs}_{\mathbb{P}}(T) = \text{Cs}_{\mathbb{P}}(T') = \text{Cs}_{\mathbb{P}'}(T') \cap \text{VF}_{\mathbb{P}}$
 - sémantický význam (v řeči modelů)
 - mějme teorii T v jazyce $\mathbb{P}$ a teorii T' v jazyce $\mathbb{P}'$, kde $\mathbb{P} \subseteq \mathbb{P}'$
 - T' je jednoduchou extenzí T , právě když $\mathbb{P}' = \mathbb{P}$ a $M_{\mathbb{P}}(T') \subseteq M_{\mathbb{P}}(T)$
 - T' je extenzí T , právě když $M_{\mathbb{P}'}(T') \subseteq M_{\mathbb{P}'}(T)$
 - T' je konzervativní extenzí T , pokud je extenzí a navíc platí, že každý model T lze nějak expandovat na model T'
 - T' je extenzí T a zároveň T je extenzí T' , právě když $T' \sim T$ (jazyky a množiny modelů se rovnají)
 - kompletní jednoduché extenze T jednoznačně až na ekvivalenci odpovídají modelům T
 - T' je extenzí T o definice, pokud vznikla z T postupnou extenzí o definice relačních a funkčních (případně konstantních) symbolů
 - je-li T' extenze teorie T o definice, potom platí:
 - každý model teorie T lze jednoznačně expandovat na model T'
 - T' je konzervativní extenze T
 - pro každou L' -formuli φ' existuje L -formule φ taková, že $T' \models \varphi' \leftrightarrow \varphi$
 - Skolemova varianta sentence vzniká z původní PNF sentence skolemizací
 - skolemizace spočívá v tom, že tyto kroky iterujeme přes všechny existenční kvantifikátory ($\exists y_i$)
 - * odstraníme z prefixu existenční kvantifikátor ($\exists y_i$)
 - * za proměnnou y_i substituujeme term $f_i(x_1, \dots, x_{n_i})$, kde $x_1, \dots, x_{n_i}$ jsou proměnné, jejichž univerzální kvantifikátory předcházejí ($\exists y_i$)

- začínáme s L -sentencí v PNF, jejíž všechny vázané proměnné jsou různé
- dostaneme L' -sentenci v PNF, kde L' je rozšíření L o nové n_i -ární funkční symboly
- Dokazatelnost - pojem formálního důkazu, zamítnutí; schopnost práce v některém z formálních dokazovacích systémů (např. tablo)
 - důkaz, zamítnutí
 - důkaz faktu, že v teorii T platí výrok φ je konečný syntaktický objekt vycházející z axiomů T a výroku φ
 - důkaz konstruuje syntakticky, aniž bychom se museli zabývat modely
 - po dokazovacím systému požadujeme dvě vlastnosti: korektnost (je-li výrok dokazatelný z T , je pravdivý v T) a úplnost (je-li výrok pravdivý v T , je dokazatelný z T)
 - důkaz říká, že φ platí, zamítnutí říká, že φ neplatí
 - * rezoluční důkaz φ v teorii S má v kořeni φ , rezoluční zamítnutí teorie S má v kořeni $\square$
 - tablo důkaz
 - konečné tablo z teorie T je uspořádaný, položkami označovaný strom zkonstruovaný aplikací konečně mnoha následujících pravidel:
 - * jednoprvkový strom označovaný libovolnou položkou je tablo z teorie T
 - * pro libovolnou položku P na libovolné větvi V můžeme na konec větve V připojit atomické tablo pro položku P
 - * na konec libovolné větve můžeme připojit položku $T\alpha$ pro libovolný axiom teorie $\alpha \in T$
 - tablo je konečné nebo nekonečné, každopádně vzniklo ve spočetně mnoha krocích
 - tablo pro položku P je tablo s položkou P v kořeni
 - tablo důkaz výroku φ z teorie T je sporné tablo z teorie T s položkou $F\varphi$ v kořeni
 - * pokud existuje, je φ tablo dokazatelný z T , píšeme $T \vdash \varphi$
 - * podobně definujeme tablo zamítnutí s $T\varphi$ v kořeni, tablo zamítnutelnost se značí $T \vdash \neg\varphi$
 - tablo je sporné, pokud je každá jeho větev sporná
 - větev je sporná, pokud obsahuje položky $T\psi$ a $F\psi$ pro nějaký výrok ψ , jinak je bezesporná
 - tablo je dokončené, pokud je každá jeho větev dokončená
 - větev je dokončená...
 - * pokud je sporná
 - * nebo pokud je každá její položka na této větvi redukována a pokud větev zároveň obsahuje položku s T pro každý axiom teorie
 - položka je redukována na dané větvi, pokud obsahuje pouze výrokovou proměnnou nebo se na dané větvi vyskytuje jako kořen atomického tabla (tedy došlo k jejímu rozvoji na dané větvi)
 - v predikátové logice navíc řešíme kvantifikátory - položky typu *svědek* a *všichni*
 - Věty o kompaktnosti a úplnosti výrokové a predikátové logiky - znění a porozumění významu; použití na příkladech, důsledky
 - věta o kompaktnosti: teorie má model, právě když každá její konečná část má model
 - aplikace
 - popíšeme požadovanou vlastnost nekonečného objektu pomocí (nekonečné) výrokové teorie
 - z konečné části teorie sestojíme konečný podobjekt mající danou vlastnost
 - příklady
 - spočetně nekonečný graf je bipartitní $\iff$ každý jeho konečný podgraf je bipartitní
 - * $T = \{ p_u \rightarrow \neg p_v \mid \{u, v\} \in E(G) \}$
 - nestandardní model přirozených čísel
 - * vezmeme standardní model přirozených čísel (jeho teorii)
 - * přidáme nový konstantní symbol

- * přidáme k teorii výrok, že je ta konstanta ostře větší než každý n -tý numerál
- * každá konečná část této nové teorie má model, takže i celá ta nová teorie má model
- První věta o neúplnosti: Pro každou bezespornou rekurzivně axiomatizovanou extenzi T Robinsonovy aritmetiky existuje sentence, která je pravdivá v $\mathbb{N}$, ale není dokazatelná v T .
 - Robinsonova aritmetika je teorie jazyka aritmetiky $L = \langle S, +, \cdot, 0, \leq \rangle$ s rovností
 - * má osm (tradičně spíše sedm) axiomů, některé z nich:
 - $S(x) \neq 0$
 - $S(x) = S(y) \rightarrow x = y$
 - $x + 0 = x$
 - $x + S(y) = S(x + y)$
 - $\mathbb{N} = \langle \mathbb{N}, S, +, \cdot, 0, \leq \rangle \dots$ standardní model přirozených čísel
- důsledek 1.1: je-li T rekurzivně axiomatizovaná extenze Robinsonovy aritmetiky a je-li navíc $\mathbb{N}$ modelem teorie T , potom T není kompletní
 - pro sentenci, která není dokazatelná v T , nemůže být dokazatelná ani její negace (byl by to spor s její pravdivostí v $\mathbb{N}$)
- důsledek 1.2: teorie $\text{Th}(\mathbb{N})$ není rekurzivně axiomatizovatelná
 - kdyby byla, nemohla by být kompletní - ale ona kompletní je
 - značení: $\text{Th}(\mathbb{N}) \dots$ množina všech sentencí pravdivých v $\mathbb{N}$ (tzv. teorie struktury $\mathbb{N}$)
- věta (Rosserův trik): v každé bezesporné rekurzivně axiomatizované extenzi Robinsonovy aritmetiky existuje nezávislá sentence - tedy taková není kompletní
 - v podstatě plyne z důsledku 1.1, jen se zbavuje $\mathbb{N}$
- Druhá věta o neúplnosti: Pro každou bezespornou rekurzivně axiomatizovanou extenzi T Peanovy aritmetiky platí, že Con_T není dokazatelná v T .
- $\text{Con}_T \dots$ sentence vyjadřující bezespornost (konzistence) teorie T
 - $\text{Con}_T = \neg(\exists y) \text{Prf}_T(0 = S(0), y)$
 - $\text{Prf}_T(x, y) \dots y$ je důkaz x v T
- důsledek 2.1: existuje model Peanovy aritmetiky (PA), ve kterém platí sentence $(\exists y) \text{Prf}_{PA}(0 = S(0), y)$
- důsledek 2.2: existuje bezesporná rekurzivně axiomatizovaná extenze T Peanovy aritmetiky, která dokazuje svou spornost, tj. taková, že $T \vdash \neg \text{Con}_T$
- důsledek 2.3: je-li teorie množin ZFC bezesporná, nemůže být sentence Con_{ZFC} v teorii ZFC dokazatelná
- Rozhodnutelnost - pojem kompletnosti a její kritéria, význam pro rozhodnutelnost; příklady rozhodnutelných a nerozhodnutelných teorií
 - teorie je sporná, jestliže... (ekvivalentně)
 - v ní platí spor
 - nemá žádný model
 - v ní platí všechny výroky
 - teorie je bezesporná (splnitelná), pokud není sporná, tj. má nějaký model
 - teorie je kompletní jestliže není sporná a každý výrok (respektive sentence) je v ní pravdivý nebo lživý (tj. nemá žádné nezávislé výroky), ekvivalentně, pokud má právě jeden model (až na elementární ekvivalenci v predikátové logice)
 - struktury $\mathcal{A}, \mathcal{B}$ (v témž jazyce) jsou elementárně ekvivalentní, pokud v nich platí tytéž sentence (značíme $\mathcal{A} \equiv \mathcal{B}$)
 - zjevně $\mathcal{A} \equiv \mathcal{B} \iff \text{Th}(\mathcal{A}) = \text{Th}(\mathcal{B})$
 - o teorii T říkáme, že je...
 - rozhodnutelná, pokud existuje algoritmus, který pro každou vstupní formuli φ doběhne a

- odpoví, zda $T \models \varphi$
- částečně rozhodnutelná, pokud existuje algoritmus, který pro každou vstupní formuli $\varphi \dots$
 - * pokud $T \models \varphi$, doběhne a odpoví „ano“
 - * pokud $T \not\models \varphi$, buď nedoběhne, nebo doběhne a odpoví „ne“
- „rekurzivní“ pojmy
 - teorie T je rekurzivně axiomatizovaná, pokud existuje algoritmus, který pro každou vstupní formuli φ doběhne a odpoví, zda $\varphi \in T$
 - třída L -struktur $K \subseteq M_L$ je rekurzivně axiomatizovatelná, pokud existuje rekurzivně axiomatizovaná L -teorie T taková, že $K = M_L(T)$
 - teorie T' je rekurzivně axiomatizovatelná, pokud je rekurzivně axiomatizovatelná třída jejích modelů, neboli pokud je T' ekvivalentní nějaké rekurzivně axiomatizované teorii
 - řekneme, že teorie T má rekurzivně spočetnou kompletaci, pokud (nějaká) množina (až na ekvivalenci) všech jednoduchých kompletních extenzí teorie T je rekurzivně spočetná, tj. existuje algoritmus, který pro danou vstupní dvojici přirozených čísel (i, j) vypíše na výstup i -tý axiom j -té extenze (v nějakém pevně daném uspořádání), nebo odpoví, že takový axiom už neexistuje
- tvrzení: je-li T rekurzivně axiomatizovaná, potom je částečně rozhodnutelná, je-li navíc kompletní, je rozhodnutelná
- tvrzení: je-li $\mathcal{A}$ konečná struktura v konečném jazyce s rovností, potom je teorie $\text{Th}(\mathcal{A})$ rekurzivně axiomatizovatelná
- tvrzení: pokud je teorie T rekurzivně axiomatizovaná a má rekurzivně spočetnou kompletaci, potom je T rozhodnutelná
- nerozhodnutelnost
 - mějme formuli φ ve tvaru $(\exists x_1) \dots (\exists x_n) p(x_1, \dots, x_n) = q(x_1, \dots, x_n)$
 - * kde p a q jsou polynomy s přirozenými koeficienty
 - * věta: neexistuje algoritmus, který by pro danou dvojici polynomů s přirozenými koeficienty rozhodl, zda mají přirozené řešení, tj. zda platí $\mathbb{N} \models \varphi$
 - věta: neexistuje algoritmus, který by pro danou vstupní formuli φ rozhodl, zda je logicky platná (tedy zda je tautologie)
- příklady rozhodnutelných teorií
 - teorie čisté rovnosti (bez axiomů, v jazyce $L = \langle \rangle$ s rovností)
 - teorie unárního predikátu (bez axiomů, v jazyce $L = \langle U \rangle$ s rovností, kde U je unární relační symbol)
 - teorie hustých lineárních uspořádání DeLO*
 - teorie komutativních grup
 - teorie Booleových algeber
 - teorie následníka s nulou
 - Presburgerova aritmetika
- příklady nerozhodnutelných teorií
 - Robinsonova aritmetika (a také Peanova aritmetika a ZFC, což jsou její extenze)
 - teorie (obecných) grup

5 Část II: Společná informatika

5.1 1. Automaty a jazyky

- Regulární jazyky
 - deterministický konečný automat (DFA) je pětice $A = (Q, \Sigma, \delta, q_0, F)$
 - $Q \dots$ konečná množina stavů
 - $\Sigma \dots$ konečná neprázdná množina vstupních symbolů (abeceda)
 - $\delta : Q \times \Sigma \rightarrow Q \dots$ přechodová funkce
 - $q_0 \in Q \dots$ počáteční stav
 - $F \subseteq Q \dots$ množina koncových (přijímajících) stavů
 - jazykem rozpoznávaným (přijímaným) deterministickým konečným automatem $A = (Q, \Sigma, \delta, q_0, F)$ nazveme jazyk $L(A) = \{ w \mid w \in \Sigma^* \wedge \delta^*(q_0, w) \in F \}$
 - slovo je přijímáno automatem $A \equiv w \in L(A)$
 - jazyk L je rozpoznatelný konečným automatem $\equiv$ existuje konečný automat A takový, že $L = L(A)$
 - třídu jazyků rozpoznatelných konečnými automaty označíme $\mathcal{F}$, nazveme regulární jazyky
 - iterační (pumping) lemma
 - nechť L je regulární jazyk
 - $(\exists n \in \mathbb{N})(\forall w \in L) : |w| \geq n \implies w = xyz$
 - * $y \neq \epsilon$
 - * $|xy| \leq n$
 - * $\forall k \geq 0 : xy^kz \in L$
 - věta: neregularita $L_{01} = \{ 0^i 1^i \mid i \geq 0 \}$
 - předpokládejme regularitu L_{01}
 - vezměme n z pumping lemmatu
 - zvolme $w = 0^n 1^n$
 - rozdělme $w = xyz$ dle pumping lemmatu
 - * $|xy| \leq n$ je na začátku w , takže obsahuje jen nuly
 - * $y \neq \epsilon$
 - z pumping lemmatu $xz \in L_{01}$, což je spor, protože xz obsahuje méně nul než jedniček
 - kongruence
 - mějme konečnou abecedu Σ a relaci ekvivalence $\sim$ na Σ^*
 - $\sim$ je pravá kongruence $\equiv (\forall u, v, w \in \Sigma^*) : u \sim v \implies uw \sim vw$
 - $\sim$ je konečného indexu $\equiv$ rozklad $\Sigma^* / \sim$ má konečný počet tříd
 - třídu kongruence $\sim$ obsahující slovo u značíme $[u]_{\sim}$ nebo $[u]$
 - příklady
 - * relace „končí stejným písmenem“ je pravá kongruence
 - * relace „mají stejný počet znaků“ není konečného indexu
 - Myhill-Nerodova věta
 - nechť L je jazyk nad konečnou abecedou Σ
 - potom následující trzení jsou ekvivalentní:
 - * L je rozpoznatelný konečným automatem
 - * existuje pravá kongruence $\sim$ konečného indexu nad Σ^* tak, že L je sjednocením jistých tříd rozkladu $\Sigma^* / \sim$
 - použití k důkazu neregularity
 - * ukážeme pro pumpovatelný jazyk slov ve tvaru $a^+ b^i c^i$ nebo $b^i c^j$

- * pro regulární L musí existovat pravá kongruence konečného indexu m
- * mějme množinu řetězců $S = \{ab, abb, abbb, \dots, ab^{m+1}\}$
- * $|S| = m + 1$, tedy nutně existují dvě slova $i \neq j$, která padnou do stejné třídy
 - $ab^i \sim ab^j$
 - tedy $ab^i c^i \sim ab^j c^i$
 - ale $ab^i c^i \in L$ a $ab^j c^i \notin L$, což je spor, takže jazyk není regulární
- věta: jazyk L je rozpoznatelný ϵ NFA, právě když je L regulární
 - pro libovolný ϵ NFA N zkonstruuujeme DFA D přijímající stejný jazyk jako N
 - nové stavy jsou ϵ -uzavřené podmnožiny Q_N
- Deterministický a nedeterministický konečný automat
 - deterministický konečný automat (DFA) je pětice $A = (Q, \Sigma, \delta, q_0, F)$
 - $Q \dots$ konečná množina stavů
 - $\Sigma \dots$ konečná neprázdná množina vstupních symbolů (abeceda)
 - $\delta : Q \times \Sigma \rightarrow Q \dots$ přechodová funkce
 - $q_0 \in Q \dots$ počáteční stav
 - $F \subseteq Q \dots$ množina koncových (přijímajících) stavů
 - nedeterministický konečný automat s ϵ přechody je pětice $A = (Q, \Sigma, \delta, q_0, F)$
 - $Q \dots$ konečná množina stavů
 - $\Sigma \dots$ konečná množina vstupních symbolů (abeceda)
 - $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q) \dots$ přechodová funkce vracející podmnožinu Q
 - $q_0 \in Q \dots$ počáteční stav
 - * alternativa: množina počátečních stavů $S_0 \subseteq Q$
 - $F \subseteq Q \dots$ množina koncových (přijímajících) stavů
 - ϵ -uzávěr $\dots$ všechny stavy, kam se z daného stavu dostaneme prázdným slovem
 - podmnožinová konstrukce
 - věta: jazyk L je rozpoznatelný ϵ NFA, právě když je L regulární
 - pro libovolný ϵ NFA N zkonstruuujeme DFA D přijímající stejný jazyk jako N
 - nové stavy jsou ϵ -uzavřené podmnožiny Q_N
 - * $Q_D \subseteq \mathcal{P}(Q_N)$
 - * $\forall S \subseteq Q_N : \epsilon\text{closure}(S) \in Q_D$
 - * v Q_D může být i $\emptyset$
 - počáteční stav je ϵ -uzávěr q_0
 - * $q_{D_0} = \epsilon\text{closure}(q_{N_0})$
 - přijímající stavy jsou všechny množiny obsahující nějaký přijímající stav
 - * $F_D = \{S \in Q_D : S \cap F_N \neq \emptyset\}$
 - přechodová funkce sjednotí a ϵ -uzavře přechody z jednotlivých stavů
 - * pro $S \in Q_D, x \in \Sigma$ definujeme $\delta_D(S, x) = \epsilon\text{closure}\left(\bigcup_{p \in S} \delta_N(p, x)\right)$
 - Regulární výrazy
 - regulární výrazy
 - $\text{RegE}(\Sigma) \dots$ množina všech regulárních výrazů nad konečnou neprázdnou abecedou
 - $L(\alpha) \dots$ hodnota regulárního výrazu α
 - regulární výraz a jeho hodnota jsou definovány induktivně
 - * základ
 - $\epsilon \dots$ prázdný řetězec, $L(\epsilon) = \{\epsilon\}$
 - $\emptyset \dots$ prázdný výraz, $L(\emptyset) = \emptyset$
 - $a \dots$ znak $a \in \Sigma$, $L(a) = \{a\}$

- * indukce
 - $\alpha + \beta \dots$ jako OR, $L(\alpha + \beta) = L(\alpha) \cup L(\beta)$
 - $\alpha\beta \dots$ $L(\alpha\beta) = L(\alpha)L(\beta)$
 - $\alpha^* \dots$ $L(\alpha^*) = L(\alpha)^*$
 - $(\alpha) \dots$ závorky nemění hodnotu, $L((\alpha)) = L(\alpha)$
- nejvyšší prioritu má iterace $*$, pak zřetězení, pak sjednocení $+$
- třída $\text{RegE}(\Sigma)$ je nejmenší třída uzavřená na uvedené operace
- Kleeneho věta
 - každý jazyk reprezentovaný konečným automatem lze zapsat jako regulární výraz
 - každý jazyk popsany regulárním výrazem lze zapsat jako ϵ NFA
- Gramatika, jazyk generovaný gramatikou
 - formální (generativní) gramatika je čtveřice $G = (V, T, P, S)$
 - $V \dots$ konečná množina neterminálů (variables)
 - $T \dots$ neprázdná konečná množina terminálních symbolů (terminálů)
 - $S \in V \dots$ počáteční symbol
 - $P \dots$ konečná množina pravidel (produkcí) reprezentující rekurzivní definici jazyka
 - * každé pravidlo má tvar $\beta A \gamma \rightarrow \omega$
 - $A \in V$
 - $\beta, \gamma, \omega \in (V \cup T)^*$
 - * tj. levá strana obsahuje aspoň jeden neterminální symbol
 - jazyk $L(G)$ generovaný gramatikou $G = (V, T, P, S)$ je množina terminálních řetězců, pro které existuje derivace ze startovního symbolu $L(G) = \{ w \in T^* : S \Rightarrow_G^* w \}$
 - jazyk neterminálu $A \in V$ definujeme $L(A) = \{ w \in T^* : A \Rightarrow_G^* w \}$
- Zásobníkový automat, třída jazyků přijímaných zásobníkovými automaty
 - zásobníkový automat (PDA) je sedmice $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
 - $Q \dots$ konečná množina stavů
 - $\Sigma \dots$ neprázdná konečná množina vstupních symbolů
 - $\Gamma \dots$ neprázdná konečná zásobníková abeceda
 - $\delta \dots$ přechodová funkce
 - * $\delta : Q \times (\Sigma \cup \{ \epsilon \}) \times \Gamma \rightarrow \mathcal{P}_{FIN}(Q \times \Gamma^*)$
 - kde $\mathcal{P}_{FIN}(A)$ je množina všech konečných podmnožin množiny A
 - * tedy $\delta(p, a, X) \ni (q, \gamma)$
 - q je nový stav
 - γ je řetězec zásobníkových symbolů, který nahradí X na vrcholu zásobníku
 - $q_0 \in Q \dots$ počáteční stav
 - $Z_0 \in \Gamma \dots$ počáteční zásobníkový symbol
 - $F \dots$ množina přijímajících (koncových) stavů (může být nedefinovaná)
 - jak PDA funguje
 - * je nedeterministický
 - * v jednom časovém kroku
 - na vstupu přečte žádný nebo jeden symbol
 - přejde do nového stavu
 - nahradí symbol na vrchu zásobníku libovolným řetězcem (nejlevější znak bude na vrchu)
 - přechodový diagram
 - * jako pro konečný automat

- * hrany popisujeme ve tvaru: *vstupní znak, zásobníkový znak* $\rightarrow$ *push řetězec*
- jazyk přijímaný koncovým stavem je $L(P_{da}) = \{ w \in \Sigma^* : (\exists q \in F)(\exists \alpha \in \Gamma^*)((q_0, w, Z_0) \vdash_{P_{da}}^* (q, \epsilon, \alpha)) \}$
- jazyk přijímaný prázdným zásobníkem je $N(P_{da}) = \{ w \in \Sigma^* : (\exists q \in Q)((q_0, w, Z_0) \vdash_{P_{da}}^* (q, \epsilon, \epsilon)) \}$
 - množina F je nerelevantní, takže ji lze vynechat - pak je PDA šestice
- lemma: od přijímajícího stavu k prázdnému zásobníku
 - na dno zásobníku přidáme špunt
 - z přijímajících stavů vedeme hranu do vypouštěcího stavu, kde zásobník vyprázdníme
- lemma: od prázdného zásobníku ke koncovému stavu
 - na dno zásobníku přidáme špunt
 - z každého stavu uděláme přechod takový, že pokud je vidět špunt, přemístíme se do koncového stavu
- věta: následující tvrzení jsou ekvivalentní
 - jazyk L je bezkontextový, tj. generovaný bezkontextovou gramatikou
 - jazyk L je přijímaný nějakým zásobníkovým automatem koncovým stavem
 - jazyk L je přijímaný nějakým zásobníkovým automatem prázdným zásobníkem
- konstrukce PDA z CFG
 - mějme gramatiku $G = (V, T, P, S)$
 - konstruujeme PDA $P = (\{q\}, T, V \cup T, \delta, q, S)$
 - * $\forall A \in V : \delta(q, \epsilon, A) = \{ (q, \beta) \mid (A \rightarrow \beta) \in P \}$
 - * $\forall a \in T : \delta(q, a, a) = \{ (q, \epsilon) \}$
 - * P přijímá prázdným zásobníkem
 - idea
 - * stavy nás nezajímají - stačí nám jeden
 - * na zásobníku nedeterministicky generujeme všechny možné posloupnosti terminálů (přičemž ale postupně zpracováváme vstup)
- Pumping lemma pro bezkontextové jazyky
 - lemma o vkládání (pumping) pro bezkontextové jazyky
 - nechť L je bezkontextový jazyk
 - $(\exists n)(\forall w \in L) : |w| \geq n \implies w = u_1 u_2 u_3 u_4 u_5$
 - * $u_2 u_4 \neq \epsilon$
 - * $|u_2 u_3 u_4| \leq n$
 - * $\forall k \geq 0 : u_1 u_2^k u_3 u_4^k u_5 \in L$
 - idea důkazu
 - vezmeme derivační strom pro w
 - najdeme nejdelší cestu, na ní dva stejné neterminály
 - tyto neterminály určí dva podstromy, které definují rozklad slova
 - větší podstrom můžeme posunout ($k > 1$) nebo nahradit menším podstromem ($k = 0$)
 - příklad: $L_{012} = \{ 0^i 1^i 2^i \mid i \geq 1 \}$ není bezkontextový
 - důkaz sporem - předpokládejme bezkontextovost
 - vezmeme n z pumping lemmatu
 - zvolíme $w = 0^n 1^n 2^n$
 - délka pumpovacího slova $u_2 u_3 u_4$ musí být nejvýše n , tedy vždy můžeme pumpovat maximálně dva různé symboly
 - $u_2 u_4 \neq \epsilon \implies$ iterací se slovo změní
 - tím se poruší rovnost symbolů

- tedy ať už zvolíme libovolné dělení $u_1u_2u_3u_4u_5$, nové slovo nebude patřit do L_{012} , což je spor
- Chomského normální tvar
 - o bezkontextové gramatice $G = (V, T, P, S)$ bez zbytečných symbolů, kde jsou všechna pravidla ve tvaru $A \rightarrow BC$ nebo $A \rightarrow a$, kde $A, B, C \in V$, $a \in T$, říkáme, že je v Chomského normálním tvaru (ChNF)
 - algoritmus převodu
 - eliminujeme ϵ -pravidla
 - * označíme nulovatelné symboly
 - * přidáme verzi pravidel bez nulovatelných symbolů
 - * odstraníme ϵ -pravidla
 - eliminujeme jednotková pravidla (tím nepřidáme ϵ -pravidla)
 - * najdeme jednotkové páry
 - * přidáme odpovídající pravidla
 - * odstraníme jednotková pravidla
 - eliminujeme zbytečné symboly (tím nepřidáme žádná pravidla)
 - * nejdříve eliminujeme negenerující, pak nedosažitelné
 - pravé strany délky aspoň 2 předěláme na samé neterminály
 - pravé strany s aspoň třemi neterminály rozdělíme na více pravidel
- Turingův stroj
 - turingův stroj (TM) je sedmice $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$
 - $Q \dots$ konečná množina stavů
 - $\Sigma \dots$ konečná neprázdná množina vstupních symbolů
 - $\Gamma \dots$ konečná množina všech symbolů pro pásku
 - * vždy $\Gamma \supseteq \Sigma$, $Q \cap \Gamma = \emptyset$
 - $\delta \dots$ (částečná) přechodová funkce
 - * zobrazení $(Q - F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
 - * $\delta(q, X) = (p, Y, D)$
 - $q \in (Q - F) \dots$ aktuální stav
 - $X \in \Gamma \dots$ aktuální symbol na pásce
 - $p \in Q \dots$ nový stav
 - $Y \in \Gamma \dots$ symbol pro zapsání do aktuální buňky, přepíše aktuální obsah
 - $D \in \{L, R\} \dots$ směr pohybu hlavy (doleva, doprava)
 - $q_0 \in Q \dots$ počáteční stav
 - $B \in \Gamma - \Sigma \dots$ blank = symbol pro prázdné buňky, na začátku všude kromě konečného počtu buněk se vstupem
 - $F \subseteq Q \dots$ množina koncových neboli přijímajících stavů
 - poznámka: někdy se nerozlišuje Γ a Σ a neuvádí se B , pak je TM pětice
 - turingův stroj $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ přijímá jazyk $L(M) = \{w \in \Sigma^* : (\exists p \in F)(\exists \alpha, \beta \in \Gamma^*)(q_0w \vdash_M^* \alpha p \beta)\}$
 - tj. množinu slov, po jejichž přečtení se dostane do koncového stavu
 - pásku nemusí uklízet (v naší definici)
 - jazyk nazveme rekurzivně spočteným, pokud je přijímán nějakým Turingovým strojem
 - říkáme, že TM M rozhoduje jazyk L , pokud $L = L(M)$ a pro každé $w \in \Sigma^*$ stroj nad w zastaví
 - jazyky rozhodnutelné TM nazýváme rekurzivní jazyky
 - rekurzivní jazyky jsou podmnožinou rekurzivně spočtených jazyků

- Algoritmicky nerozhodnutelné problémy
 - rozhodnutelný problém
 - problém ... množina otázek (nebo spíše vstupů) kódovatelná řetězci nad abecedou Σ s odpověďmi $\in \{ \text{ano}, \text{ne} \}$
 - * pokud problém definuji jako množinu, jde o otázku, zda vstup kóduje prvek dané množiny (např. polynom s celočíselným kořenem)
 - problém je (algoritmicky) rozhodnutelný, pokud existuje TM takový, že pro každý vstup $w \in P$ TM zastaví a navíc přijme, právě když $P(w) = \text{ano}$ (tj. pro $P(w) = \text{ne}$ zastaví v nepřijímajícím stavu)
 - nerozhodnutelný problém ... není rozhodnutelný
 - existuje analogie mezi rozhodnutelným problémem a rekurzivním jazykem
 - redukce problému
 - definice: redukcí problému P_1 na P_2 nazýváme algoritmus R , který pro každou instanci $w \in P_1$ zastaví a vydá $R(w) \in P_2$, přičemž...
 - * $P_1(w) = \text{ano} \iff P_2(R(w)) = \text{ano}$
 - * tedy i $P_1(w) = \text{ne} \iff P_2(R(w)) = \text{ne}$
 - věta: existuje-li redukce problému P_1 na P_2 , pak...
 - * pokud P_1 je nerozhodnutelný, pak je nerozhodnutelný i P_2
 - * pokud P_1 není rekurzivně spočetný, pak není rekurzivně spočetný ani P_2
 - problém zastavení není rozhodnutelný
 - definice: instanci problému zastavení je dvojice řetězců $M, w \in \{0, 1\}^*$
 - definice: problém zastavení je najít algoritmus $\text{Halt}(M, w)$, který vydá 1, právě když stroj M zastaví na vstupu w , jinak vydá 0
 - věta: problém zastavení není rozhodnutelný
 - idea důkazu: redukuje se na Halt
 - diagonální jazyk
 - diagonální jazyk L_d je definovaný jako jazyk slov $w \in \{0, 1\}^*$ takových, že TM reprezentovaný jako w nepřijímá slovo w
 - věta: L_d není rekurzivně spočetný jazyk, tj. neexistuje TM přijímající L_d
 - idea důkazu
 - * kdyby existoval TM přijímající L_d , spuštění takového stroje na vlastním kódu by vedlo k paradoxu
 - univerzální jazyk L_u definujeme jako množinu binárních řetězců, které kódují pár (M, w) , kde M je TM a $w \in L(M)$
 - TM přijímající L_u se nazývá *univerzální Turingův stroj*
 - existuje
 - problém univerzálního jazyka není rozhodnutelný
 - věta: L_u je rekurzivně spočetný, ale není rekurzivní
 - mohli bychom zkonstruovat TM přijímající L_d
 - problém prázdnoty jazyka daného TM není rozhodnutelný
- Chomského hierarchie
 - gramatiky typu 0 (rekurzivně spočetné jazyky $\mathcal{L}_0$)
 - pravidla v obecné formě $\alpha \rightarrow \omega$
 - * $\alpha, \omega \in (V \cup T)^*$
 - * α obsahuje neterminál
 - rekurzivně spočetný jazyk je rozpoznatelný (nějakým) Turingovým strojem
 - gramatiky typu 1 (kontextové gramatiky, jazyky $\mathcal{L}_1$)

- pouze pravidla ve tvaru $\gamma A \beta \rightarrow \gamma \omega \beta$
 - * $A \in V$
 - * $\gamma, \beta \in (V \cup T)^*$
 - * $\omega \in (V \cup T)^+$
 - * jedinou výjimkou je pravidlo $S \rightarrow \epsilon$, pak se ale S nevyskytuje na pravé straně žádného pravidla
- kontextový jazyk je rozpoznatelný lineárně omezeným automatem (LBA, lineárně omezený Turingův stroj)
- gramatiky typu 2 (bezkontextové gramatiky, jazyky $\mathcal{L}_2$)
 - pouze pravidla ve tvaru $A \rightarrow \omega$
 - * $A \in V$
 - * $\omega \in (V \cup T)^*$
 - bezkontextový jazyk je rozpoznatelný nedeterministickým zásobníkovým automatem
- gramatiky typu 3 (regulární gramatiky / pravé lineární gramatiky, regulární jazyky $\mathcal{L}_3$)
 - pouze pravidla ve tvaru $A \rightarrow \omega B$ nebo $A \rightarrow \omega$
 - * $A, B \in V$
 - * $\omega \in T^*$
 - idea
 - * každý neterminál odpovídá stavu konečného automatu
 - * pravidla odpovídají přechodové funkci
 - regulární jazyk je rozpoznatelná konečným automatem
- Schopnost zařazení konkrétního jazyka do Chomského hierarchie (zpravidla sestavení odpovídajícího automatu či gramatiky)
 - obvykle chceme jazyk zařadit co nejpřesněji - např. říct, že není regulární (dokázat pomocí iteračního lemmatu) a že je bezkontextový (sestavit CFG)
 - je regulární: sestavíme konečný automat
 - není regulární: použijeme iterační lemma nebo Myhill-Nerodovu větu
 - je bezkontextový: sestavíme bezkontextovou gramatiku
 - není bezkontextový: použijeme lemma o vkládání
 - kontextovostí se obvykle nezabýváme
 - lemma: jazyk $L = \{ a^i b^j c^k \mid 1 \leq i \leq j \leq k \}$ je kontextový jazyk, není bezkontextový
 - je rekurzivně spočetný: popíšeme algoritmus
 - algoritmus pro $0^n 1^n$
 - * jezdíme tam a zpátky po pásce, přičemž 0 přepíšeme na X , znak 1 přepíšeme na Y
 - * přijmeme, pokud nám jedničky a nuly dojdou ve stejnou chvíli (páska je popsána samými X a Y)
 - není rekurzivně spočetný (nebo případně jenom rekurzivní): dokážeme pomocí L_d
 - uzavřenost operací
 - regulární jazyky jsou uzavřené na množinové (sjednocení, průnik, rozdíl, doplněk) i řetězcové operace ($L.M, L^*, L^+, L^R, M \setminus L, L/M$), na homomorfismus a inverzní homomorfismus
 - * doplněk: obrátíme „konecovost“ stavů
 - * průnik, sjednocení, rozdíl
 - zkonstruuje součinový automat $(Q_1 \times Q_2, \Sigma, \delta, (q_{01}, q_{02}), F)$
 - kde $\delta((p, q), x) = (\delta_1(p, x), \delta_2(q, x))$
 - průnik ... $F = F_1 \times F_2$
 - sjednocení ... $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

- rozdíl ... $F = F_1 \times (Q_2 - F_2)$
- bezkontextové jazyky jsou uzavřené na sjednocení, konkatenaci, iteraci, reverzi, naopak neuzavřené na průnik (ale jsou uzavřené na průnik s regulárním jazykem)
- uzavřenost můžeme použít k důkazu regularity nebo bezkontextovosti
- ale můžeme ji použít i k důkazu neregularity jazyka L - kdybychom aplikací uzavřených operací na jazyk L a regulární jazyky dostali jazyk L' , který zjevně není regulární (např. $0^i 1^i$)
- Příklad kontextového jazyka
 - lemma: jazyk $L = \{ a^i b^j c^k \mid 1 \leq i \leq j \leq k \}$ je kontextový jazyk, není bezkontextový
 - pravidla
 - $S \rightarrow aSBC \mid aBC \dots$ generování symbolů a
 - $B \rightarrow BBC \dots$ množení symbolů B
 - $C \rightarrow CC \dots$ množení symbolů C
 - $CB \rightarrow BC \dots$ uspořádání symbolů B a C
 - * pravidlo není kontextové, je potřeba ho upravit nadtrháváním
 - * tzn. písmenka měníme po jednom (střídavě), např. takto: $CB \rightarrow \bar{C}B \rightarrow \bar{C}\bar{B} \rightarrow B\bar{B} \rightarrow BC$
 - $aB \rightarrow ab \dots$ začátek přepisu B na b
 - $bB \rightarrow bb \dots$ pokračování přepisu B na b
 - $bC \rightarrow bc \dots$ začátek přepisu C na c
 - $cC \rightarrow cc \dots$ pokračování přepisu C na c
 - je důležité, že tam chybí $cB \rightarrow cb$

5.2 2. Algoritmy a datové struktury

- Časová a prostorová složitost algoritmu
 - pro různé úlohy používáme různé parametrizace vstupu (tedy velikost vstupu může být trochu něco jiného - někdy počet čísel, někdy jejich hodnota...)
 - doba běhu pro vstup x
 - $t(x) :=$ součet cen provedených instrukcí (může být ∞)
 - časová složitost výpočtu
 - $T(n) := \max\{t(x) \mid x \text{ vstup velikosti } n\}$
 - prostor běhu pro vstup x
 - $s(x) := \text{max. adresa} - \text{min. adresa} + 1$
 - máme na mysli maximální a minimální adresy navštívené během výpočtu
 - prostorová složitost výpočtu
 - $S(n) := \max\{s(x) \mid x \text{ vstup velikosti } n\}$
 - takhle jsme definovali složitosti v nejhorším případě, někdy se může hodit i průměrný případ
- Měření velikosti dat
 - spotřebovanou paměť definujeme jako rozdíl mezi nejvyšším a nejnižším použitým indexem paměti
 - paměťová složitost pak odpovídá maximu ze spotřeby paměti přes všechny vstupy dané velikosti
 - do časové složitosti nepočítáme načtení vstupu
 - podobně do prostorové složitosti nepočítáme vstup, do nějž se nezapisuje, ani výstup, pokud se jenom zapíše a pak už se nečte (takhle se někdy definuje pracovní prostor výpočtu)
 - je potřeba omezit kapacitu paměťové buňky

- jednotková cena instrukce - omezíme šířku slova na W bitů
- logaritmická cena instrukce - cena odpovídá počtu bitů operandů včetně adres (je přesný, ale nepohodlný na přemýšlení o algoritmech)
- relativní logaritmická cena instrukce - u polynomiálně velkých čísel je cena jednotková, u obrovských čísel se cena zvyšuje (tohle budeme reálně používat - i když se k tomu budeme obvykle chovat jako k jednotkové ceně instrukce)
- Složitost v nejlepším, nejhorším a průměrném případě
 - nejčastěji se používá složitost v nejhorším případě
 - složitost pro nejhorší možný vstup
 - složitost v průměrném případě dobře charakterizuje kvalitu algoritmu, ale je obtížné ji odvodit
 - průměrujeme přes všechny možné vstupy
- Asymptotická notace
 - $f \in O(g) \equiv \exists c \forall^* n \ f(n) \leq c \cdot g(n)$
 - $f, g : \mathbb{N} \rightarrow \mathbb{R}$
 - $\forall^* n \dots$ pro všechna n až na konečně mnoho výjimek (existuje n_0 takové, že pro všechna větší n už to platí)
 - $f \in \Omega(g) \equiv \exists c \forall^* n \ f(n) \geq c \cdot g(n)$
 - $\Theta(g) := O(g) \cap \Omega(g)$
- Třídy P a NP
 - problém $L \in P \equiv \exists A$ algoritmus $\exists p$ polynom takový, že $\forall x$ vstup platí, že $A(x)$ doběhne do $p(|x|)$ kroků $\wedge A(x) = L(x)$
 - problém $L \in NP \equiv \exists V \in P$ (verifikátor) $\exists g$ polynom (omezení délky certifikátů) $\forall x : L(x) = 1 \iff \exists y$ certifikát $|y| \leq g(|x|)$ (krátký) $\wedge V(x, y) = 1$ (schválený)
 - $P \subseteq NP$ (rovnost se neví)
- Převoditelnost problémů, NP-těžkost a NP-úplnost
 - rozhodovací problém $\equiv$ funkce $f : \{0, 1\}^* \rightarrow \{0, 1\}$
 - převod
 - mějme rozhodovací problémy A, B
 - problém A je převoditelný na problém B právě tehdy, když existuje funkce $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ taková, že $\forall \alpha \in \{0, 1\}^* : A(\alpha) = B(f(\alpha))$ a f lze spočítat v čase polynomiálním vzhledem k $|\alpha|$
 - značíme $A \rightarrow B$ nebo $A \leq_P B$
 - funkci f říkáme převod (případně redukce)
 - vlastnosti převoditelnosti
 - je reflexivní ($A \rightarrow A$) $\dots$ f je identita
 - je tranzitivní ($A \rightarrow B \wedge B \rightarrow C \implies A \rightarrow C$) $\dots$ když f převádí A na B a g převádí B na C , tak $f \circ g$ převádí A na C (přičemž složení polynomiálně vyčíslitelných funkcí je polynomiálně vyčíslitelná funkce)
 - není antisymetrická - např. problémy „vstup má sudou délku“ a „vstup má lichou délku“ lze mezi sebou převádět oběma směry
 - existují navzájem nepřevoditelné problémy - např. mezi problémy „na každý vstup odpověz 0“ a „na každý vstup odpověz 1“ nemůže existovat převod
 - převoditelnost je částečné kvaziuspořádání na množině všech problémů
 - definice: NP-těžké a NP-úplné problémy
 - L je NP-těžký $\equiv \forall K \in NP : K \rightarrow L$
 - L je NP-úplný $\equiv L$ je NP-těžký $\wedge L \in NP$
 - věta: pokud $A \rightarrow B$ a $B \in P$, pak $A \in P$

- věta: pokud $A \rightarrow B$, $B \in \text{NP}$ a A je NP-úplný, pak B je NP-úplný
- Příklady NP-úplných problémů a převodů mezi nimi
 - logické: (CNF) SAT, 3-SAT, 3,3-SAT, obvodový SAT
 - grafové: nezávislá množina, klika, k -obarvitelnost (pro $k \geq 3$), hamiltonovská cesta/kružnice, 3D-párování
 - číselné: součet podmnožiny, batoh, 2 loupežníci, nulajedničkové řešení soustavy lineárních rovnic (v $\mathbb{Z}$)
 - popisy problémů
 - klika: existuje úplný podgraf grafu G na alespoň k vrcholech?
 - nezávislá množina: existuje nezávislá množina vrcholů grafu G velikosti aspoň k ?
 - * množina vrcholů grafu je nezávislá $\equiv$ žádné dva vrcholy ležící v této množině nejsou spojeny hranou
 - SAT: existuje dosazení 0 a 1 za proměnné tak, aby $\psi(\dots) = 1$?
 - * kde ψ je v CNF
 - 3-SAT: jako SAT, akorát každá klauzule formule ψ obsahuje nejvýše tři literály
 - 3,3-SAT: jako 3-SAT, akorát se každá proměnná vyskytuje v maximálně třech literálech
 - 3D-párování
 - * vstup: tři množiny A, B, C a množina kompatibilních trojic $T \subseteq A \times B \times C$
 - * výstup: existuje perfektní podmnožina trojic? tzn. existuje taková podmnožina, v níž se každý prvek množin A, B, C účastní právě jedné trojice
 - převod klika $\leftrightarrow$ nezávislá množina
 - pokud v grafu prohodíme hrany a nehrany, stane se z každé kliky nezávislá množina a naopak
 - převodní funkce zneguje hrany
 - při převodu problémů typu SAT vyrábíme ekvivalentní formuli
 - SAT $\rightarrow$ 3-SAT
 - $(\alpha \vee \beta) \rightarrow (\alpha \vee \zeta) \wedge (\beta \vee \neg \zeta)$
 - převod funguje i naopak (viz rezoluce)
 - jak rozštípnout dlouhou klauzuli délky ℓ ?
 - * α nechť má délku 2
 - * β nechť má délku $\ell - 2$
 - * po přidání ζ dostanu konjunkci klauzulí délky 3 a $\ell - 1$
 - * klauzuli délky $\ell - 1$ štípu dál (pokud je moc dlouhá)
 - počet štípnutí je shora omezen délkou formule
 - v polynomiálním čase postupně rozštípeme všechny dlouhé klauzule při zachování splnitelnosti
 - 3-SAT $\rightarrow$ 3,3-SAT
 - nechť x je proměnná s $k > 3$ výskyty
 - pro každý výskyt si pořídíme nové proměnné $x_1, \dots, x_k$
 - ekvivalenci všech x_i zajistíme řetězcem implikací
 - * $x_1 \implies x_2$
 - * $x_2 \implies x_3$
 - * $\vdots$
 - * $x_k \implies x_1$
 - každá proměnná x_i se tudíž vyskytne třikrát
 - 3,3-SAT* ... navíc každý literál max. 2×

- použijeme předchozí algoritmus pro všechny proměnné s $k \geq 3$ výskyty
- 3-SAT $\rightarrow$ nezávislá množina
 - pro každou z k klauzulí formule vytvoříme trojúhelník a jeho vrcholům přiřadíme literály klauzule
 - * pokud klauzule obsahuje méně než tři literály, tak přebývajících vrcholy smažeme
 - spojíme hranami všechny dvojice konfliktních literálů
 - v takovém grafu existuje nezávislá množina velikosti alespoň k právě tehdy, když je formule splnitelná
 - * máme-li splňující ohodnocení, můžeme z každé klauzule vybrat jeden pravdivý literál - ty umístíme do nezávislé množiny
 - * máme-li nezávislou množinu velikosti k , vybereme literály odpovídající těmto vrcholům a podle nich nastavíme odpovídající proměnné
 - v nezávislé množině velikosti k nemůžou být dva vrcholy ze stejného trojúhelníku - tedy z každého trojúhelníku (klauzule) bude v množině právě jeden vrchol
 - v nezávislé množině nemohou být dva vrcholy s opačnými literály, protože takové vrcholy jsou spojeny hranou
- nezávislá množina $\rightarrow$ 3-SAT
 - každému vrcholu odpovídá jedna proměnná (pokud je proměnná ohodnocena jedničkou, vrchol náleží do nezávislé množiny)
 - pro každou hranu přidáme klauzuli $(\neg x_i \vee \neg x_j)$, která zajistí, že obě proměnné nebudou splněny zároveň
 - chceme nezávislou množinu velikosti k , takže musíme vrcholy v množině spočítat
 - * pořídíme si tabulku (pomocí jiných proměnných y_{ij})
 - * vynutíme, aby v každém sloupci byla maximálně jedna jednička a v každém řádku právě jedna jednička
 - propojíme proměnné x_j a y_{ij} pomocí implikace $y_{ij} \rightarrow x_j$
- Metoda rozděl a panuj: princip rekurzivního dělení problému na podproblémy
 - problém rekurzivně dělíme na menší a menší podproblémy, dokud nedojdeme k problému konstantní velikosti, který umíme vyřešit triviálně
 - problémy v jedné úrovni dělení na sobě musí být nezávislé
- Metoda rozděl a panuj: výpočet složitosti pomocí rekurentních rovnic
 - vyjádříme složitost algoritmu (tedy $T(n)$) pomocí složitosti podproblému, tím dostaneme rekurzivní rovnici
 - do rovnice můžeme postupně dosazovat a vypočítávat nějaký obecný vzorec - chceme dojít až k $T(1)$
 - nebo můžeme rovnou použít Master theorem
 - případně ze složitosti i -té úrovně můžeme odvodit složitost celého algoritmu (součtem přes všechny úrovně)
- Master theorem (kuchařková věta) - bez důkazu
 - uvažujeme rekurzivní algoritmus, který vstup rozloží na a podproblémů velikosti n/b a z jejich výsledků složí celkovou odpověď v čase $\Theta(n^c)$
 - věta: rekurentní rovnice $T(n) = a \cdot T(n/b) + \Theta(n^c)$, $T(1) = 1$ má pro konstanty $a \in \{1, 2, \dots\}$, $b \in (1, \infty)$, $c \in [0, \infty)$ řešení...
 - $T(n) = \Theta(n^c \log n)$, pokud $a/b^c = 1$
 - $T(n) = \Theta(n^c)$, pokud $a/b^c < 1$
 - $T(n) = \Theta(n^{\log_b a})$, pokud $a/b^c > 1$
- Metoda rozděl a panuj: aplikace (Mergesort, násobení dlouhých čísel)
 - Mergesort

- na vstupu posloupnost
- pokud je délka posloupnosti menší rovna jedné, mergesort vrátí vstup
- jinak vrátí merge(mergesort(první polovina vstupu), mergesort(druhá polovina vstupu)), kde merge slévá dvě poloviční setříděné posloupnosti v lineárním čase
- čas $\Theta(n \log n)$
 - * v každém z $\log n$ pater (lineárně) sléváme kousky posloupnosti
 - * v každém patře se kousky sečtou na n
- násobení n -ciferných čísel v čase $O(n^{\log_2 3})$
 - Karacubovo násobení
 - násobíme dvě n -ciferná čísla x, y v soustavě o základu z
 - cifry obou čísel rozdělíme na dvě poloviny
 - * $x = a \cdot z^{n/2} + b$
 - * $y = c \cdot z^{n/2} + d$
 - pak $x \cdot y = (a \cdot z^{n/2} + b)(c \cdot z^{n/2} + d) = ac \cdot z^n + (ad + bc) \cdot z^{n/2} + bd$
 - $(ad + bc)$ lze vyjádřit jako $(a + b)(c + d) - ac - bd$
 - takže stačí 3 násobení
 - $T(n) = 3 \cdot T(n/2) + cn$
 - časová složitost podproblému na vrstvě a počet podproblémů
 - * 1\.. vrstva n 1
 - * 2\.. vrstva $\frac{n}{2}$ 3
 - * i -tá vrstva $\frac{n}{2^i}$ 3^i
 - * poslední vrstva 1 $3^{\log_2 n}$
 - $T(n) = \sum_{i=0}^{\log_2 n} n \cdot (3/2)^i \in \Theta(n \cdot (3/2)^{\log_2 n})$, což lze převést na $\Theta(n^{\log_2 3})$
 - podle Master theorem $a = 3, b = 2, c = 1$
- Definice (binárního) vyhledávacího stromu
 - BVS je binární strom, jehož každému vrcholu v přiřadíme unikátní klíč $k(v)$ z univerza. Přitom musí pro každý vrchol v platit:
 - kdykoliv $a \in L(v)$, pak $k(a) < k(v)$
 - kdykoliv $b \in R(v)$, pak $k(b) > k(v)$
- Operace s nevyvažovanými binárními vyhledávacími stromy
 - Find: „binární vyhledávání“ (porovnávám s vrcholem - podle toho se zanořím do jednoho z podstromů nebo vrátím hodnotu vrcholu, případně $\emptyset$, pokud se nemám kam zanořit)
 - Insert: vložím vrchol tam, kde bych ho našel, kdyby ve stromu byl (pokud tam je, tak nic nedělám)
 - Delete: vrchol najdeme a smažeme; pokud má jednoho syna, tak ho připojíme pod otce smazaného vrcholu; pokud má dva syny, nejdříve nalezneme nejlevější vrchol v pravém podstromu a s tím ho prohodíme (fungovalo by to i symetricky)
- AVL stromy (definice)
 - AVL strom = hloubkově vyvážený strom
 - pro každý jeho vrchol platí $|h(L(v)) - h(R(v))| \leq 1$
 - tedy hloubka levého a pravého podstromu se liší nejvýše o jedna
 - věta: AVL strom na n vrcholech má hloubku $\Theta(\log n)$
- Primitivní třídící algoritmy (Bubblesort, Insertsort)
 - BubbleSort = výměna dvou prvků
 - procházíme pole zleva doprava
 - bereme dvojice prvků, které jsou v poli vedle sebe - pokud jsou ve špatném pořadí, tak je prohodíme

- pokud jsme během jednoho průchodu provedli aspoň jedno prohození, musíme pole projít znova (jinak algoritmus končí)
- InsertSort = zařazení prvku mezi již setříděné
 - rozdělíme pole na setříděnou a neseříděnou část (na začátku je neseříděná část prázdná)
 - postupně bereme prvky z neseříděné části a zařazujeme je na správné místo v setříděné části (najdeme vhodné místo, setříděné prvky vpravo posuneme napravo, do vzniklého místa vložíme aktuální prvek)
- Quicksort
 - určíme pivota, rozdělíme na prvky menší rovny a větší rovny
 - algoritmus QuickSort(X)
 - pokud $n \leq 1$: vrátíme vstup
 - zvolíme pivota p
 - rozdělíme $x_1, \dots, x_n$ podle p na L, S, P
 - $L \leftarrow \text{QuickSort}(L)$
 - $P \leftarrow \text{QuickSort}(P)$
 - vrátíme L, S, P
 - věta: QuickSort s náhodnou volbou pivota má časovou složitost se střední hodnotou $\Theta(n \log n)$
 - částečné zdůvodnění průměrné složitosti
 - s pravděpodobností $\frac{1}{2}$ bude náhodně vybraný pivot v prostředních dvou čtvrtinách setříděné posloupnosti („skoromedián“)
 - střední hodnota počtu pokusů, než zvolíme skoromedián, je podle lemmatu o džbánu rovna 2 (to se schová do konstanty)
 - když jsme zvolili skoromedián, tak se problém zmenšil na $\frac{3}{4}$
 - složitost v nejhorším případě je $\Theta(n^2)$
- Dolní odhad složitosti porovnávacích třídících algoritmů
 - uvažujeme strom algoritmu - jeho listy odpovídají možným výsledkům
 - větvení v rozhodovacím stromě budou odpovídat jednotlivým porovnáním
 - listů musí být $n!$
 - hloubka stromu musí být aspoň $\log_2 n! \geq \log_2 n^{\frac{n}{2}} = \frac{n}{2} \log n$
 - každý každý porovnávací třídící algoritmus musí mít složitost $\in \Omega(n \log n)$
- Prohledávání grafu do šířky a do hloubky
 - prohledávání do šířky (BFS)
 - zpracování určitého vrcholu = odeberu ho z fronty, podívám se na sousední vrcholy a pokud jsou nenavštívené, tak je označím jako otevřené a přidám je do fronty ke zpracování, zpracováváný vrchol zavřu
 - algoritmus začíná přidáním prvního vrcholu do fronty (to je ten vrchol, ze kterého graf prohledáváme)
 - algoritmus končí, jakmile je fronta prázdná
 - složitost $\Theta(n + m)$
 - BFS najde cestu s nejmenším počtem hran (z výchozího vrcholu do libovolného vrcholu grafu) - tedy lze použít k hledání nejkratší cesty v grafech s jednotkovými hranami
 - prohledávání do hloubky (DFS)
 - lze implementovat rekurzí nebo jako BFS se zásobníkem místo fronty
 - na začátku jsou všechny vrcholy neviděné
 - DFS(v):
 - * stav(v) $\leftarrow$ otevřený
 - * pro $vw \in E$

- pokud $\text{stav}(w) = \text{neviděný}$
 - DFS(w)
 - * $\text{stav}(v) \leftarrow \text{zavřený}$
- DFS spouštíme z nějaké vrcholu - navštívíme všechny vrcholy z něj dosažitelné
- opakované DFS - algoritmus nespouštíme pouze pro jeden určitý vrchol, ale pro všechny (neviděné) vrcholy grafu $\rightarrow$ projdeme všechny komponenty souvislosti
 - * stejného efektu lze docílit přidáním jednoho superzdroje, který bude připojen ke všem vrcholům grafu
- složitost $\Theta(n + m)$
- Topologické třídění orientovaných grafů
 - topologické uspořádání grafu $G = (V, E)$ je lineární uspořádání $\leq$ na V takové, že $\forall uv \in E : u \leq v$
 - může jich být víc
 - konstrukce topologického uspořádání
 - postupným odtrháváním zdrojů (vezmeme libovolný vrchol, jdeme proti šipkám do zdroje, tento zdroj umístíme do uspořádání a odtrhneme ho z grafu) - to je ale složité na efektivní implementaci
 - opakované DFS opouští (zavírá) vrcholy v pořadí opačném topologickému uspořádání
- Nejkratší cesty v ohodnocených grafech (Dijkstrův a Bellmanův-Fordův algoritmus)
 - Dijkstra
 - hledání nejkratších cest z daného vrcholu do všech vrcholů grafu
 - je to v podstatě BFS s budíky (s prioritní frontou)
 - funguje pro kladné reálné délky hran
 - začínáme od nějakého vrcholu v_0 (otevřeme ho a jeho ohodnocení $h(v_0)$ nastavíme na nulu)
 - * ostatní vrcholy v mají ohodnocení $h(v) = +\infty$
 - dokud existují nějaké otevřené vrcholy, vybereme z nich ten, jehož $h(v)$ je nejmenší (ten zavřeme) a všechny jeho následníky otevřeme a jejich $h(w)$ snížíme na $h(v) + l(v, w)$
 - ukládáme předchůdce vrcholů, aby se cesta dala rekonstruovat
 - pozorování: každý vrchol zavřeme pouze jednou
 - pokud nejmenší $h(v)$ hledáme pokaždé znova, tak to má složitost $\Theta(n^2)$
 - cena operací
 - * ExtractMin ... T_X
 - * Insert ... T_I
 - * Decrease ... T_D
 - složitost Dijkstra je $O(n \cdot T_I + m \cdot T_D + n \cdot T_X)$
 - * vkládáme každý vrchol
 - * decrease se provádí nejvýše jednou za každou hranu
 - * extractujeme každý vrchol
 - při použití binární haldy jsou všechny tři operace $O(\log n)$, z čehož vyplývá složitost Dijkstra $O((n + m) \log n)$
 - * lepší (asymptoticky) je zvolit d -regulární haldu, kde $d = m/n$
 - * ještě lepší je Fibonacciho halda
 - Bellman-Ford
 - relaxační algoritmus (je to třída algoritmů, liší se tím, jak vybírají otevřené vrcholy - patří tam i Dijkstra, který vybírá ty nejlevnější)
 - vrcholy ukládá do fronty (takže jako Dijkstra, akorát nebere nejlevnější, ale nejstarší)
 - funguje pro grafy bez záporných cyklů

- obecný relaxační algoritmus
 - * vstup: graf G , počáteční vrchol v_0
 - * na začátku jsou všechny vrcholy nenalezené, jejich ohodnocení je nekonečné, jejich předchůdci jsou nedefinováni
 - * stav počátečního vrcholu nastavíme na otevřený, jeho ohodnocení na nulu
 - * dokud existují otevřené vrcholy
 - vezmu nějaký otevřený vrchol v (v Bellmanově-Fordově algoritmu беру ten nejstarší ve frontě)
 - pro všechny jeho následníky w provedu relaxaci, tzn.:
 - pokud $h(w) > h(v) + l(v, w)$
 - ◊ $h(w) \leftarrow h(v) + l(v, w)$
 - ◊ stav(w) $\leftarrow$ otevřený
 - ◊ $P(w) \leftarrow v$
 - stav(v) $\leftarrow$ uzavřený
 - * výstup: ohodnocení vrcholů h a pole předchůdců
- Bellmanův-Fordův algoritmus má fáze
 - * F_0 := otevření v_0
 - * F_i := zavírání vrcholů otevřených v F_{i-1} a otevírání jejich následníků
- invariant: na konci fáze F_i odpovídá $h(v)$ délce nejkratšího v_0v -sledu o nejvýše i hranách
- z toho vyplývá složitost $\Theta(n \cdot m)$
- Minimální kostra grafu (Jarníkův a Borůvkův algoritmus)
 - lemma
 - nechť G je graf opatřený unikátními vahami, R nějaký jeho elementární řez a e nejlehčí hrana tohoto řezu
 - pak e leží v každé minimální kostře grafu G
 - klasický Jarník
 - hladový algoritmus
 - na začátku máme strom T , který obsahuje vrchol v_0 (libovolný) a žádné hrany
 - vezmeme nejlehčí hranu vedoucí mezi stromem T a zbytkem grafu - tu přidáme do stromu
 - * opakujeme, dokud takové hrany existují
 - složitost $O(n \cdot m)$
 - Jarník podle Dijkstra
 - udržuju si haldy aktivních hran - to jsou hrany v aktuálním řezu
 - do stromu T vždycky přidávám minimum z haldy
 - když zjistím, že mám dvě aktivní hrany, které vedou do jednoho vrcholu mimo T , tak tu těžší zahodím
 - s haldou složitost $O(m \log n)$
 - * protože $n \log n$ se díky souvislosti grafu vejde do $m \log n$
 - Borůvkův algoritmus
 - paralelní verze Jarníkova algoritmu
 - na začátku T obsahuje izolované vrcholy grafu
 - dokud T není souvislý:
 - * rozložíme T na komponenty souvislosti
 - * pro každou komponentu najdeme nejlehčí hranu mezi komponentou a zbytkem vrcholů a přidáme ji do T
 - složitost

- * rozklad na komponenty v $O(n + m)$, ale taky $O(m)$ díky souvislosti
- * nalezení nejlehčích hran v $O(m)$
- * algoritmus se zastaví po nejvýše $\lfloor \log n \rfloor$ iteracích
- * z toho plyne složitost $O(m \log n)$
- Toky v sítích (Ford-Fulkerson algoritmus)
 - cesta je nenasycená $\equiv$ žádná její hrana není nasycená / všechny mají kladné rezervy
 - algoritmus
 - iterujeme, dokud existuje nenasycená cesta ze zdroje do stoku
 - spočítáme rezervu celé cesty (minimum přes rezervy hran cesty)
 - pro každou hranu upravíme tok - v protisměru odečteme co nejvíc, zbytek přičteme po směru
 - pro celočíselné kapacity vrátí celočíselný tok
 - racionální kapacity převedeme na celočíselné
 - pro iracionální kapacity se může rozbít
 - lemma: pokud se algoritmus zastaví, vydá maximální tok
 - věta: pro každou síť s racionálními kapacitami se Fordův-Fulkersonův algoritmus zastaví a vydá maximální tok a minimální řez
 - vzhledem k operacím, které algoritmus provádí, nemůže z celých čísel vytvořit necelá
 - důsledek: velikost maximálního toku je rovna kapacitě minimálního řezu
 - důsledek: síť s celočíselnými kapacitami má aspoň jeden z maximálních toků celočíselný a Fordův-Fulkersonův algoritmus takový tok najde

5.3 3. Programovací jazyky

- Abstrakce, zapouzdření, polymorfismus - související konstrukty programovacích jazyků (třídy, rozhraní, metody, datové položky, dědičnost, viditelnost)
 - v C# i v C++ jsou třídy (class) i struktury (struct)
 - deklarace musí být v C++ zakončena středníkem
 - v C++ to neovlivňuje způsob uložení a předávání, v C# jsou struktury hodnotového typu
 - rozhraní (interface) v C#
 - konvence psát na začátek názvu I
 - dovnitř se píšou metody / method-like věci (třeba props), nesmí tam být fields
 - jeden řádek může být např. `public int m(int a, int b);`
 - dříve muselo být všechno public, dneska je dobré tam to public explicitně psát (i když defaultně je pořád všechno public)
 - názvy parametrů metod v interfacu slouží k dokumentaci
 - nemůže existovat instance interfacu, pouze proměnná s typem interfacu (ta může být nullable)
 - `class Kachna : IPostovniZvire {...}`
 - `IPostovniZvire zvire1 = new Kachna();`
 - třída může implementovat víc interfaců
 - rozhraní může dědit od jiného rozhraní
 - v C++ můžeme rozhraní definovat pomocí dědičnosti (obvykle virtuální)
 - v C++ můžu při dědění definovat viditelnost
 - viditelnost v C# - některá klíčová slova
 - public - přístup není omezen
 - private - přístup je omezen na kód v daném typu

- * C# podporuje vnořené typy - takže kód ve vnořeném typu má taky přístup k private položkám typu, v němž je vnořen
 - protected - přístup je omezen na daný typ a typy, které z něj dědí
- výchozí viditelnost - private ve třídě, public v interfacu
- C#: abstract method vs. interface
 - vynucení kontraktu - interface ho vynucuje hned
 - veřejný kontrakt - abstraktní metody můžou být protected
 - slib rozšiřitelnosti - u virtuální metody je to opt-out (pomocí sealed), u interfaců opt-in (musím znova říct, že implementuju interface)
 - data - u interfaců nelze (kvůli vícenásobné dědičnosti)
 - vícenásobná dědičnost - nelze u abstraktních metod
- (Dynamický) polymorfismus, statické a dynamické typování
 - statické typování
 - proměnné mají pevně dané typy známé při kompilaci
 - je jasné, jaké metody podporují
 - dobře se provádí statická analýza kódu, lze hlásit chybná volání metod apod.
 - dynamické typování
 - typy proměnných nejsou pevně definované, určují se za běhu
 - volání chybné metody selže až za běhu
 - tzv. duck typing
 - v C# lze aktivovat klíčovým slovem `dynamic`
 - za jistou formu dynamického typování lze považovat i používání obecných rozhraní nebo rodičovských typů (např. `object`)
 - statický polymorfismus se implementuje tak, že přetížíme metodu nebo operátor nebo že zakryjeme rodičovskou metodu
 - dynamický polymorfismus funguje pomocí virtuálních metod
 - při volání interfacové metody je důležité, která třída ji implementuje
 - koncept virtuálních metod s interfacem přímo nesouvisí, ale požadavek interfacu můžeme uspokojit virtuální metodou
 - poznámka: interfacová metoda se dá implementovat explicitně
- Vícenásobná dědičnost a její problémy (vícenásobná a virtuální dědičnost v C++, interfaces v C#)
 - diamantový problém - pokud mají moji rodiče společného rodiče a já volám jeho metodu, kterou oba implementují, jaká metoda se má volat?
- Číselné a výčtové typy
 - enum se v C# definuje třeba takto: `enum Season { Spring, Summer, Autumn, Winter }`
 - když za Season napíšu : `ushort`, tak se místo `intu` použije `ushort`
 - můžu nadefinovat hodnoty jednotlivých možností
 - v C# jsou tyto číselné typy: `byte`, `sbyte`, `decimal`, `double`, `float`, `int`, `uint`, `nint`, `nuint`, `long`, `ulong`, `short`, `ushort`
 - `nint` a `nuint` mají nativní velikost (64 nebo 32 bitů, podle platformy)
- Ukazatele a reference v C++
 - smart pointery
 - raw pointery
 - reference (`const`, `rvalue`)
- Hodnotové a referenční typy v C#
 - základní dělení všech typů

- pointery - jsou ošklivé, nebudeme se o nich bavit
- hodnotové typy - alokované na místě (s výjimkami)
 - * enums
 - * structures
 - simple types (Int32, Int64, Double, Boolean, Char, ...)
 - nullables
 - user defined structures (struct)
- referenční typy - alokované na spravované (managed) haldě
 - * classes (e.g. strings)
 - * interfaces
 - * arrays
 - * delegates
- halda je garbage-collectovaná (GC), funguje chytřeji než v Pythonu, není potřeba reference counter, ale používá se graf dosažitelnosti
- overhead u každého objektu na haldě (má typicky 16 B)
 - syncblock - kvůli práci s více vlákny (u jednovláknových programů zbytečný), má 8 B, skládá se ze dvou částí:
 - * lock
 - owner thread + counter (kvůli rekurzivnímu zamykání)
 - waiting threads list - nějaký férový seznam (nebo fronta, na tom nesejde)
 - * condition variable
 - sleeping threads list
 - pointer na typ (zjednodušeně řečeno)
 - * třída System.Type, má instance na GC haldě
 - * každý datový typ odpovídá jedné instanci třídy
 - na referenčních proměnných jde volat `.GetType()`
- co může být v proměnné
 - hodnota (u hodnotových typů)
 - * velikost odpovídá velikosti datového typu
 - * u struktur lze použít **new**, které nic nealokuje, ale zavolá konstruktor
 - reference (u referenčních typů)
 - * je tam uložená adresa, takže velikost odpovídá velikosti adres
 - * adresa vede na GC haldu
 - * adresa ukazuje na celý objekt
 - * explicitní alokace pomocí **new**
 - * tu adresu nelze zjistit (zčásti proto, že GC objekty na haldě někdy přesouvá, proto se adresa může měnit)
- Pokročilé prvky syntaxe C#
 - vícerozměrná pole
 - zubatá (jagged)
 - * pole polí
 - `int[] [] a = new int[2] [];`
 - `a[0] = new int[3];`
 - `a[1] = new int[4];`
 - `int x = a[0][1];`
 - * jako v Javě

- * nevýhoda: každé pole má svůj vlastní overhead
 - obdélníková (rectangular)
 - * mezi hranaté závorky napíšu jednu nebo více čárek
 - * `int[,] a = new int[2, 3];`
 - * `int x = a[0, 1];`
 - * jako v C/C++
 - jagged jsou obecně rychlejší (cca 2×)
 - * ale pokud už s každou naindexovanou položkou pole budu provádět nějaký výpočet, tak tam nebude velký rozdíl v rychlosti
 - obdélníková jsou pamětově úspornější (v určitých situacích), lépe se zapisují
- indexer je vlastně speciální property
 - `public T this[int i] { get { return arr[i]; } set { arr[i] = value; } }`
- nullable value types mají vlastnost `bool HasValue` a taky `T Value`
- přetěžování operátorů
 - `public static Fraction operator *(Fraction left, Fraction right) => new Fraction(left.numerator * right.numerator, left.denominator * right.denominator);`
- přetížení konverze
 - `public static implicit operator byte(Digit d) => d.digit;`
 - `public static explicit operator Digit(byte b) => new Digit(b);`
- Reference, imutabilní typy a boxing v C#
 - boxing
 - každý hodnotový typ je potomkem objectu
 - tedy do proměnné typu object musí jít přiřadit proměnnou hodnotového typu
 - při takovém přiřazení se provádí boxing - je to implicitní alokace na GC haldě, alokuje se tam instance hodnotového typu (bude tam standardní overhead)
 - * pozor: v Javě jsou dva různé typy, `int` (hodnotový) a `Integer` (referenční) - v C# je to ten samý typ `System.Int32`
 - co můžu dělat s proměnnou typu object?
 - * můžu volat `ToString()`
 - unboxing - hodnotový typ alokovaný na haldě uložený referencí se uloží jako hodnota
 - není vhodné používat object pro slabé typování - vlastně takhle implementujeme duck typing
 - když potřebujeme hodnotový typ předávat referencí, tak object nedává smysl, vhodnější je použít jednoduchou třídu
- imutabilní typy
 - do fieldu označeného jako `readonly` lze zapisovat pouze v konstruktoru (a při inicializaci apod.)
 - případně u vlastností můžeme schovat setter - pak budou taky `readonly`
 - strukturu můžeme celou označit jako `readonly`
- reference
 - hodí se pro snazší manipulaci s hodnotovými proměnnými (abychom je všude nemuseli předávat hodnotou a abychom mohli z funkce vrátit víc hodnot než jednu)
 - * u referenčních typů obvykle nedává smysl používat tracking referenci
 - * výjimečným případem, kdy to smysl dává, je třeba zvětšení pole - vyrobím nové pole, položky překopíruju a do tracking reference přiřadím nové pole
 - používají se tzv. tracking reference

- klíčové slovo **ref** - používá se u parametrů funkcí, uvádí se dvakrát, v hlavičce funkce i při volání
- když používám **ref**, tak proměnná musí být inicializovaná (aby se z ní dalo číst)
- proto se u výstupních parametrů funkcí používá klíčové slovo **out** - na úrovni CIL kódu je to to samé, ale nevyžaduje to, aby proměnné byly inicializované
 - * uvnitř funkce s výstupními parametry se s nimi zachází jako s neinicializovanými (dá se z nich číst až po prvním zápisu)
 - * před standardním ukončením funkce s výstupními parametry se do každého z nich musí alespoň jednou zapsat
 - * pokud funkce skončí výjimkou, tak se do výstupních parametrů zapsat nemusí, ale to není problém, protože catch blok nemůže spoléhat na inicializaci v try bloku
 - * pokud proměnnou deklaruji nad try/catch, v try bloku ji inicializuji a v catch bloku vyhodím výjimku dál (pomocí **throw;**), tak pod try/catch můžu proměnnou považovat za inicializovanou
 - * pokud napíšu **if (int.Parse(s, out int x))**, tak se proměnná deklaruje před ifem
- klíčové slovo **in ...** proměnná uvnitř funkce funguje jako readonly (podobně celá struktura funguje jako readonly)
 - * při volání funkce se dá volat i hodnotově bez klíčového slova
 - * **in** má smysl u výkonnostních optimalizací hodnotových typů (typicky u počítačových her)
- místo **in** se dá použít taky **ref readonly** - je to něco velmi podobného
- tracking reference se dají používat i jinde (nejen jako parametry funkcí) - je lepší je moc nepoužívat
- Šablony (templates) a statický polymorfismus v C++
 - např. `template<typename T> const T& max(const T& a, const T& b) { return a < b ? b : a; }`
 - templatovaná funkce
 - v C# třeba pomocí generické metody a omezení na IComparable
- Generické typy v C# (bez omezení typových parametrů)
 - `class GenericList<T> { }`
 - při vytvoření instance je potřeba uvést typový parametr (při volání generických metod to není potřeba)
- Typy reprezentující funkce v C++, C#
 - v C# se používají delegáti
 - pomocí klíčového slova **delegate** definujeme nový delegátní typ: `public delegate void Callback(string message);`
 - za předpokladu, že někde existuje **DelegateMethod** s jedním parametrem typu string, která vrací void, tak můžeme provést přiřazení `Callback handler = DelegateMethod;`
 - * kdybychom chtěli vynutit vytvoření nové instance delegáta, tak bychom použili syntaxi `Callback handler = new Callback(DelegateMethod);`
 - pak ji můžeme použít: `handler("Hello World");`
 - k viditelnosti: můžu mít public metodu, která vrací delegáta, který ukazuje na private statickou metodu
 - delegáti můžou ukazovat i na instanční metody
 - * v delegátu jsou dva ukazatele - na funkci a na **this**
 - u statických metod je tam null
 - * stále platí, že delegáti jsou immutable - to konkrétní **this** je tam navždy
 - * u struktur - do **this** se zkopíruje struktura (zaboxuje se)

- delegáti mohou být generičtí
- u delegátů se dává suffix Delegate
- občas mi jde čistě o parametry - je mi úplně jedno, co ten delegát dělá
 - * bylo by zbytečné pro každou takovou metodu deklarovat delegáta
 - * proto existuje spousta předdefinovaných delegátů
 - `Action<...>(...) → void`
 - `Func<..., TResult>(...) → TResult`
 - `Predicate<T>(T obj) → bool`
 - * pozor na přehlednost - typicky je lepší používat silně typované delegáty
- `var` funguje i pro delegáty
- v C++ je víc způsobů, jak pracovat s funkcemi
 - templates
 - function pointers
 - `std::function`
- Lambda funkce a funkcionální rozhraní
 - lambda funkce v C# se píšou jako `(input-parameters) => expression` nebo `(input-parameters) => { <sequence-of-statements> }`
 - mohou být statické
 - např. `static x => x + 1`
 - pokud nejsou statické, mohou zachytávat proměnné z kontextu (provádí se zachycení podobné capture by reference v C++)
 - lambda funkce jsou přiřaditelné do proměnných typu delegát
- Správa životního cyklu zdrojů v případě výskytu chyb - RAII v C++, using v C#, obsluha a propagace výjimek
 - RAII ... Resource Acquisition is Initialization
 - zdroje, které třída používá, by se měly pořídit v konstruktoru a uvolnit v destrukturu, aby to bylo bezpečné
 - `lock & unlock → lock_guard`
 - `new & delete → smart pointers`
 - pokud chceme ohraničit část kódu, kde má být zámek zamčený (nebo soubor otevřený), tak ji uzavřeme do složených závorek
 - v C# pomocí usingu
 - `using (var f1 = new FileStream("...")) { ... }`
 - je to syntaktická zkratka pro try & finally, přičemž ve finally bloku se volá Dispose na hodnotě výrazu v závorce
 - výjimky
 - vyhazují se pomocí `throw new Ex();`, kde Ex je nějaký typ výjimky (konstruktor může mít parametr - třeba nějakou zprávu)
 - zachycují/zpracovávají se pomocí try-catch-finally bloku
 - catch bloků může být víc pro různé typy výjimek
 - v catch bloku můžeme tu stejnou výjimku vyhodit znova pomocí `throw;`
- Alokace (alokace statická, na zásobníku, na haldě)
 - statická alokace se provádí už během kompilace, týká se proměnných, které mají být k dispozici během celého běhu programu
 - alokace na zásobník se děje při volání funkcí - lokální proměnné se takto alokují a po doběhnutí funkce se zase smažou
 - je to rychlejší než alokace na haldě, ale na (volacím) zásobníku je méně místa než na haldě

- opravdu to funguje jako zásobník (FIFO)
- velikost alokované paměti je známá během kompilace
- dynamickou alokaci na haldě použijeme v situacích, kdy nám první dva přístupy nestačí
 - potřebujeme sdílet objekty nějak napříč
 - potřebujeme polymorfismus a nestačí nám reference (v C++)
 - potřebujeme alokovat hodně paměti
- v C++ je nutné dynamicky alokovanou paměť ručně dealokovat (smart pointers tohle řeší)
- v C# jsou všechny instance referenčních typů (a jejich fieldy) na haldě
- Inicializace (konstruktory, volání zděděných konstruktorů)
 - v C# je konstruktor označen viditelností a názvem třídy, volá se s new
 - volání rodičovského konstrukturu zajišťuje klíčové slovo base
 - `public B(params1) : base(params2) {`
 - není to optional, rodičovský konstruktor se volá pokaždé (defaultně bez parametrů)
 - konstruktor předka se vždy volá před konstruktorem potomka
 - v C++ je syntaxe velmi podobná, akorát místo `base` píšeme přímo název rodičovské třídy
- Destrukce (destruktory, finalizátory)
 - v C# se finalizátory používají k explicitnímu uvolňování unmanaged zdrojů (ve specifických use-casech)
 - v C++ se destruktory používají častěji, nejtypičtější to je, pokud potřebujeme mít pod kontrolou vytváření kopií objektu apod. (viz rule of five)
 - rodičovské třídy v C++ by měly mít virtuální destruktory (stačí prázdný)
- Explicitní uvolňování objektů, reference counting, garbage collector
 - v C++ se na haldě alokuje pomocí new, pak je nutné objekt smazat pomocí delete (právě jednou)
 - alternativou jsou smart pointers
 - * `unique_ptr` dealokuje, jakmile dochází se smazání pointeru (proměnná vypadne ze scope)
 - * `shared_ptr` umožňuje ukazovat na jeden objekt z více míst, počítá reference - dealokuje, jakmile je referencí nula
 - může být problém s cyklickými referencemi, proto se někdy používá `weak_ptr`
 - v C se k podobnému účelu používá `malloc` a `free`
 - v C# je halda garbage collectovaná
 - garbage collector se pouští podle toho, zda je potřeba uvolnit paměť
 - fungují tam tři generace (vrstvy) objektů
 - vytváří se graf objektů, aby bylo jasné, které jsou používány a které je možné smazat
 - po smazání se zbylé objekty na haldě přeskupují (tzv. heap compacting)
 - GC má dva módy - workstation a server (v server módu běží GC na více vláknech)
- Reprezentace vláken v programovacích jazycích
 - v C# je ve jmenném prostoru `System.Threading` třída `Thread`, která obvykle reprezentuje jedno vlákno
 - ale vytvoření vlákna je drahé, takže může být užitečnější použít `ThreadPool`
- Specifikace funkce vykonávané vláknem a základní operace nad vlákny
 - v C# můžeme vytvořit instanci třídy `Thread`
 - instanci předáváme delegáta, ten může být bez parametrů nebo s jedním parametrem typu `object`
 - vlákno spustíme pomocí volání `Start()`
 - pokud v nějaký moment chceme počkat, než doběhne, můžeme použít `Join()`

- Časově závislé chyby (race condition) a mechanismy pro synchronizaci vláken
 - race condition nastává, když je výsledek operace závislý na pořadí, v jakém se provedou události, jejichž pořadí nemáme pod kontrolou
 - nejjednodušší řešení je mutual exclusion - k tomu se používá zámek
 - `Monitor.Enter(obj)` zamkne zámek na daném objektu (nebo pasivně čeká, dokud se neodemkne)
 - `Monitor.Exit(obj)` odemkne zámek
 - existuje syntaktická zkratka `lock(obj) { ... }`, na začátku bloku se volá `Enter`, na konci se volá `Exit`
 - zámek si v syncblocku ukládá referenci na vlákno, které ho vlastní (jinak je tam null)
 - pokud ho zamkneme ze stejného vlákna několikrát, musíme ho několikrát odemknout (k tomu má počítadlo)
 - syncblock má každá referenční proměnná
 - co budeme zamykat?
 - chceme mít jistotu, že nám to nezamkne nikdo jiný proti naší vůli
 - tedy pokud zamykáme například nějakou naši vlastní implementaci seznamu, dává smysl, aby ta třída měla soukromý field typu object, který bude sloužit jenom jako zámek
 - neměli bychom používat `lock (this)`
 - součástí syncblocku je taky seznam čekajících vláken určený pro tzv. condition variable
 - vlákna čekají na to, až se s objektem něco stane
 - * třeba až se ve frontě objeví nějaké položky
 - * tuhle podmínku je vhodné někam poznamenat, třeba do komentáře, je to ta „condition variable“
 - když vlákno vidí, že je fronta prázdná, tak se pomocí `Monitor.Wait(obj)` zařadí do seznamu
 - jakmile se do fronty zařadí další položka, fronta může čekající vlákna notifikovat pomocí `Monitor.Pulse(obj)` nebo `Monitor.PulseAll(obj)`
 - aby se dal použít `Wait`, `Pulse` nebo `PulseAll`, musí být obalený v lock bloku (pro stejný objekt) - jinak to vyhodí výjimku
 - dalším synchronizačním primitivem je semafor, používá se k tomu, aby s objektem najednou mohlo pracovat jen omezené množství vláken
- Implementace dynamického polymorfismu (tabulka virtuálních metod)
 - každý typ s virtuálními metodami má tabulku virtuálních metod (VMT), ta se při dědění kopíruje (a prodlužuje), u overrideovaných virtuálních metod se změní odkaz na implementaci
 - takže to, která implementace se reálně zavolá, závisí na třídě, jejíž instanci jsme vytvořili (nehledě na typ)
- Nativní a interpretovaný běh, reprezentace programu, bytecode, interpret jazyka, just-in-time (JIT) a ahead-of-time (AOT) překlad
 - nativní běh - zdrojový kód se přeloží a vznikne program, který lze přímo spustit
 - interpretovaný běh - zdrojový kód je zpracováván interpretem jazyka
 - C++
 - zdrojáky .cpp, jeden po druhém je překládáme, vznikají object fily .obj (nebo .o), tam je přímo strojový kód
 - * ty se linkerem zpracují do jednoho .exe souboru
 - strojový kód typicky cílí na konkrétní platformu
 - kdybych chtěl, aby program běžel na jiné platformě, vezmu zdrojový kód a celý ho znova přeložím
 - 64bitové Windows podporují 32bitové programy

- uživatelé musí vědět, jakou verzi programu si mají vybrat
- knihovny
 - * překladač programu používá hlavičkové soubory knihoven
- Apple
 - Apple 6502 → Motorola 68000 (32bit) → IBM PowerPC (32bit) → Intel x86 (32bit) → Intel x64 (64bit) → Apple M1/M2 (ARM64, 64bit)
 - vymysleli fat binary (universal executable) - jeden spustitelný soubor, v něm je více verzí programu najednou
 - řeší to problém zpětné kompatibility, ne dopředné
- C#
 - soubory .cs
 - .csproj, předává se build systému dotnetu, ten pustí C# compiler csc.exe (dnes csc.dll)
 - vypadne z toho executable, uvnitř je CIL kód (common intermediate language)
 - ke spuštění programu potřebujeme CLR (common language runtime), obvykle je tam JIT (just-in-time) překladač, ten zajišťuje překlad a spuštění pro konkrétní platformu
 - * alternativou bylo vykonávat CIL instrukce pomocí běhové podpory, ale to je hrozně pomalé
 - CLR ... virtual machine (VM)
 - CIL kód ... managed/řízený kód
 - dotnet executable soubor ... assembly
 - Java to má podobně
 - * Java intermediate language = Java bytecode
 - všechny programovací jazyky dotnetu se překládají do CIL kódu
 - optimalizace dělá až JIT
 - * ale má na to omezený čas, aby se program spustil dostatečně rychle
 - JIT používá překlad po metodách
 - ale dá se použít AOT (ahead of time) překlad - takže pak v .exe souboru je nejen assembly (ten se použije pro nepodporované platformy), ale taky přímo strojový kód, který tak může být efektivnější
 - * některé věci tam nefungují
- Proces sestavení programu, oddělený překlad, linkování, staticky a dynamicky linkované knihovny
 - sestavení programu
 - preprocesor
 - kompilátor (překladač)
 - assembler
 - linker
 - knihovna - statická nebo dynamická sada binárek
 - linking - spojení binárek do jedné
 - loader - načte program do paměti
 - knihovna se linkuje do souboru .lib (statická knihovna)
 - lidi už pak používají přímo hlavičkový soubor a statickou zkompilovanou knihovnu
 - pokud je knihovna statická, použije ji linker, pokud je dynamická, použije ji až loader
 - v C# se o tohle všechno stará CLR (tedy .NET runtime), viz výše
 - používají se dynamicky linkované knihovny (DLL)
- Běhové prostředí procesu a vazba na operační systém
 - program vs. proces (proces je spuštěný program, entita operačního systému)
 - operační systém zajišťuje, aby měl proces k dispozici virtuální paměť, aby mohl přistupovat k

- souborům na disku, k periferiím apod.
 - v C# je běhovým prostředím CLR
- Návrhové vzory
 - decorator
 - chceme nějaký objekt obalit, přidat mu funkce
 - dekorátor obvykle přijímá původní objekt v konstruktoru
 - má stejné rozhraní jako původní objekt
 - adapter
 - převádí rozhraní jedné třídy na rozhraní jiné třídy
 - abychom instanci jiné třídy mohli použít tam, kde bychom normálně potřebovali instanci té původní třídy
 - factory
 - vyrábí instance objektů (typicky podle zadaných parametrů může vyrábět instance různých objektů)
 - nebo může mít factory těch vyráběcích metod více (objekty, které padají z různých metod, spolu typicky nějak souvisí)
 - pokud jsou vyráběcí metody virtuální a můžeme definovat různé factory třídy, které vyrábějí různé typy objektů, tak se tomu říká abstract factory
 - visitor
 - uvažujeme potomky A, B, C třídy Base
 - chceme pro každého potomka umět definovat speciální operaci, která se na něm má provést (a pak to chceme umět případně i nějak rozšiřovat)
 - mohli bychom použít `switch (obj) { case Type t: something(t); },` ale to není objektové ;)
 - místo toho nadefinujeme rozhraní (nebo abstraktní typ) pro visitora, bude obsahovat metody `VisitA`, `VisitB` a `VisitC` (metody se nemusí lišit názvy, stačí, když se liší typem parametrů - jako parametr přijímají ten objekt typu A, B nebo C)
 - třída Base pak bude mít abstraktní metodu `Accept(Visitor v)`, kterou musí implementovat každý potomek
 - implementace v potomkovi A vypadá tak, že se volá `v.VisitA(this)`;
 - použití
 - * máme konkrétního visitora `vis1` a chceme ho použít na objekt `obj` (který je typu A, B nebo C)
 - * zavoláme `obj.Accept(vis1)`;

5.4 4. Architektura počítačů a operačních systémů

- Základní architektura počítače, reprezentace čísel, dat a programů
 - Harvardská architektura počítače
 - CPU - procesor, vykonává příkazy
 - kódová paměť (code memory) - uložení informací o příkazech
 - komunikační linka - umožňuje, aby procesor četl data z paměti
 - * v základním počítači CPU nemusí zapisovat do kódové paměti, do ní se zapisuje nějak z venku
 - datová paměť (data memory) - CPU z ní čte a zapisuje do ní data
 - * data se ukládají do proměnných
 - k procesoru jsou dále připojena vstupně-výstupní zařízení - I/O, periferie

- von Neumannova architektura
 - operační paměť, kde je obojí - v jedné části kód programu, v jiné části data programu
 - zajistíme, aby IP (instruction pointer, jinak také PC, program counter) obsahoval adresy z určitého rozsahu a aby adresy proměnných byly zase z jiného rozsahu
 - je to hardwarově komplikovanější na implementaci, ale přináší to spoustu výhod
 - lze zařídit, aby IP ukazoval na začátek přeloženého programu
 - potenciální zranitelnost, pokud útočník do dat určených ke čtení uloží spustitelné byty a zařídí jejich spuštění
 - operační paměť bude volatile - RAM
 - * potřebujeme jiné magické zařízení, které do RAM zkopíruje počáteční program z non-volatile paměti
- Reprezentace a přístup k datům v paměti, adresa, adresový prostor
 - data i programy reprezentujeme binárně
 - každému bytu v paměti přiřadíme adresu (n-bit unsigned), definujeme si paměťový adresový prostor
 - u 256bytové paměti budeme mít 8bitový adresový prostor (0 až $2^8 - 1$)
 - 32bitový adresový prostor stačí pro 4 GB paměti, pro větší paměť už potřebujeme 64 bitů
- Ukládání jednoduchých a složených datových typů
 - jednoduché datové typy
 - čísla se ukládají ve dvojkové soustavě
 - znaménková čísla se ukládají ve dvojkovém doplňku
 - * kladná čísla jako unsigned
 - * záporná čísla jsou negace absolutní hodnoty plus 1
 - * rozsah -128 až 127
 - * nefunguje porovnání mezi kladnými a zápornými čísly - procesor musí podporovat znaménkové porovnání
 - * MSb určuje znaménko
 - * nejběžněji používaná implementace záporných čísel
 - čísla s plovoucí desetinnou čárkou (floating-point) se reprezentují takto: $\text{value} = (-1)^{\text{sign}} \cdot \text{significand} \cdot 2^{\text{exponent} - \text{bias}}$
 - číslo se naseká na byty a pak se ukládá v paměti, pořadí uložení bytů je dáno endianitou (little-endian vs. big-endian)
 - text se ukládá pomocí kódování, základní znaky používají ASCII, dále se obvykle používá Unicode
 - složené datové typy
 - moderní procesory vyžadují, aby byla data v paměti zarovnaná podle jejich velikosti
 - např. 4bajtový int musí mít adresu zarovnanou na 4 (dělitelnou čtyřmi)
 - celá struktura (struct) je zarovnaná na největší datový typ dostupný na CPU (např. 16B)
 - některé jazyky přehazují položky ve struktuře, aby se eliminovalo volné místo
 - sizeof vrátí velikost včetně těch mezer
- Implementovat běžné programové konstrukce vyšších jazyků (přiřazení, podmínka, cyklus, volání funkce) pomocí instrukcí procesoru

Procvičit (poznámky odkazují na vlastní trénink)

Překlad konstrukcí (přiřazení, podmínka, cyklus, volání funkce) do instrukcí procesoru je dovednost - natrénuj na konkrétní (i fiktivní) instrukční sadě.

- nacvičit
- Zapsat běžnou konstrukci vyššího jazyka (přiřazení, podmínka, cyklus, volání funkce), která

odpovídá zadané sekvenci (vysvětlených) instrukcí procesoru

Procvičit (poznámky odkazují na vlastní trénink)

Opačný směr (z dané sekvence instrukcí poznat konstrukci vyššího jazyka) je třeba natrénovat na příkladech.

- nacvičit
- Podpora pro běh operačního systému - privilegovaný a neprivilegovaný režim procesoru, jádro operačního systému
 - režimy procesoru (bývá jich několik, ale nejdůležitější jsou dva)
 - uživatelský režim (neprivilegovaný)
 - přístupný všem aplikacím
 - omezený přístup ke zdrojům (systémovým registrům, instrukcím)
 - kernel mode (privilegovaný)
 - je používán operačním systémem nebo jeho částí
 - má plný přístup ke zdrojům
 - přechod mezi režimy je jasně definovaný - je pouze omezené množství způsobů, jak přepnout z uživatelského do kernel režimu (aby bylo možné vyhodnotit, zda je to záměr, nebo chyba)
 - syscall = volání systémového API
 - tenká vrstva nad OS zajišťuje syscalls
 - např. k vypísání textu do konzole je potřeba, abychom se přepnuli do kernel módu a zpátky (zrychluje se to použitím bufferu)
 - jádro OS zajišťuje inicializaci hardwaru, následně řeší správu prostředků a spouštění/obsahu programů
 - podle velikosti jádra OS se rozlišují architektury
 - monolitická
 - * obslužná vrstva, jejím prostřednictvím se volají interní funkce
 - * takhle funguje linuxový kernel
 - * zranitelnost - programy v jádře mohou dělat cokoli
 - * původně byly součástí jádra pevně nastavené od instalaci
 - problém - při připojení klávesnice chceme nainstalovat driver zařízení → dnes lze obsah jádra měnit
 - vrstvená
 - * každá vrstva má svoje rozhraní
 - * každá vrstva může používat jenom vrstvu přímo pod ní (její rozhraní)
 - mikrokernel
 - * většina služeb se z kernel space vystrčí do user space, jsou tam jako jednotlivé moduly, ty komunikují bez sebou a s jádrem
 - * je to rozšiřitelné, spolehlivé (jednotlivé služby lze v případě chyby restartovat) a bezpečné, ale poměrně pomalé
 - * takhle fungují Windows
 - mají ještě hardwarovou abstrakční vrstvu, takže mikrokernel je přenositelný
 - služby běží v kernel režimu (což je v rozporu s filozofií mikrokernel)
- Popsat roli řadiče zařízení při programem řízené obsluze zařízení (PIO), pro zadané adresy a funkce vstupních a výstupních portů implementovat programem řízenou obsluhu zadaného zařízení (myš, disk)
 - implementaci nacvičit

Procvičit (poznámky odkazují na vlastní trénink)

Programem řízenou obsluhu zařízení (PIO) pro zadané adresy a porty je potřeba natrénovat na konkrétních zadáních.

- řadič zařízení (controller) - HW součástka, komunikuje se zařízením nějakým protokolem
- ovladač zařízení (driver) - SW komponenta, součást operačního systému, realizuje komunikaci s řadičem, poskytuje operačnímu systému jednotné komunikační rozhraní
- operační systém pak zařízení zprostředkovává běžícímu programu/procesu (a uživateli)
- Popsat roli přerušení při programem řízené obsluze zařízení (PIO), na úrovni vykonávání instrukcí popsat reakci procesoru (hardware) a operačního systému (software) na žádost o přerušení
 - komunikace se zařízeními
 - polling - CPU se aktivně ptá na změnu stavu zařízení (pomale, dnes už se nepoužívá)
 - přerušení - řadič vyšle signál, že je operace hotová, a přeruší chod procesoru (ale pořád musím kopírovat data, což je pomalé)
 - DMA (Direct Memory Access) - přenos dat ze zařízení do paměti probíhá hardwarově bez účasti procesoru, až pak se provede přerušení; funkce scatter/gather umožňuje využívat nesouvislé části paměti
 - typy přerušení
 - externí - hardwarové pomocí IRQ pinu, lze ho zamaskovat (deaktivovat - pomocí registru)
 - (hardwarová) výjimka - nastal problém s instrukcemi → procesor vyvolá přerušení a nechá na OS, aby situaci vyřešil
 - * výjimka typu trap se volá po instrukci, např. dělení nulou
 - * u výjimek typu fault se procesor vrátí na stav před instrukcí, nechám OS, aby problém opravil, a potom instrukci pustím znova
 - softwarové - speciální instrukce
 - jak funguje přerušení
 - CPU zjistí zdroj přerušení, získá adresu handleru (kdo přerušení zpracuje - je fixní nebo se použije tabulka)
 - proud instrukcí je přerušen, CPU začne vykonávat instrukce handleru (obvykle se přepne do privilegovaného režimu, protože handler je součástí kernelu)
 - handler uloží stav CPU
 - handler udělá něco užitečného
 - handler obnoví stav CPU
 - CPU pokračuje v původním proudu instrukcí
- Neprivilegované (uživatelské) procesy
 - k prostředkům přistupují skrze operační systém (ten pak využívá ovladače apod.)
 - mají k dispozici virtuální paměť
- Procesy, vlákna, kontext procesu a vlákna
 - program
 - pasivní soubor na disku
 - loader - vezme program a načte ho do paměti
 - někde v hlavičce programu je informace o tom, kde program (výkon instrukcí) začíná
 - proces
 - instance programu vytvořená operačním systémem
 - je to datová struktura uvnitř operačního systému, v níž jsou uloženy různá data, která daný program potřebuje
 - vlastní: kód, prostor v paměti, další prostředky
 - vlákno (thread)

- místo uvnitř procesu, kde se vykonávají instrukce
- kontext procesoru - datová struktura, ve které jsou uloženy registry procesoru, pokud dané vlákno zrovna neběží
- vlastní: pozici v kódu (program counter / instruction pointer), svůj zásobník, stav procesoru
- fiber
 - lightweight vlákno
 - nemá celý kontext
 - scheduling se dělá kooperativně (jednotlivé fibery se vzájemně vyměňují v běhu)
- Přepínání kontextu, kooperativní a preemptivní multitasking
 - scheduler - část operačního systému, používá schedulingové algoritmy k přidělení zdrojů (jader) jednotkám
 - když se přeruší vykonávání vlákna, provede se context switch - kontext procesoru (registry včetně PC) se uloží
 - multitasking
 - cooperative - všechny procesy kooperují, předávají si řízení
 - preemptive - každé vlákno dostane od scheduleru svůj time slice; jakmile čas vyprší, dojde k přerušení
 - jinými slovy
 - při kooperativním multitaskingu musí proces dobrovolně předat řízení (yield)
 - při preemptivním multitaskingu plánovač proces prostě přeruší, jakmile mu dojde čas
- Plánování běhu procesů a vláken, stavy vlákna
 - stavy vlákna
 - created
 - ready
 - running
 - blocked
 - terminated (zombie)
 - scheduler - část operačního systému, používá schedulingové algoritmy k přidělení zdrojů (jader) jednotkám
 - real-time scheduling: real-time proces má čas, kdy se má spustit, a čas, do kdy má skončit (hard deadline - nemá smysl pokračovat, soft deadline - má smysl pokračovat ve výpočtu)
 - priorita - vázaná na proces
 - je dvousložková - při spuštění se přiděluje statická priorita (podle uživatele), dynamická priorita se v časových intervalech pravidelně zvyšuje všem ready vláknům (po spuštění se dynamická priorita sníží na nula); celková priorita je daná součtem
 - kooperativní algoritmy
 - first come, first serve (FCFS)
 - * FIFO řada, procesy se vykonávají v pořadí, ve kterém přišly
 - shortest job first
 - * musí být známý očekávaný čas běhu
 - longest job first
 - preemptivní algoritmy
 - round robin
 - * jako FCFS
 - * když vlákno trvá moc dlouho, tak se zařadí na konec fronty
 - multilevel feedback-queue
 - * každá fronta má jinak dlouhý time slice

- * první (horní) ho má kratší, ty pod ní ho mají delší
 - * když vlákno trvá moc dlouho, tak se zařadí na konec nižší fronty
 - * když se vlákno brzo zablokuje, tak se zařadí na konec vyšší fronty
- completely fair scheduler (CFS)
 - * používá červeno-černý strom
- Vysvětlit rozdíl mezi virtuální a fyzickou adresou a identifikovat, zda se v zadaném kontextu či fragmentu kódu používá virtuální nebo fyzická adresa
 - instrukce procesoru pracují s virtuálními adresami
 - operační paměť má fyzické adresy
 - hardwarově je implementován převodní mechanismus
 - musí vyhodnotit, jestli existuje převod
 - zajistí přepočítání, pokud existuje
 - tyto kroky musí být extrémně rychlé
 - existují dva mechanismy - segmentace (už se nepoužívá) a stránkování
 - MMU = Memory Management Unit, provádí převod
 - převod paměti je transparentní
 - výhody konceptu
 - větší adresový prostor - abych mohl spustit proces s většími požadavky na paměť
 - bezpečnost - aby si procesy navzájem nemohly koukat do paměti, každý má vlastní mapování; to je dnes ten hlavní důvod
- Na zadaném příkladu identifikovat a vysvětlit význam komponent virtuální a fyzické adresy (číslo stránky, číslo rámce, offset)
 - virtuální adresový prostor (VAS) je rozdělen na stejné části (stránky, jejich velikost odpovídá mocninám dvojky)
 - fyzický adresový prostor (PAS) je rozdělen na stejné části (rámce, framy, mají stejnou velikost jako stránky)
 - VAS je jednorozměrný, instrukce pracují s jedním číslem
 - page table (stránkovací tabulka)
 - zajišťuje překlad adres
 - je to pole stránek (indexované číslem stránky)
 - každý záznam obsahuje číslo rámce a atributy
 - je tam jednička/nula - aby se dalo určit, jestli stránka fyzicky existuje
 - v případě chyby - page fault
 - virtuální adresa se navenek jeví jako jedno číslo, ale protože velikost stránky je mocnina dvojky, tak část binárního čísla odpovídá číslu stránky a část offsetu
 - offset je u virtuální i fyzické adresy stejný, tedy se překládá jenom prefix odpovídající stránce (přeloží se na číslo rámce)
 - když mi dojde fyzická paměť, tak nějakou stránku uložím na disk - je to obvykle méně dat, než by to bylo při výpadku segmentu (segmenty měly různé velikosti, stránky jsou všechny stejně velké)
- Pro konkrétní adresy a obsah jednoúrovňové stránkovací tabulky řešit úlohy překladu adres
 - ukázkový příklad:
 - 32bitová virtuální adresa
 - posledních 12 bitů určuje offset
 - prvních 20 bitů určuje stránku, použiju je k indexaci do stránkovací tabulky
 - dozvím se číslo rámce dat
 - k číslu rámce připojím offset

- dostanu 32bitovou fyzickou adresu
- Vysvětlit roli virtuálních adresových prostorů v ochraně paměti procesů a vláken
 - proces má vlastní virtuální adresový prostor, takže se nemůže stát, že by sahal na paměť ostatních procesů
 - ale vlákna stejného procesu virtuální adresový prostor sdílejí
 - někdy naopak chceme, aby různé procesy mohly sdílet nějaké místo v paměti - dá se zařídit, aby konkrétní virtuální adresy různých procesů ukazovaly na stejné místo do fyzické paměti
- Sdílení úložného prostoru - soubory, analogie s adresovým prostorem; abstrakce a rozhraní pro práci se soubory
 - soubor
 - jednotka organizace dat, kolekce souvisejících informací
 - jádro OS nerozumí formátům souborů
 - soubory se typicky neukládají v paměti, ale na perzistentním úložišti
 - soubor je chápán jako unikátní číslo - kvůli lidem mají soubory jméno a cestu (aby v souborech nebyl chaos)
 - * analogie s adresovým prostorem - cestu souboru je nutné přeložit (jako se virtuální adresa překládá na fyzickou)
 - některé části názvu souboru (počáteční tečka, přípona) mají speciální význam (ale přípona souboru nemusí souviset s reálným formátem souboru)
 - operace se soubory - vyčištění cache, změna atributů, vytvoření, smazání
 - file handle - číslo specifické pro proces, odkazuje na konkrétní soubor (stdin, stdout, stderr mají handles 0, 1, 2)
 - buffering
 - cachování sektorů disku (když disk čte po bajtech, první se načítá přímo z disku, všechny ostatní v daném sektoru už se pak načítají z cache)
 - existuje několik úrovní cache (systémová, runtime)
 - sekvenční vs. náhodný přístup
 - alternativy - memory mapping, asynchronní přístup souborům
 - adresář
 - kolekce souborů
 - význam - rychlejší hledání souboru, lepší navigace pro uživatele, logické seskupení souborů
 - obvykle reprezentován jako soubor
 - někdy v něm můžou být uloženy některé atributy jeho souborů
 - obvykle existuje kořenový adresář
 - operace - vytvořit, smazat, přejmenovat, najít název, vypsat soubory
 - ukládání souborů
 - tradiční úložiště
 - * na sekundárním nebo externím úložišti
 - * file system in RAM (pro dočasné soubory)
 - síťové úložiště - soubory se tváří, jako by byly na disku, ale přitom jsou uloženy někde jinde na síti
 - virtuální soubory - např. /dev/null
 - file links
 - links (hard links)
 - * více adresářových záznamů odkazuje na stejný fyzický soubor
 - * většina operací je transparentních
 - * šetří místo, ale může dělat problémy (smazání hardlinku nesmí smazat fyzický soubor -

takže OS musí počítat odkazy na daný soubor)

- symlinks (soft links)
 - * symlinky jsou speciální soubory, v nichž je uložena cesta jiného souboru
- file system
 - řeší překlad jmen, správu datových bloků, správu dat souborů
 - pro file system je jednotkou jeden blok, blok je určitý počet sektorů
 - řeší abstrakci práce se soubory a adresáři
 - může být lokální (FAT, NTFS) nebo síťový (NFS, CIFS/SMB)
- Časově závislé chyby (race conditions)
 - více vláken přistupuje ke sdílenému prostředku
 - to vede k tomu, že výsledek výpočtu závisí na plánování operačního systému nebo na chování procesoru
 - takový výsledek je k ničemu
 - řešila by to atomizace read-modify-write operace
 - jak to vyřešit?
 - definuju kritickou sekci
 - pomocí synchronizačního primitiva zajistím, že v kritické sekci je jenom jedno vlákno
- Kritická sekce, vzájemné vyloučení
 - kritická sekce - (nejmenší) kus zdrojového kódu, kde dochází k přístupu ke sdílenému prostředku, který nesmí používat víc procesů najednou
 - vzájemné vyloučení (mutex) - jednoduchý algoritmus (synchronizační primitivum) zajišťující, že do kritické sekce nevstoupí víc vláken/procesů najednou
 - je to vlastně zámek (ale zámek se obvykle týká vláken, kdežto mutex se vztahuje na procesy)
 - pokud je mutex zamčený nějakým procesem, jiný proces nemůže sdílený prostředek používat
 - umožňuje vícenásobné zamčení tím stejným procesem (pak je nezbytný stejný počet odemčení)
- Základní synchronizační primitiva, jejich rozhraní a použití - aktivní a pasivní čekání, zámky
 - aktivní a pasivní/blokující synchronizační primitiva
 - aktivní vykonávají instrukce a pořád koukají do kritické sekce, jestli tam můžou
 - pasivní/blokující jsou zablokovány, dokud není přístup povolen
 - hardwarová podpora - atomické instrukce test-and-set (TSL), compare-and-swap (CAS)
 - primitiva
 - spin-lock - aktivní čekání pomocí TSL/CAS, ideální pro krátké čekání
 - semafor - counter a fronta čekajících vláken, atomické operace UP a DOWN
 - mutex - semafor s počítadlem rovným jedné (umožňuje vstup jednoho vlákna do kritické sekce v danou chvíli), atomické operace LOCK a UNLOCK
 - barrier - několik vláken se v jednu chvíli setká u stejné bariéry

6 Část III: Specializace - Základy umělé inteligence

6.1 Řešení úloh prohledáváním

- formulace úlohy
 - agent - vnímá prostředí pomocí senzorů, aktuátory mu umožňují jednat
 - racionální agent
 - * zvolí akci, která maximalizuje jeho výkon
 - simple reflex agent
 - * observation $\rightarrow$ action
 - model-based reflex agent
 - * past state + past action + observation $\rightarrow$ state
 - * state $\rightarrow$ action
 - goal-based agent
 - * state + goal $\rightarrow$ action
 - reprezentace stavů agenta
 - atomic - stav je blackbox
 - * dá se použít prohledávání
 - factored - stav je vektor
 - * dá se použít constraint satisfaction, výroková logika, plánování
 - structured - stav je sada objektů, ty jsou propojeny
 - * dá se použít (predikátová) logika prvního řádu
 - problem solving agent - typ goal-based agenta
 - atomická reprezentace stavů
 - cíl = množina cílových stavů
 - akce = přechody mezi stavy
 - hledáme posloupnost akcí, kterými se dostaneme z výchozího do nějakého cílového stavu
 - očekáváme, že prostředí je plně pozorovatelné, diskrétní, statické a deterministické
 - příklad - Lloydova patnáctka
 - * ne všechny stavy jsou dosažitelné - zachovává se parita permutace
 - formulace problému
 - budeme potřebovat abstrakci prostředí (odstraníme detaily z prostředí)
 - * validní abstrakce = abstraktní řešení můžeme expandovat do reálného řešení
 - * užitečná abstrakce = vykonávání akcí v řešení je jednodušší než v původním problému
 - dobře definovaný problém
 - * výchozí stav
 - * přechodový model (**state**, **action**) $\rightarrow$ **state**
 - * goal test (funkce, která odpoví true/false podle toho, zda je daný stav cílový)
- stromové vs. grafové prohledávání
 - udržujeme si hranici (frontier) prozkoumané oblasti
 - graph search si navíc ukládá „uzavřenost“ vrcholu (jestli už jsme vrchol viděli)
 - takže nenastávají problémy s cykly
 - kvůli tomu, že stavový prostor může být nekonečný, neinicilizujeme všechny vrcholy, ale místo toho použijeme hešovací tabulku
 - i tree search lze použít k prohledávání grafů s cykly, ale může se chovat divně
 - u prohledávání DAGů tree search nezaručuje, že se prohledané stavy nebudou opakovat

- (opakovat se můžou, pokud do stejného vrcholu vedou dvě orientované cesty)
- neinformované prohledávání (DFS, BFS, uniform-cost search)
 - BFS (fronta)
 - nejmenší neexpandovaný vrchol je zvolen k expanzi
 - úplný pro konečně větvení grafy
 - optimální - pokud je cena cesty nerostoucí funkce hloubky
 - časová a prostorová složitost $O(b^{d+1})$, kde b je stupeň větvení (branching factor, maximální výstupní stupeň) a d je hloubka nejbližšího cíle
 - problém je s prostorovou složitostí
 - DFS (zásobník)
 - nejhlubší neexpandovaný vrchol je zvolen k expanzi
 - úplný pro graph search
 - * úplný = nalezne řešení vždy, když existuje (a v opačném případě hlásí, že řešení neexistuje)
 - neúplný pro tree search (pokud algoritmus pustíme na grafu, který není strom)
 - suboptimální - nesměruje k cíli
 - časová složitost $O(b^m)$, kde m je maximální hloubka libovolného vrcholu
 - prostorová složitost $O(bm)$
 - pokud použijeme backtracking (generujeme jenom jednoho následníka místo všech), tak nám stačí dokonce jenom $O(m)$ paměti
 - * DFS využívá to, že nám stav může vrátit všechny následníky (pak je držíme v paměti)
 - * backtracking v daném vrcholu generuje jednoho následníka (takže všechny ostatní nemusíme držet v paměti)
 - ale někdy se nedá generovat jenom jeden následník
 - rozšíření BFS o funkci určující cenu kroku (tedy cenu hrany)
 - Dijkstrův algoritmus
 - taky se tomu říká uniform-cost search
 - $g(n)$ označuje cenu nejlevnější cesty ze startu do n
 - místo fronty použijeme prioritní frontu
 - informované prohledávání (algoritmus A*, přípustné a konzistentní heuristiky)
 - best-first search
 - pro vrchol máme ohodnocovací funkci $f(n)$
 - $f(n)$ použijeme v Dijkstrově algoritmu místo $g(n)$
 - máme heuristickou funkci $h(n)$, která pro daný vrchol označuje odhadovanou cenu nejlevnější cesty do cílového stavu
 - best-first je třída algoritmů, kde máme uzly ohodnoceny nějakou funkcí a vybíráme nejmenší z nich
 - greedy best-first search
 - * $f(n) = h(n)$
 - * není optimální ani úplný
 - A* algoritmus
 - * $f(n) = g(n) + h(n)$, kde $h(n)$ je heuristika
 - * optimální a úplný
 - * typicky mu dojde paměť dřív než čas
 - co chceme od heuristiky $h(n)$ u algoritmu A*
 - přípustnost - $h(n)$ musí být menší rovna nejkratší cestě z daného vrcholu do cíle, musí být nezáporná

- monotónnost (= konzistence)
 - * mějme vrchol n a jeho souseda n'
 - * $c(n, a, n')$ je cena přechodu z n do n' (akcí a)
 - * heuristika je monotónní, když $h(n) \leq c(n, a, n') + h(n')$
- tvrzení: monotonie implikuje přípustnost, opačná implikace neplatí
- tvrzení: pro monotónní heuristiku jsou hodnoty $f(n)$ neklesající po libovolné cestě
- tvrzení: pokud je $h(n)$ přípustná, pak je A^* u tree search optimální
 - * do cíle nevedou žádné suboptimální cesty, takže si vystačíme s důkazem optimality Dijkstrova algoritmu
- tvrzení: pokud je $h(n)$ monotónní, pak je A^* u graph search optimální
 - * s nemonotónní heuristikou jsme mohli najít suboptimální cestu do cíle
- když heuristika h_2 dává větší hodnoty než h_1 , tak se říká, že h_2 dominuje h_1
 - pokud je h_2 přípustná a pokud se h_2 nepočítá výrazně déle než h_1 , tak je h_2 zjevně lepší než h_1

6.2 Splňování omezujících podmínek

- problém splňování podmínek
 - = constraint satisfaction problem (CSP)
 - konečná množina proměnných
 - domény - konečné množiny možných hodnot pro každou proměnnou
 - konečná množina podmínek (constraints)
 - constraint je relace na podmnožině proměnných (je to podmnožina kartézského součinu domén)
 - constraint arity = počet proměnných, které podmínka omezuje
 - chceme přípustné řešení (feasible solution)
 - úplné konzistentní přiřazení hodnot proměnným
 - konzistentní ... všechny podmínky jsou splněny
 - CSP můžeme řešit tree-search backtrackingem
 - úpravou proměnných v určitém pořadí můžeme získat větší efektivitu
 - můžeme pročistit hodnoty, které zjevně nesplňují podmínky
 - * příklad
 - $a \in \{3, 4, 5, 6, 7\}$
 - $b \in \{1, 2, 3, 4, 5\}$
 - constraint: $a < b$
 - ale např. $a = 6$ vůbec nedává smysl
 - odstraněním nekonzistentních hodnot dostaneme $a \in \{3, 4\}$, $b \in \{4, 5\}$
 - * můžeme použít metodu hranové konzistence
- hranová konzistence (algoritmus AC-3)
 - = arc consistency (arc ... orientovaná hrana)
 - uvažujeme binární podmínky
 - libovolnou n -ární podmínku lze převést na balík binárních podmínek
 - každá binární podmínka odpovídá dvojici orientovaných hran v síti podmínek (vrcholy odpovídají proměnným)
 - pokud je mezi dvěma proměnnými více podmínek, můžeme je sloučit do jedné (abychom neměli multigraf)

- orientovaná hrana (V_i, V_j) je hranově konzistentní $\equiv$ pro každé $x \in D_i$ existuje $y \in D_j$ takové, že přiřazení $V_i = x$ a $V_j = y$ splní všechny binární podmínky pro V_i, V_j
 - může platit, že (V_i, V_j) je hranově konzistentní, ale (V_j, V_i) není
- CSP je hranově konzistentní $\equiv$ každá hrana je hranově konzistentní (v obou směrech)
- algoritmus AC-3
 - začínám se všemi hranami (podmínkami) ve frontě
 - postupně беру hrany z fronty a pro každou odstraním nekonzistentní prvky z domény
 - pokud jsem odstranil nějaké prvky z domény, musím zopakovat kontrolu pro hrany směřující do dané proměnné (kromě hrany opačné k té aktuálně zpracované)
 - složitost $O(cd^3)$, kde c je počet podmínek, d je velikost domény
 - * $O(d^2)$... kontrola konzistence
 - * každá podmínka se ve frontě může objevit nejvýše d -krát, protože se z dané domény dá odstranit nejvýše d prvků
 - optimální algoritmus má složitost $O(cd^2)$
- hranová konzistence je typ lokální konzistence, negarantuje globální konzistenci
- arc consistency (AC) vs. forward checking (FC)
 - FC je jednodušší (a rychlejší) než AC, slouží ke stejnému účelu - zrychlení prohledávání
 - FC na základě přiřazení do proměnné (v daném kroku prohledávání) pročistí domény dosud nepřirazených proměnných
 - AC k tomu navíc zajišťuje vzájemnou kompatibilitu domén nepřirazených proměnných
- algoritmus hledání řešení (MAC)
 - chceme zkombinovat AC a backtracking
 - problém uděláme hranově konzistentní
 - po každém přiřazení obnovíme hranovou konzistenci
 - této technice se říká *look ahead* nebo *constraint propagation* nebo *udržování hranové konzistence* (MAC, maintaining arc consistency)
 - kontroly FC/AC jsou v polynomiálním čase, kdežto strom stavů se větví exponenciálně

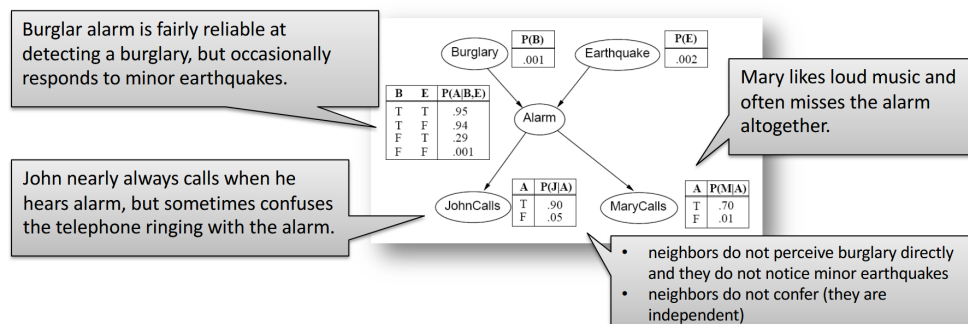
6.3 Logické uvažování

- základy výrokové logiky (konjunktivní a disjunktivní normální forma)
 - viz Společná matematika
 - výrokovou logiku lze použít k logickému odvozování znalostí na základě znalostní báze
- algoritmus DPLL
 - literál ℓ má čistý výskyt ve φ , pokud se ℓ vyskytuje ve φ a opačný literál $\bar{\ell}$ se ve φ nevyskytuje
 - takový literál můžu nastavit na 1
 - to neovlivní splnitelnost výroku, ale zmenší to množinu modelů, které jsem schopen nalézt
 - algoritmus
 - dokud φ obsahuje jednotkovou klauzuli ℓ , ohodnot $\ell = 1$ a proveď jednotkovou propagaci
 - * každou klauzuli obsahující ℓ odstraníme (protože je takto splněna)
 - * $\bar{\ell}$ odstraníme ze všech klauzulí, které ho obsahují (protože $\bar{\ell}$ nemůže zajistit splnění dané klauzule)
 - dokud existuje literál ℓ , který má ve φ čistý výskyt, ohodnot $\ell = 1$ a odstraň klauzule obsahující ℓ
 - pokud φ neobsahuje žádnou klauzuli, je splnitelný
 - pokud φ obsahuje prázdnou klauzuli, není splnitelný
 - jinak zvol dosud neohodnocenou výrokovou proměnnou p a zavolej algoritmus rekurzivně

- na $\varphi \wedge p$ a na $\varphi \wedge \neg p$
- algoritmus běží v exponenciálním čase
- praktické poznámky
 - čistý výskyt se zas tak moc nepoužívá
 - jednotková propagace
 - * hledám klauzule o jedné proměnné
 - * je v zásadě ekvivalentní hranové konzistenci
 - ke zkoušení hodnot proměnných se používá tree search
- dopředné a zpětné řetězení (Hornovské klauzule)
 - mám dotaz α , ten vyjádřím jako výrokovou formuli
 - znalostní bázi KB vyjádřím jako teorii
 - zajímá mě, zda $KB \models \alpha$ (to platí, právě když $KB \wedge \neg \alpha$ je nesplnitelné)
 - řetězení můžeme použít, pokud znalostní báze obsahuje pouze Hornovy klauzule
 - Hornova klauzule = disjunkce literálů, z nichž nejvýše jeden je pozitivní
 - Hornovu klauzuli $\neg p \vee \neg q \vee \neg r \vee s$ lze zapsat jako implikaci $p \wedge q \wedge r \implies s$
 - forward chaining ... data-driven reasoning
 - $p \wedge q \wedge r \implies s$
 - pokud vím, že platí p , pak převedu na $q \wedge r \implies s$
 - jakmile se počet předpokladů sníží na 0, vím, že s platí
 - je to v podstatě speciální případ použití rezolučního pravidla
 - * rezolvuji to, co vím, s libovolnou klauzulí
 - algoritmus
 - * každá proměnná má počítadlo proměnných, které na ni ukazují
 - * postupně беру pravdivé proměnné, snižuju počítadla u proměnných, na které ukazují
 - * jakmile u proměnné klesne počítadlo na nulu, zařadím mezi pravdivé proměnné
 - backward chaining ... goal-driven reasoning
 - něco mě zajímá - pokouším se to odvodit
 - * najdu klauzule, kde je nějaký literál z α na pravé straně implikace
 - * tak dostanu další literály, které musím dokázat
 - * postupuju tak dlouho, dokud se nedostanu k faktům (literálům, o nichž vím, že platí)
 - tohle se používá v Prologu
 - opět vlastně používám rezoluční pravidlo
 - * to, co chci zjistit, rezolvuji s libovolnou klauzulí
 - forward chaining prochází spoustu klauzulí, které nás nezajímají - backward chaining může mít výrazně menší složitost
- rezoluce
 - mám dotaz α , ten vyjádřím jako výrokovou formuli
 - znalostní bázi KB vyjádřím jako teorii
 - zajímá mě, zda $KB \models \alpha$ (to platí, právě když $KB \wedge \neg \alpha$ je nesplnitelné)
 - $KB \wedge \neg \alpha$ převedeme do CNF $\rightarrow$ dostaneme množinu klauzulí (formulí S)
 - použijeme rezoluci (zkonstruujeme rezoluční zamítnutí formule S)
 - použitím rezolučního pravidla z klauzulí $\varphi \vee p$ a $\psi \vee \neg p$ dostaneme $\varphi \vee \psi$
 - zkusíme spolu rezolvovat postupně všechny dvojice klauzulí
 - pokud v průběhu dostaneme prázdnou klauzuli, máme rezoluční důkaz α z KB
 - pokud se dostaneme do stavu, kdy nelze použít rezoluční pravidlo na žádnou dvojici klauzulí, tak víme, že neplatí $KB \models \alpha$

6.4 Pravděpodobnostní uvažování

- základy teorie pravděpodobnosti (úplná sdružená distribuce, nezávislost, Bayesovo pravidlo)
 - viz Společná matematika
 - pravděpodobnostní uvažování umožňuje agentovi zacházet s nejistotou
 - zdroji nejistoty jsou částečná pozorovatelnost (nelze snadno zjistit současný stav) a nedeterminismus (není jisté, jak akce dopadne)
 - agent si pamatuje belief state - pravděpodobnostní distribuce přes všechny možné stavy světa, v nichž se agent může nacházet
- pravděpodobnostní uvažování (vysčítání, normalizace)
 - někdy si můžeme udělat tabulku všech možností a určit jejich pravděpodobnosti (full joint probability distribution)
 - odpověď lze vyjádřit z ní (tétto metodě se říká enumeration, výčet) - ale u velkých tabulek je někdy potřeba počítat příliš mnoho hodnot
 - $P(Y) = \sum_z P(Y | z) \cdot P(z)$
 - $P(Y | e) = \frac{P(Y \wedge e)}{P(e)}$
 - typicky nemusíme znát $P(e)$, stačí použít $\alpha P(Y \wedge e)$, kde α je normalizační konstanta
 - velkou tabulku můžeme někdy reprezentovat menším způsobem - pokud jsou proměnné podmíněně nezávislé
 - Bayesovo pravidlo ... $P(Y | X) = \frac{P(X|Y)P(Y)}{P(X)} = \alpha P(X | Y)P(Y)$
 - lze odvodit z definice podmíněné pravděpodobnosti
 - pomůže nám při přechodu z příčinného k diagnostickému směru
 - * $P(\text{effect} | \text{cause})$... casual direction (příčinný směr), obvykle známý
 - * $P(\text{cause} | \text{effect})$... diagnostic direction (diagnostický směr), obvykle chceme zjistit
 - * známe důsledky; chceme zjistit, jaké příčiny k nim mohly vést
 - naivní bayesovský model
 - $P(\text{Cause}, \text{Effect}_1, \dots, \text{Effect}_n) = P(\text{Cause}) \prod_i P(\text{Effect}_i | \text{Cause})$
 - předpokládáme nezávislost důsledků
 - chain rule
 - $P(x_1, \dots, x_n) = \prod_i P(x_i | x_{i-1}, \dots, x_1)$
 - využívá product rule (lze odvodit z definice podmíněné pravděpodobnosti)
- Bayesovské sítě: konstrukce a vztah k úplné sdružené distribuci
 - bayesovská síť je DAG, kde vrcholy odpovídají náhodným proměnným
 - šipky popisují závislost
 - u každého vrcholu jsou CPD tabulky, které popisují jeho závislost na rodičích
 - * CPD = conditional probability distribution
 - * pro každou kombinaci hodnot rodičů jsou v CPD tabulce pravděpodobnosti hodnot dané náhodné proměnné



- autorem obrázku je [Roman Barták](#)
- proměnné jsou binární, proto má tabulka vždy jen jeden sloupec (druhý sloupec by mohl obsahovat pravděpodobnost, že jev nenastane - lze ho dopočítat z prvního sloupce)
- konstrukce bayesovské sítě
 - nějak si uspořádáme proměnné
 - * lepší je řadit je od příčin k důsledkům - budeme mít menší síť a CPD tabulky se nám budou vyplňovat snáze
 - jdeme odshora dolů, přidáváme správné hrany
 - příklad: MaryCalls, JohnCalls, Alarm, Burglary, Earthquake (záměrně jdeme od důsledků k příčinám, byť to není praktické)
 - * z MaryCalls povede hrana do JohnCalls, protože nejsou nezávislé (když volá Mary, tak je větší šance, že volá i John)
 - * z obou povede hrana do Alarmu
 - * z Alarmu vedou hrany do Burglary a Earthquake (ale hrany z JohnCalls a MaryCalls tam nepovedou - na těch hranách nezáleží, jsou nezávislé)
 - * z Burglary povede hrana do Earthquake, protože pokud zní alarm a k vloupání nedošlo, tak pravděpodobně došlo k zemětřesení
 - * akorát se nám v tomto pořadí budou špatně určovat CPD
 - bayesovská síť zachycuje úplnou sdruženou (joint) distribuci
 - dokážeme určit pravděpodobnost libovolné kombinace hodnot náhodných proměnných
 - pronásobíme mezi sebou hodnoty z tabulek
- Bayesovské sítě: exaktní odvozování (enumerace, eliminace proměnných)
 - z bayesovských sítí můžeme provádět inferenci - odvozovat pravděpodobnost proměnných pomocí pravděpodobností *skrytých* proměnných
 - $P(X | e) = \alpha P(X, e) = \alpha \sum_y P(X, e, y)$
 - kde pravděpodobnost X odvozujeme, e jsou pozorované proměnné (evidence) a y jsou hodnoty další skryté proměnné Y
 - přičemž $P(X, e, y)$ lze určit pomocí CPD tabulek
 - když nemůžeme určit konkrétní hodnotu pravděpodobnosti přímo, tak výpočet rozvětvíme
 - některé větve se objeví vícekrát - hodí se nám dynamické programování
 - používáme *faktory* (tabulky odpovídající CPD tabulkám)
 - tabulky s faktory mezi sebou můžeme násobit
 - eliminace
 - * mějme faktor s pravděpodobnostmi $P(A, B, C)$ pro všechny možné kombinace hodnot proměnných
 - * můžeme ho rozdělit na dvě půlky (v jedné bude všude platit A , v druhé $\neg A$)
 - * tyto dvě půlky můžeme sečíst (vždy odpovídající řádky)
 - * tak eliminujeme proměnnou A , dostaneme faktor s pravděpodobnostmi $P(B, C)$
- Bayesovské sítě: aproximační odvozování (Monte Carlo, zamítání, vážení věrohodností)
 - odhadujeme hodnoty, které je těžké přímo spočítat
 - třeba když je síť moc velká a hustě propojená
 - zajímá nás $P(X | e)$
 - generujeme hodně vzorků, použijeme statistiku
 - pravděpodobnostní rozdělení náhodných veličin známe (postupujeme topologicky bayesovskou sítí a generujeme hodnoty)
 - jak generovat vzorky?
 - rejection sampling = zahodíme vzorky, kde neplatí e

- * odhadneme $P(X | e) \approx \frac{N(X,e)}{N(e)}$
 - $N(X, e)$... počet vzorků, kde platí X, e
- * riziko, že zahodíme příliš mnoho vzorků
- likelihood weighting
 - * zafixujeme (ignorujeme) e , generujeme jen zbylé proměnné
 - * počtům N přidělíme váhy podle pravděpodobnosti e (např. místo toho, abychom zahodili cca 90 % vzorků, přiřadíme těm počtům váhu 0.1)
 - * problém nastává, když jsou váhy příliš malé

6.5 Reprezentace znalostí

- situační kalkulus, problém rámce
 - problém rámce
 - návrh: stav světa/agenta budeme popisovat pomocí časově anotovaných výrokových proměnných
 - musíme definovat spoustu axiomů, které zajistí, že se hodnoty proměnných mezi časovými kroky nemění náhodně
 - tak bychom snadno zahltili odvození pomocí výrokové logiky, protože bychom měli hodně akcí a stavů
 - jak předejít opakování axiomů pro různé časy a podobné akce?
 - použijeme predikátovou logiku prvního řádu (*situation calculus*)
 - náš modelovaný svět se vyvíjí - postupně nastávají různé *situace*
 - * úvodní situace ... s_0
 - * situace po aplikaci akce a na situaci s ... $\text{Results}(s, a)$
 - stavy jsou definovány jako relace mezi objekty
 - * pokud se relace mění, musíme přidat další argument označující situaci, v níž relace platí: $\text{at}(\text{robot}, \text{location}, s)$
 - * pro stálé relace takový argument není potřeba: $\text{connected}(l, l')$
 - podmínky (předpoklady) akcí jsou definovány possibility axiomem
 - * obecný tvar: $\varphi(s) \implies \text{Poss}(a, s)$
 - kde φ je formule popisující předpoklady akce a
 - * např. $\text{at}(a, l, s) \wedge \text{connected}(l, l') \implies \text{Poss}(\text{go}(a, l, l'), s)$
 - vlastnosti dalšího stavu jsou popsány pomocí successor-state axiomů
 - * např. $\text{Poss}(a, s) \implies (\text{at}(a, y, \text{Results}(s, a)) \iff a = \text{go}(a, x, y) \vee (\text{at}(a, y, s) \wedge a \neq \text{go}(a, y, z)))$
 - plánování se provádí tak, že se zeptáme, zda existuje situace, kde je cílová podmínka pravdivá
- Markovské modely - filtrace, predikce, vyhlazování, nejpravděpodobnější průchod
 - každý stav světa je popsán množinou náhodných proměnných
 - skryté náhodné proměnné X_t - popisují reálný stav
 - pozorovatelné náhodné proměnné e_t - popisují, co pozorujeme
 - uvažujeme diskrétní čas, takže t označuje konkrétní okamžik
 - množinu proměnných od X_a do X_b budeme značit jako $X_{a:b}$
 - formální model
 - skrytý Markovův model (HMM) je dvojice (X_n, e_n)
 - * X_n a e_n jsou náhodné (stochastické) procesy, takže vlastně posloupnosti náhodných

- proměnných
 - * X_n je markovský proces, který nemůžeme přímo pozorovat
- transition model
 - * určuje pravděpodobnostní rozložení proměnných posledního stavu, když známe jejich předchozí hodnoty ... $P(X_t | X_{0:t-1})$
 - * zjednodušující předpoklady
 - další stav závisí jen na předchozím stavu ... *Markov assumption*
 - všechny přechodové tabulky $P(X_t | X_{t-1})$ jsou identické přes všechna t ... *stationary process*
- sensor (observation) model
 - * popisuje, jak pozorované proměnné závisí na ostatních proměnných ... $P(e_t | X_{0:t}, e_{1:t-1})$
 - * zjednodušující předpoklad: pozorování záleží jen na aktuálním stavu ... *sensor Markov assumption*
- základní inferenční úlohy
 - filtrování - znám minulé pozorování, zajímá mě přítomný stav
 - * $P(X_{t+1} | e_{1:t+1}) = P(X_{t+1} | e_{1:t}, e_{t+1})$
 - použijeme Bayesovo pravidlo
 - * $= \alpha \cdot P(e_{t+1} | X_{t+1}, e_{1:t}) \cdot P(X_{t+1} | e_{1:t})$
 - použijeme *sensor Markov assumption*
 - * $= \alpha \cdot P(e_{t+1} | X_{t+1}) \cdot P(X_{t+1} | e_{1:t})$
 - * $= \alpha \cdot P(e_{t+1} | X_{t+1}) \cdot \sum_{x_t} P(X_{t+1} | x_t, e_{1:t}) \cdot P(x_t | e_{1:t})$
 - * $= \alpha \cdot P(e_{t+1} | X_{t+1}) \cdot \sum_{x_t} P(X_{t+1} | x_t) \cdot P(x_t | e_{1:t})$
 - * užitečný filtrační algoritmus si musí udržovat odhad současného stavu, jelikož je vzorec rekurzivní
 - * používá se *forward* algoritmus
 - předpověď (prediction) - znám minulé pozorování, zajímají mě budoucí stavy
 - * vlastně jako filtering, akorát nepřidávám další pozorování (měření)
 - * $P(X_{t+k+1} | e_{1:t}) = \sum_{x_{t+k}} P(X_{t+k+1} | x_{t+k}) \cdot P(x_{t+k} | e_{1:t})$
 - * po určitém čase (*mixing time*) distribuce předpovědi konverguje ke stacionárnímu rozdělení daného markovského procesu a zůstane konstantní
 - vyhlazování (smoothing) - znám minulé pozorování, zajímají mě minulé stavy
 - * $P(X_k | e_{1:t}) = P(X_k | e_{1:k}, e_{k+1:t})$
 - použijeme Bayesovo pravidlo
 - * $= \alpha \cdot P(X_k | e_{1:k}) \cdot P(e_{k+1:t} | X_k, e_{1:k})$
 - použijeme *sensor Markov assumption*
 - * $= \alpha \cdot P(X_k | e_{1:k}) \cdot P(e_{k+1:t} | X_k)$
 - * přitom levý člen známe z filteringu (je to „dopředný směr“), pravý je „zpětný směr“ z naměřených hodnot *v budoucnosti* (relativně vůči danému okamžiku)
 - * používá se *forward-backward* algoritmus
 - nejpravděpodobnější vysvětlení (most likely explanation) - znám minulé pozorování, zajímá mě nejpravděpodobnější posloupnost minulých stavů
 - * hledáme posloupnost $X_{1:t}$ takovou, že má největší $P(X_{1:t+1} | e_{1:t+1})$
 - * opět existuje rekurzivní vzorec - podíváme se na pravděpodobnosti předchozích stavů a na nejpravděpodobnější „cesty“ do těchto stavů
 - * používá se *Viterbiho* algoritmus
- skryté Markovské modely (HMM) vs. dynamické Bayesovské sítě

- skryté Markovovy modely
 - předpokládejme, že stav procesu je popsán jedinou diskretní náhodnou proměnnou X_t (a máme jednu proměnnou E_t odpovídající pozorování)
 - pak můžeme všechny základní algoritmy (filtering, prediction, smoothing, ...) implementovat maticově
- dynamická bayesovská síť
 - umožňuje popsat víc náhodných proměnných
 - reprezentuje časový pravděpodobnostní model
 - zachycuje vztah mezi minulým a současným časovým okamžikem (minulou a současnou vrstvou)
 - každá stavová proměnná má rodiče ve stejné vrstvě nebo v předchozí (podle Markovova předpokladu)
- dynamická bayesovská síť (DBN) vs. skrytý Markovův model (HMM)
 - skrytý Markovův model je speciální případ dynamické bayesovské sítě
 - dynamická bayesovská síť může být kódovaná jako skrytý Markovův model
 - * jedna náhodná proměnná ve skrytém Markovově modelu je n-tice hodnot stavových proměnných v dynamické bayesovské síti
 - * v HMM vlastně celý stav světa komprimujeme do jedné náhodné proměnné - v DBN jich můžeme mít víc
 - vztah mezi DBN a HMM je podobný jako vztah mezi běžnými bayesovskými sítěmi a tabulkou s *full joint probability distribution*
 - * DBN je úspornější

6.6 Automatické plánování

- formulace plánovacího problému (definice operátoru)
 - stav světa je reprezentován jako množina proměnných
 - stav je popsán atomy, které platí (ostatní neplatí, používáme předpoklad uzavřeného světa)
 - pravdivostní hodnota některých atomů se mění ... fluents
 - u jiných atomů je pravdivostní hodnota stálá ... rigid atoms
 - schémata akcí (operátory) popisují možnosti agenta
 - schéma akce (operátor) popisuje akci, aniž by specifikovalo objekty, na které se akce vztahuje
 - operátor je trojice (název, předpoklady, důsledky)
 - * název - obsahuje rovněž parametry operátoru (proměnné, které se objevují v předpokladech a důsledcích)
 - * předpoklady - musí platit v daném stavu, aby se operátor dal použít
 - * důsledky - literály (fluenty!), které budou platit, jakmile se operátor provede
 - akce je plně zinstanciovaný operátor (za proměnné jsme dosadili konstanty)
 - akce je relevantní pro cíl $g \equiv$ přispívá cíli g a její důsledky (efekty) nejsou v konfliktu s g
 - dále definujeme výsledek aplikace akce na stav a regresní množinu pro cíl a akci
 - terminologie
 - popisu operátorů se obvykle říká doménový model
 - plánovací problém je trojice (O, s_0, g) , kde O je doménový model, s_0 je výchozí stav a g je cílová podmínka
 - plán je posloupnost akcí
 - plánování je realizováno prohledáváním stavového prostoru

- ten je mnohdy dost velký
- dopředné a zpětné plánování
 - forward (progression) planning
 - postupujeme od výchozího k cílovému stavu
 - prohledávací algoritmus potřebuje dobrou heuristiku
 - backward (regression) planning
 - prozkoumávají se jen relevantní akce → menší větvení než u forward planningu
 - používá množiny stavů spíše než individuální stavy → je těžké najít dobrou heuristiku

6.7 Markovské rozhodovací procesy (MDP)

- formulace problému (výpočet užitku, strategie)
 - řešíme sekvenční rozhodovací problém
 - pro plně pozorovatelné stochastické (nedeterministické) prostředí
 - máme Markovův přechodový model a reward
 - přechodový model $P(s' | s, a)$... pravděpodobnost dosažení stavu s' , když ve stavu s použiji akci $a \in A(s)$
 - odměna (reward) $R(s)$... získá ji agent ve stavu s , je krátkodobá
 - utility function $U([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$
 - * kde γ je *discount factor*, číslo mezi 0 a 1
 - jak moc si ceníme budoucích odměn
 - * utility je „dlouhodobá lokální odměna“
 - * hodnota utility funkce je konečná i pro nekonečnou posloupnost stavů, protože zjevně $U([s_0, \dots]) \leq \frac{R_{\max}}{1-\gamma}$
 - řešením MDP je policy (strategie) - funkce doporučující akci pro každý stav
 - tzn. strategie je zobrazení $\pi : S \rightarrow A$
 - potřebujeme zobrazení, protože kdyby řešením byla fixní sekvence akcí, tak by to nefungovalo pro stochastická prostředí (mohlo by se stát, že skončíme v jiném stavu, než jsme mysleli)
 - optimální strategie $\equiv$ strategie, která vrací největší střední hodnotu užitku (utility)
 - optimální strategie nezávisí na počátečním stavu - je jedno, kde jsme začali, pro nalezení nejvhodnější další akce by měl být důležitý jen aktuální stav
- Bellmanova rovnice
 - opravdový užitek stavu je reward (odměna) stavu ve spojení se střední hodnotou užitku následného stavu → to vede na Bellmanovu rovnici
 - vezmu reward a discountovaný užitek okolí
 - akorát nevím, kterou akci provedu, tak vezmu všechny a maximum přes ně (násobím jejich užitek pravděpodobnostmi)
 - $U(s) = R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) \cdot U(s')$
- iterace hodnot, iterace strategií
 - předpoklad: známe reward R a transition model P
 - chceme najít užitek U nebo optimální strategii π
 - soustava Bellmanových rovnic není lineární - obsahuje maximum
 - můžeme je řešit aproximací
 - použijeme iterativní přístup
 - * nastavíme nějak užitek - třeba jako nuly
 - * provedeme update - aplikujeme Bellmanovu rovnici

- * iterujeme, dokud to nezkonverguje
 - → **value iteration**
- strategie se ustálí dřív než užítiky
 - policy loss ... vzdálenost mezi optimálním užitkem a užitkem strategie
 - můžeme iterativně zlepšovat policy, dokud to jde
 - → **policy iteration**
 - z rovnic nám zmizí maximum → máme lineární rovnice
 - * policy evaluation = nalezení řešení těchto rovnic (tak najdeme utilities stavů)
 - * gaussovka je $O(n^3)$
 - * u velkých stavových prostorů dává smysl k policy evaluation použít value iteration
 - algoritmus doběhne, protože policies je jenom konečně mnoho a vždycky hledáme tu lepší
- POMDP (základní definice)
 - máme přechodový model $P(s' | s, a)$, akce $A(s)$ i odměny $R(s)$
 - navíc nám přibude sensor model $P(s | e)$
 - místo reálných stavů můžeme používat ty domnělé (belief states)
 - modely přechodů a senzorů jsou reprezentovány dynamickou bayesovskou sítí
 - přidáme rozhodování a užítiky, čímž dostaneme dynamickou rozhodovací síť
 - generujeme si stromček, kde se střídají vrstvy akcí a belief states
 - používá se algoritmus podobný algoritmu *expected minimax*

6.8 Hry a teorie her

- algoritmy Minimax a alfa-beta prořezávání
 - algoritmus minimax umožňuje u hry dvou hráčů s nulovým součtem a s úplnou informací určit optimální strategie obou hráčů
 - procházíme strom hry
 - v listech nebo v určité hloubce získáme ohodnocení vrcholů
 - * omezení hloubky je důležité zvláště u her, jejichž stromy se hodně větví - je potřeba zvolit vhodný přístup k ohodnocení konkrétní herní situace
 - následně vždy v rodiči zvolíme minimum nebo maximum ze synů (v závislosti na tom, který hráč je zrovna na tahu - volba minima a maxima se střídají stejně jako tahy hráčů)
 - takto pro každý vrchol určíme, jaký tah je v dané situaci optimální
 - * za předpokladu, že protihráč hraje optimálně - což je nejhorší možný scénář
 - alfa-beta prořezávání
 - je to rozšíření minimaxu, které může zrychlit jeho běh tak, že omezuje počet větví, které je potřeba projít a vyhodnotit
 - vycházíme z následujícího pozorování
 - * v algoritmu typicky hledáme maximum z několika hodnot, které jsou minimy (nebo naopak)
 - * v určitém vrcholu máme seznam doposud určených minim z jednotlivých větví (z nich je nějaké maximální)
 - * snažíme se určit další minimum, na základě jedné z podvětví máme nějaké prozatímní minimum (museli bychom projít další podvětev pod daným vrcholem, abychom dostali výsledné minimum)
 - * pokud je tohle prozatímní minimum menší než prozatímní maximum (ve vyšší úrovni), tak už ty další větve nemusíme procházet, protože výsledné minimum nemůže být větší než prozatímní minimum

- alfa-beta prořezávání nemá vliv na výsledek algoritmu
- záleží na pořadí, ve kterém tahy (větvě) procházíme
- minimax se obvykle používá rekurzivně, meze určující prořezávání (pro jednoho hráče je to maximum, pro druhého minimum) se do funkce předávají jako parametry alfa a beta
- základy teorie her (věžňovo dilemma, Nashovo ekvilibrium)
 - jednou z úloh teorie her je volba vhodné strategie - tedy návodu, jak hru hrát
 - uvažujeme hry v normálním tvaru: hru tvoří hráči, jejich akce a užitkové funkce
 - hráči táhnou zároveň, každý provede jedinou akci
 - strategie může být čistá (přímo konkrétní akce) nebo smíšená (pravděpodobnostní rozdělení přes akce)
 - díváme se na optimalitu strategie nebo strategického profilu (množiny strategií všech hráčů) z různých úhlů pohledu
 - strategie s_1 dominuje strategii s_2 , pokud při volbě s_1 získá hráč větší užitek než při volbě s_2 nehlédě na strategie ostatních hráčů
 - * strategie je dominantní, pokud dominuje všechny ostatní strategie
 - *Nashovo ekvilibrium* je takový strategický profil, že by žádný hráč neměnil svou strategii, kdyby znal strategie ostatních
 - strategický profil s *Pareto dominuje* profil s' , pokud si změnou z s' na s žádný hráč nepohorší, ale aspoň jeden hráč si polepší
 - * strategický profil je *Pareto optimální*, pokud neexistuje profil, který by ho Pareto dominoval
 - další zajímavým typem ekvilibria je korelované ekvilibrium
 - věžňovo dilemma
 - dva hráči (vězni)
 - vězeň může vypovídat (testify/defect) nebo odmítnout vypovídat (refuse/cooperate)
 - * oba vypovídat → oba ztrácejí málo (užitek -5)
 - * jeden vypovídat, druhý ne → jeden neztratí nic (užitek 0), druhý hodně ztratí (užitek -10)
 - * oba odmítnou → oba ztrácejí velmi málo (užitek -1)
 - *vypovídat* je čistá dominantní strategie
 - * ale když vypovídat oba hráči, tak vyhrála policie - lepší by bylo, kdyby oba odmítli vypovídat
 - * našli jsme Nashovo ekvilibrium - nikdo neprofituje ze změny strategie, pokud druhý hráč zůstane u stejné strategie
- mechanism design (typy aukcí)
 - jak se pozná dobrý mechanismus aukce
 - maximalizuje výnos pro prodejce
 - vítěz aukce je agent, který si věci nejvíc cení
 - zájemci by měli mít dominantní strategii
 - anglická aukce (ascending-bid)
 - začnu s $b_{\min}$, pokud je to nějaký zájemce ochotný zaplatit, tak se ptám na $b_{\min} + d$
 - strategie: přihazuju, dokud cena není vyšší než moje hodnota
 - * dominantní strategie
 - holandská aukce
 - začíná se vyšší cenou, snižuje se, dokud ji někdo nepřijme
 - obálková aukce
 - největší nabídka vítězí

- neexistuje dominantní strategie
 - * pokud věříš, že maximum ostatních nabídek je b_0 , tak bys měl nabídnout $b_0 + \varepsilon$, pokud je to menší částka než hodnota, kterou pro tebe věc má
- obálková second-price aukce (Vickrey)
 - vyhraje ten, kdo nabídne nejvyšší cenu, platí druhou nejvyšší nabídnutou cenu
 - dominantní strategie je tam dát svoji hodnotu
- VCG mechanismus
 - lze použít při obecnějších (víceparametrických) aukcích

6.9 Strojové učení

- základní druhy učení (s učitelem, bez učitele, zpětnovazební)
 - učení bez učitele (unsupervised learning) - agent se učí vzory ve vstupu, aniž by měl nějakou explicitní zpětnou vazbu
 - zpětnovazební učení (reinforcement learning) - agent dostává odměny nebo tresty podle toho, jak se chová v prostředí
 - učení s učitelem (supervised learning) - agent se učí funkci, která mapuje vstupy na výstupy
- rozhodovací stromy (definice, konstrukce)
 - přijímá vektor hodnot atributů, vrací výslednou hodnotu
 - na základě tabulky příkladů se dá postavit strom
 - každý vnitřní vrchol odpovídá testu jednoho atributu
 - dá se zjistit odůvodnění konkrétního rozhodnutí
 - strom se dá postavit hladovou metodou rozdělení a panuj
 - * k dělení se používá entropie
 - každý list určuje hodnotu, kterou klasifikační funkce vrátí
- regrese, SVM (základní principy)
 - při regresi přiřazujeme položce x vhodnou funkční hodnotu y
 - je to jedna z úloh strojového učení s učitelem
 - support-vector machine (SVM)
 - stojí na lineární regresi
 - pokud lze třídy oddělit nadrovinou, zvolí takovou, která je nejdál od všech dat (příkladů)
 - nadrovina ... maximum margin separator
 - pokud nejde použít nadrovinu, provede mapování do vícedimenzionálního prostoru (pomocí kernel function), kde už příklady půjde oddělit
 - SVMs jsou neparametrická metoda - příklady blíže k separátoru jsou důležitější, říká se jim support vectors (vlastně definují ten separátor)
- Bayesovské učení, EM algoritmus
 - bayesovské učení
 - máme několik hypotéz
 - na základě pozorování upravujeme jejich pravděpodobnost
 - *maximálně věrohodná hypotéza*
 - * z množiny hypotéz je to ta, která přiřazuje naměřeným datům největší pravděpodobnost
 - * $\operatorname{argmax}_h p(\text{data} | h)$ nebo také $\operatorname{argmax}_h p_h(\text{data})$
 - * pokud je hypotéza parametrizovaná (hledáme parametr), tak můžeme hledat hodnotu parametru, pro níž je derivace logaritmu pravděpodobnosti nulová
 - tomu se říká maximum-likelihood parameter learning

- z $p(\text{data} \mid h)$ můžeme získat $p(h \mid \text{data}) = \frac{p(\text{data} \mid h)p(h)}{\sum_{h'} p(\text{data} \mid h')p(h')}$
 - * pokud mají všechny hypotézy stejné apriorní pravděpodobnosti, stačí vzít $p(\text{data} \mid h)$ a normalizovat je, jelikož $\frac{p(h)}{\sum_{h'} p(\text{data} \mid h')p(h')}$ bude konstantní
- *bayesovsky optimální predikce* pravděpodobnosti pro vstupní data a konkrétní predikci z
 - * $p(z \mid \text{data}) = \sum_h p(z \mid h) \cdot p(h \mid \text{data})$
- algoritmus expectation-maximization (EM)
 - používá se k maximum-likelihood odhadu, pokud se nedá jednoduše spočítat
 - předstíráme, že známe parametry modelu (na začátku je určíme náhodně)
 - iterujeme dva kroky (dokud to nekonverguje)
 - * dopočteme střední hodnoty skrytých proměnných (E-step, expectation)
 - * upravíme parametry, abychom maximalizovali likelihood modelu (M-step, maximization)
 - příklad
 - * chceme odhadnout průměrnou výšku mužů a žen
 - * máme data o naměřených výškách, ale chybí nám údaje o pohlaví (zda je to výška muže nebo ženy)
 - třídy označíme jako 0 (muži), 1 (ženy)
 - * začneme tak, že oba parametry (průměrná výška mužů μ_0 , průměrná výška žen μ_1) inicializujeme náhodně
 - * v E-kroku spočítáme střední hodnoty skrytých proměnných
 - pohlaví je skrytá proměnná (pro každou naměřenou výšku)
 - střední hodnota bude odpovídat pravděpodobnosti, že konkrétní výška patří ženě
 - * v M-kroku upravíme hodnoty μ_0, μ_1 , abychom maximalizovali likelihood modelu
 - tedy spočítáme vážený průměr výšek
 - váhy odpovídají pravděpodobnosti, že výška patří muži (u μ_0) / ženě (u μ_1)
 - * poznámka: může se jednoduše stát, že nám parametry μ_0, μ_1 vyjdou naopak (EM je metoda učení bez učitele, hledá vzory v datech, nezajišťuje jejich interpretaci)
- pasivní zpětnovazební učení (definice, metody ADP a TD)
 - uvažujeme agenta, který se účastní MDP (markovského rozhodovacího procesu)
 - pasivní učení - známe strategii π (je pevně daná) a učíme se, jak je dobrá (učíme se utility funkci)
 - agent nezná přechodový model ani reward function
 - přímý odhad utility
 - máme trace (běh) mezi stavy, z toho počítáme utility function
 - odhadujeme užitky jednotlivých stavů
 - * užitek stavu = *reward-to-go* (tedy odměna tohoto a všech budoucích stavů)
 - * odměny z budoucích stavů bychom mohli zohledňovat s discount factorem γ (viz výše)
 - užitky pro jednotlivé stavy prostě průměrujeme
 - v podstatě učení s učitelem - na vstupu je stav, na výstupu užitek stavu (neboli *reward-to-go*)
 - nevýhoda
 - * utilities nejsou nezávislé, ale řídí se Bellmanovými rovnicemi
 - * hledáme v prostoru hypotéz, který je výrazně větší, než by musel být (obsahuje spoustu funkcí, co porušují Bellmanovy rovnice) $\rightarrow$ algoritmus konverguje pomalu
 - adaptivní dynamické programování (ADP)
 - agent se učí přechodový model a odměny
 - přechodový model odhaduje přímo z pozorovaných přechodů

- utilitu počítá z Bellmanových rovnic třeba pomocí value iteration
 - * využíváme toho, že užítky nejsou nezávislé
- ADP zajišťuje, že jsou odhady užitek konzistentní
- temporal-difference (TD) learning
 - upravuje stav, aby odpovídal aktuálně pozorovanému následníkovi
 - když dojde k přechodu ze stavu s do stavu s' , tak na $U^\pi(s)$ použijeme update: $U^\pi(s) \leftarrow U^\pi(s) + \alpha \cdot (R(s) + \gamma \cdot U^\pi(s') - U^\pi(s))$
 - * α je learning rate
 - vlastnosti
 - * nepotřebuju přechodový model (je to model-free metoda)
 - * konverguje to pomaleji než ADP
- aktivní zpětnovazební učení (definice, explorace vs. exploitace, Q-učení, SARSA)
 - uvažujeme agenta, který se účastní MDP (markovského rozhodovacího procesu)
 - aktivní učení - agent se učí strategii (policy; jak se rozhodovat) a taky utility funkci
 - mohli bychom použít ADP, tím vznikne hladový agent, který opakuje zažité vzory (nezkouší nové věci)
 - jak může volba optimální akce vést k suboptimálním výsledkům?
 - naučený model není stejný jako reálné prostředí
 - akce nejsou pouze zdrojem odměn, ale také přispívají k učení - ovlivňují vstupy
 - zlepšováním modelu se postupně zvětšují odměny, které agent získává
 - je potřeba najít optimum mezi exploration a exploitation
 - pure exploration - agent nepoužívá naučené znalosti
 - pure exploitation - riskujeme, že bude agent pořád dokola opakovat zažité vzory
 - základní myšlenka: na začátku preferujeme exploration, později lépe rozumíme světu, takže nepotřebujeme tolik prozkoumávat
 - exploration policies
 - první možná exploration policy: agent si zvolí náhodnou akci s pravděpodobností $\frac{1}{t}$ (kde t je čas), jinak se řídí hladovou strategií
 - * nakonec to konverguje k optimální strategii, ale může to být extrémně pomalé
 - rozumnější přístup je přiřadit váhu akcím, které agent ještě nevyzkoušel
 - existuje alternativní TD metoda, říká se jí Q-learning
 - * $Q(s, a)$ označuje hodnotu toho, že provedeme akci a ve stavu s
 - * q-hodnoty jsou s utilitou ve vztahu $U(s) = \max_a Q(s, a)$
 - * můžeme napsat omezující podmínku $Q(s, a) = R(s) + \gamma \sum_{s'} P(s' | s, a) \cdot \max_{a'} Q(s', a')$
 - tohle vyžaduje, aby se model naučil i $P(s' | s, a)$
 - * Q-learning ale nevyžaduje model přechodů - je to bezmodelová (model-free) metoda, potřebuje akorát q-hodnoty
 - * $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot (R(s) + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a))$
 - počítá se, když je akce a vykonána ve stavu s a vede do stavu s'
 - state-action-reward-state-action (SARSA)
 - * je to varianta Q-learningu
 - * $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot (R(s) + \gamma \cdot Q(s', a') - Q(s, a))$
 - pravidlo se aplikuje na konci pětice s, a, r, s', a' , tedy po aplikaci akce a'
 - pro hladového agenta (který volí podle největšího Q) jsou algoritmy SARSA a základní Q-learning stejné
 - * rozdíl je vidět až u agenta, který není úplně hladový, ale provádí i průzkum (exploration)

- u SARSA se bere v úvahu reálně zvolená akce
 - * SARSA je on-policy - jeho strategie se upravuje přímo za běhu algoritmu
 - * Q-learning je off-policy - algoritmus se učí optimální strategii, ale během učení se může používat úplně jiná strategie
- při aktivním učení je výhodnější učit se q-hodnoty než užitkovou funkci
 - abychom našli nejvýhodnější akci v daném stavu pomocí utility, museli bychom při výpočtu použít pravděpodobnost, že danou akci přejdeme do určitého stavu
 - q-hodnota nám dá přímo říkne, jak je užitečné provést danou akci - k volbě nejlepší akce nepotřebujeme přechodový model

7 Část IV: Specializace - Strojové učení

7.1 Učení s učitelem

- klasifikace, regrese (základní typy úloh)
 - úloha klasifikace spočívá v přiřazení vhodné třídy (nebo sady tříd) konkrétní položce
 - typická je binární klasifikace
 - při regresi přiřazujeme položce x vhodnou funkční hodnotu y
- standardní evaluační metriky
 - regrese
 - mean square error, $MSE = \frac{1}{N} \sum_{i=1}^N (y_i - t_i)^2$
 - * y_i ... predikce funkční hodnoty pro x_i
 - * t_i ... reálná funkční hodnota pro x_i (v trénovacích/testovacích datech)
 - klasifikace
 - negative log likelihood, $NLL = -\sum_{i=1}^N \log p_i(t_i)$
 - * t_i ... reálná třída x_i
 - * p_i ... pravděpodobnostní rozdělení, které model vrací pro x_i (pravděpodobnosti, že x_i náleží jednotlivým třídám)
 - * místo $p_i(t_i)$ lze také zapsat $p(t_i | x_i)$
 - accuracy (přesnost)
 - * počet správně klasifikovaných $\div$ celkový počet
 - * $\frac{tp+tn}{tp+tn+fp+fn}$ (u binární klasifikace)
 - * nevhodná metrika, pokud jsou třídy nevyvážené (např. testujeme vzácné onemocnění)
 - confusion matrix
 - * řádky jsou označeny pravdivými hodnotami, sloupce predikovanými hodnotami (nebo naopak - je to potřeba uvést)
 - * v každé buňce je počet testovacích dat s konkrétní kombinací targetu a predikce
 - * na diagonále jsou tedy počty správně klasifikovaných testovacích dat
 - * při binární klasifikaci buňky vlastně obsahují počty TP, FN, FP, TN
 - * [obr.: confusion matrix]
 - binární klasifikace (navíc)
 - základní dělení výsledků klasifikace
 - * predikce je pozitivní, realita (target) rovněž $\rightarrow$ true positive (TP)
 - * predikce je pozitivní, realita je negativní $\rightarrow$ false positive (FP)
 - * predikce je negativní, realita je pozitivní $\rightarrow$ false negative (FN)
 - * predikce je negativní, realita také $\rightarrow$ true negative (TN)
 - precision, $P = \frac{tp}{tp+fp}$
 - recall, $R = \frac{tp}{tp+fn}$
 - F-measure
 - * vážený harmonický průměr precision a recallu
 - * $F_\beta = \frac{(\beta^2+1)PR}{\beta^2 P + R} = \frac{tp+\beta^2 \cdot tp}{tp+fp+\beta^2(tp+fn)}$
 - * $F_1 = \frac{2PR}{P+R} = \frac{tp+tp}{tp+fp+tp+fn}$
 - * je užitečné mít jednu hodnotu místo dvou (P, R)
 - ROC (receiver operating characteristic) křivka
 - * zachycuje, jak se mění vlastnosti binárního klasifikátoru, když posouváme práh (threshold, tzn. hodnotu, od níž výsledek prohlásíme za pozitivní - typicky uvažujeme

0.5)

- * na ose x je false positive rate, $FPR = \frac{fp}{fp+tn}$ (v čitateli jsou všechny negativní instance)
- * na ose y je true positive rate (recall), $TPR = \frac{tp}{tp+fn}$ (v čitateli jsou všechny pozitivní instance)
- * každý bod na křivce odpovídá jinému thresholdu
- * pro náhodný „klasifikátor“ bude ROC křivka diagonála
- * pro dokonalý klasifikátor bude ROC křivka jeden bod vlevo nahoře
 - $FPR = 0, TPR = 1$
- AUC (area under curve)
 - * plocha pod ROC křivkou
 - * pro náhodný „klasifikátor“ je $AUC = 0.5$, pro dokonalý klasifikátor to bude $AUC = 1$
- hodnoty metrik typu true positive rate, false negative rate, ... se vždycky určují tak, že v čitateli jsou všechny instance odpovídající názvu a ve jmenovateli jsou všechny instance, co sdílí „realitu“ s čitatelem
 - * např. true negative rate = true negative / healthy
 - = true negative / (true negative + false positive)
 - * false negative rate = false negative / sick
 - = false negative / (false negative + true positive)
- senzitivita ... true positive rate (recall)
 - * pravděpodobnost pozitivního výsledku u pozitivního (nemocného) jedince
- specificita ... true negative rate
 - * pravděpodobnost negativního výsledku u negativního (zdravého) jedince
- ohodnocení modelu (testovací data, křížová validace, maximální věrohodnost)
 - testovací data slouží k ověření toho, jak dobře náš model generalizuje
 - snažíme se zjistit, jak dobré výsledky model vrací pro data, která dosud neviděl
 - testovací data nesmím použít v procesu trénování modelu a ladění hyperparametrů
 - pokud je potřeba ladit hyperparametry, používá se train-dev-test split
 - parametry modelu trénujeme na trénovacích datech
 - hyperparametry určujeme pomocí vývojových dat (zjistíme, pro které hodnoty hyperparametrů má model nejlepší výkon)
 - * vývojová data se někdy označují jako validační (validation set)
 - k finálnímu vyhodnocení modelu slouží testovací data
 - pokud mám dat málo a nelze z nich jednoduše vyčlenit testovací (nebo vývojová) data, používá se křížová validace (cross-validation)
 - trénovací data rozdělím na n dílů
 - model natrénuju na datech z $n - 1$ dílů
 - na zbylých datech model vyhodnotím
 - proces opakuju pro n možných voleb
 - jako výsledné skóre modelu použiju průměr průběžných hodnocení
 - maximální věrohodnost (maximum likelihood)
 - máme-li fixní trénovací data, pak likelihood $L(w)$ (kde w jsou váhy modelu) se rovná součinu pravděpodobností targetů trénovacích dat
 - odhadujeme váhy modelu tak, aby byl likelihood (věrohodnost) trénovacích dat co největší
 - přesněji
 - * pro model s fixními vahami w máme pravděpodobnostní distribuci $p_{\text{model}}(x; w)$, že model vygeneruje x

- * pro model s fixními vahami w a konkrétní řádek x máme pravděpodobnostní distribuci $p_{\text{model}}(t | x; w)$, jak model klasifikuje x
- * místo toho zafixujeme trénovací data a budeme měnit váhy $\rightarrow$ dostaneme $L(w) = p_{\text{model}}(X; w) = \prod_i p_{\text{model}}(x_i; w)$
- * $w_{\text{MLE}} = \arg \max_w p_{\text{model}}(X; w)$
 - MLE ... maximum likelihood estimation
- přeučení a regularizace (generalizační chyba, včasné zastavení trénování, L2 a L1 regularizace)
 - underfitting - model je příliš slabý (příliš jednoduchý), plně jsme nevyužili jeho potenciál
 - overfitting (přeučení) - model špatně generalizuje, příliš se přizpůsobil trénovacím datům
 - generalizační chyba
 - zachycuje, jak dobře model generalizuje
 - typicky odpovídá výkonu modelu na testovacích datech (případně lze k jejímu určení použít křížovou validaci)
 - * výkon = vhodná metrika (F-measure, accuracy, ...)
 - generalizační chybu lze zachytit v grafu vedle trénovací chyby (výkonu modelu na trénovacích datech)
 - pokud se testovací (generalizační) křivka přibližovala k trénovací a v určitý moment se začala vzdalovat, došlo k přetrénování (overfitting) modelu
 - * je potřeba zesílit regularizaci nebo včas zastavit trénování
 - pokud se testovací křivka ani nezačala přibližovat, je potřeba trénovat déle
 - regularizace
 - intuice
 - * penalizujeme modely s většími vahami
 - * preferujeme menší změny modelu
 - * omezujeme efektivní kapacitu modelu
 - * čím je model složitější, tím větší má váhy
 - ke ztrátové funkci (loss $\rightsquigarrow \cdot$; $\rightsquigarrow$) přičteme normu vah vynásobenou parametrem λ
 - * L^1 regularizace ... $\lambda(|w_1| + |w_2| + \dots + |w_n|)$
 - * L^2 regularizace ... $\lambda(w_1^2 + w_2^2 + \dots + w_n^2)$
 - nebo také $\lambda\|w\|^2$, případně $\frac{\lambda}{2}\|w\|^2$, aby nám vyšla hezká derivace
 - pozor, při regularizaci obvykle vynecháváme váhu, která odpovídá biasu
 - jak bojovat s overfittingem
 - L^2 regularizací
 - early stoppingem (s trénováním přestaneme, když začne validační chyba růst)
 - vhodným nastavením learning ratu α (případně volbou rozumného learning rate schedule)
- prokletí dimenzionality
 - s rostoucím počtem dimenzí (features) potřebujeme čím dál víc (exponenciálně) trénovacích dat, aby model rozumně generalizoval
 - chceme mít v trénovacích datech několik vzorků pro každou kombinaci features $\rightarrow$ s každou další dimenzí potřebujeme násobně víc vzorků
 - k redukcí počtu dimenzí se obvykle používají následující přístupy
 - filter
 - * nezávislý na modelu, features vybírá na základě jejich vlastností
 - * k hodnocení relevance features se používají statistické metody
 - konzistence - nemůžeme odstranit features, pokud bychom tak ztratili možnost odlišit prvky s různými targety (třídami)
 - korelace - dobré features korelují s třídami a nekorelují s jinými features

- vzdálenost mezi třídami
- metriky z teorie informace - např. vzájemná informace
- wrapper
 - * závislý na modelu, features vybírá tak, aby optimalizoval jeho výsledky
 - * model se natrénuje, pak se vyhodnotí pomocí standardních technik (accuracy, F-measure apod.)

7.2 Učení založené na příkladech

- algoritmus k-nejbližších sousedů
 - regrese: $t = \sum_i \frac{w_i}{\sum_j w_j} \cdot t_i$
 - vážený průměr
 - klasifikace
 - pro uniformní váhy můžeme hlasovat (nejčastější třída vyhrává, remízy rozhodujeme náhodně)
 - jinak uvažujeme one-hot kódování t_i opět můžeme použít predikci $t = \sum_i \frac{w_i}{\sum_j w_j} \cdot t_i$ (zvolíme třídu s největší predikovanou pravděpodobností)
 - volba vah (weighting)
 - uniform: všech k nejbližších sousedů má stejné váhy
 - inverse: váhy jsou nepřímo úměrné vzdálenosti
 - softmax: váhy jsou přímo úměrné softmaxu záporných vzdáleností
 - jak určit nejbližší sousedy / vzdálenosti?
 - p -norma: $\|x\|_p = \sqrt[p]{\sum_i |x_i|^p}$
 - obvykle se používá $p \in \{1, 2, 3, \infty\}$
 - vzdálenost x a y pak lze určit jako $\|x - y\|_p$

7.3 Lineární regrese

- analytické řešení metodou nejmenších čtverců
 - sum of squares error: $\frac{1}{2} \sum_i (x_i^T w - t_i)^2$
 - lze ho zapsat jako $\frac{1}{2} \|Xw - t\|^2$
 - snažíme se ho minimalizovat $\rightarrow$ hledáme hodnoty, kde je derivace chybové funkce podle každé z vah nulová
 - $\frac{\partial}{\partial w_j} \frac{1}{2} \sum_i (x_i^T w - t_i)^2 = \sum_i x_{ij} (x_i^T w - t_i)$
 - chceme, aby $\forall j : \sum_i x_{ij} (x_i^T w - t_i) = 0$
 - * $X^T (Xw - t) = 0$
 - * $X^T Xw = X^T t$
 - je-li $X^T X$ invertibilní, pak lze určit explicitní řešení jako $w = (X^T X)^{-1} X^T t$
 - přičemž bias reprezentujeme jako součást vah, takže matici X musíme rozšířit o sloupec jedniček
- trénování pomocí stochastic gradient descent
 - gradient descent
 - snažíme se minimalizovat hodnotu chybové funkce $E(w)$ pro dané váhy w volbou lepších vah
 - * spočítáme $\nabla_w E(w) \rightarrow$ tak zjistíme, jak váhy upravit

- * nastavíme $w \leftarrow w - \alpha \nabla_w E(w)$
 - $\alpha \dots$ learning rate (hyperparametr), „délka kroku“
- * tomu se říká gradient descent
- standard gradient descent - používáme všechna trénovací data k výpočtu $\nabla_w E(w)$
- stochastic (online) gradient descent - používáme jeden náhodný řádek
- minibatch stochastic gradient descent - používáme B náhodných nezávislých řádků
- algoritmus pro trénink modelu lineární regrese (minibatch SGD)
 - náhodně inicializujeme w (nebo nastavíme na nulu)
 - opakujeme, dokud to nezkonverguje nebo nám nedojde trpělivost
 - * samplujeme minibatch řádků s indexy $\mathbb{B}$
 - buď uniformně náhodně, nebo budeme chtít postupně zpracovat všechna trénovací data (to se obvykle dělá, jednomu takovému průchodu se říká epocha), tedy je nasekáme na náhodné minibatche a procházíme postupně
 - * $w \leftarrow w - \alpha \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} ((x_i^T w - t_i)x_i) - \alpha \lambda w$
 - jako E se používá polovina MSE (s L^2 regularizací): $E(w) = \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} \frac{1}{2} (x_i^T w - t_i)^2 + \frac{\lambda}{2} \|w\|^2$ (stačí to zderivovat)

7.4 Logistická regrese

- binární klasifikace (sigmoid, metody trénování)
 - aktivační funkce sigmoid $\dots \sigma(x) = \frac{1}{1+e^{-x}}$
 - predikce $y(x; w) = \sigma(\bar{y}(x; w)) = \sigma(x^T w)$
 - správnou třídu získáme zaokrouhlením
 - přesná hodnota se rovná pravděpodobnosti, že je x klasifikováno jako 1 (uvažujeme třídy 0 a 1)
 - $\bar{y} \dots$ „lineární složka“ logistické regrese
 - velikost vektoru w odpovídá počtu features
 - nesmíme zapomenout také na bias, ten opět může být reprezentován jako další váha
 - při trénování se jako ztrátová funkce používá negative log likelihood (NLL)
 - pravděpodobnost targetu t pro položku x lze vyjádřit jako $p(t | x; w) = \sigma(x^T w)^t (1 - \sigma(x^T w))^{1-t}$
 - algoritmus trénování pomocí minibatch SGD
 - klasifikujeme do tříd $\{0, 1\}$, na vstupu máme dataset X a learning rate $\alpha \in \mathbb{R}^+$
 - náhodně inicializujeme w (nebo nastavíme na nulu)
 - opakujeme, dokud to nezkonverguje nebo nám nedojde trpělivost
 - * samplujeme minibatch řádků s indexy $\mathbb{B}$
 - * $w \leftarrow w - \alpha \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} (\sigma(x^T w) - t)x - \alpha \lambda w$
 - jako E se používá NLL (s L^2 regularizací)
 - výsledek se překvapivě podobá lineární regresi
 - gradient je také $(y(x) - t)x$
 - klasifikace do více tříd (softmax, metody trénování)
 - klasifikujeme do jedné z K tříd
 - parametry modelu: váhová matice W (sloupce odpovídají třídám, řádky featurám), vektor biasů (jeden bias za každou třídu)
 - aktivační funkce $\dots \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$

- softmax je invariantní k přičtení konstanty $\text{softmax}(z + c)_i = \text{softmax}(z)_i$
- $\sigma(x) = \text{softmax}([x, 0])_0 = \frac{e^x}{e^x + e^0} = \frac{1}{1 + e^{-x}}$
- klasifikace: $p(C_i | x; W) = y(x; W)_i = \text{softmax}(\bar{y}(x; W))_i = \text{softmax}(x^T W)_i$
 - softmax zajišťuje normalizaci (aby se pravděpodobnosti sečetly na jedničku)
- z linearity $\bar{y}$ lze dokázat, že oblasti tříd jsou konvexní - nemůže se stát, že by na úsečce mezi dvěma body v jedné třídě byl bod v jiné třídě
- algoritmus trénování pomocí minibatch SGD
 - klasifikujeme do tříd $\{0, \dots, K - 1\}$, na vstupu máme dataset X a learning rate $\alpha \in \mathbb{R}^+$
 - náhodně inicializujeme w (nebo nastavíme na nulu)
 - opakujeme, dokud to nezkonverguje nebo nám nedojde trpělivost
 - * samplujeme minibatch řádků s indexy $\mathbb{B}$
 - * $W \leftarrow W - \alpha \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} \left((\text{softmax}(x_i^T W) - 1_{t_i}) x_i^T \right)^T - \alpha \lambda W$
 - jako E se používá NLL (s L^2 regularizací)
 - srovnání gradientu logistické regrese
 - binary: $(y(x) - t)x$
 - multiclass: $((y(x) - 1_t)x^T)^T$
 - přičemž 1_t je one-hot reprezentace targetu t (tedy vektor s jedničkou na t -té pozici a nulami všude jinde)

7.5 Rozhodovací stromy

- algoritmus učení a kritéria větvení
 - v každém vnitřním vrcholu je uložena feature, podle které dělíme, a konkrétní hodnota, která určuje rozhraní
 - každému vrcholu náleží podmnožina trénovacích dat
 - predikce listu odpovídá tomu, jaká data v něm jsou (při regresi průměrujeme, při klasifikaci volíme největší třídu = většinové hlasování / majority vote)
 - možná kritéria větvení ve vrcholu $\mathcal{T}$
 - squared error criterion (používá se při regresi)
 - * squared error criterion říká, jak homogenní je podmnožina trénovacích dat, která náleží vrcholu $\mathcal{T}$
 - * $c_{SE}(\mathcal{T}) = \sum_{i \in I_{\mathcal{T}}} (t_i - \hat{t}_{\mathcal{T}})^2$
 - * $I_{\mathcal{T}}$ jsou indexy řádků trénovacích dat, které náleží danému vrcholu
 - * $\hat{t}_{\mathcal{T}}$ je průměr targetů v daném vrcholu
 - Gini index / Gini impurity (používá se při klasifikaci)
 - * $c_{Gini}(\mathcal{T}) = |I_{\mathcal{T}}| \sum_k p_{\mathcal{T}}(k)(1 - p_{\mathcal{T}}(k))$
 - * jak často by byl náhodně vybraný prvek špatně klasifikován, kdyby byl klasifikován náhodně podle rozdělení $p_{\mathcal{T}}$
 - $p_{\mathcal{T}}(k) \dots$ pravděpodobnost, že vybereme prvek ze třídy k
 - $(1 - p_{\mathcal{T}}(k)) \dots$ pravděpodobnost, že mu přiřadíme jinou třídu než k
 - * v binární klasifikaci Gini odpovídá squared error ztrátové funkci (skoro MSE)
 - entropy criterion (používá se při klasifikaci)
 - * $c_{entropy}(\mathcal{T}) = |I_{\mathcal{T}}| \cdot H(p_{\mathcal{T}}) = -|I_{\mathcal{T}}| \sum_k p_{\mathcal{T}}(k) \log p_{\mathcal{T}}(k)$
 - * ze součtu vynecháme třídy s nulovou pravděpodobností, aby nám nerozbily logaritmus
 - * entropy criterion lze odvodit ze ztrátové funkce negative log likelihood (NLL)
 - algoritmus CART (Classification and Regression Trees)

- trénovací data jsou uložena v listech
- začneme s jedním listem
- list můžeme rozdělit (stane se z něj vnitřní vrchol a pod něj připojíme dva nové listy)
- list dělíme tak, aby rozdíl $c_{\mathcal{T}_L} + c_{\mathcal{T}_R} - c_{\mathcal{T}}$ byl co nejmenší
 - * $c_{\mathcal{T}}$... criterion současného vrcholu
 - * $c_{\mathcal{T}_L}$... criterion nového levého syna
 - * $c_{\mathcal{T}_R}$... criterion nového pravého syna
- někdy stanovujeme omezení: maximální hloubku stromu, minimální počet dat v děleném listu (abychom nedělili zbytečně listy, co mají málo dat), maximální počet listů
 - * obvykle se listy dělí prohledáváním do hloubky
 - * pokud je stanoven maximální počet listů, je vhodnější je dělit ty s nejmenším rozdílem $c_{\mathcal{T}_L} + c_{\mathcal{T}_R} - c_{\mathcal{T}}$
- dosud nás vždycky zajímaly parametry, které minimalizují ztrátovou funkci
- tím, že minimalizujeme dělicí kritérium, minimalizujeme minimální dosažitelnou hodnotu ztrátové funkce pro daný strom
- prořezávání
 - prořezávání (pruning) lze použít ke zjednodušení rozhodovacího stromu, tím je možné omezit overfitting
 - prořezávat lze shora dolů nebo zespoda nahoru (bottom-up pruning $\times$ top-down pruning)
 - příklad: reduced error pruning
 - bottom-up přístup
 - na testovacích datech ověřujeme přesnost (accuracy)
 - postupně zkoušíme rušit větvení ve vnitřních vrcholech - pokud tím nedojde ke snížení přesnosti, tak změnu provedeme
 - * tedy se z vnitřního vrcholu stane list, který predikuje třídu s největším počtem položek

7.6 Metoda podpůrných vektorů

- klasifikátor pro lineárně separabilní třídy
 - hard-margin SVM (support-vector machine)
 - je to kernelová (jádrová) metoda strojového učení
 - model má k dispozici spoustu polynomiálních features, aniž bychom je museli explicitně počítat
 - používáme jádrovou funkci K
 - model je neparametrický - využívá se duální formulace
 - nemáme váhy, ale koeficienty u trénovacích dat
 - duální formulace vychází z pozorování, že klasické váhy jsou lineárními kombinacemi trénovacích dat
 - princip je podobný jako u perceptronu
 - jedná se o binární klasifikaci do tříd $\{-1, 1\}$
 - tentokrát ale chceme zajistit, aby byla dělicí nadrovina (decision boundary) co nejlepší - aby byl margin (mezera mezi nadrovinou a body) co největší
 - jak se odvozuje
 - uvažujeme váhy w a bias b , dále mapování φ do prostoru features, predikci označíme $y(x) = \varphi(x)^T w + b$
 - pro všechny body chceme maximalizovat vzdálenost od decision boundary
 - * váhy i bias můžeme libovolně škálovat, proto určíme, že pro body nejbližší k decision

- boundary bude platit $t_i y(x_i) = 1$
 - * díky tomu nám stačí „minimalizovat váhy“ takto: $\operatorname{argmin}_{w,b} \frac{1}{2} \|w\|^2$
 - za předpokladu, že $t_i y(x_i) \geq 1$
 - tuhle minimalizaci s omezující podmínkou provádíme pomocí lagrangiánu a KKT podmínek
 - * místo multiplikátorů λ_i se tradičně používají a_i
 - díky KKT podmínkám víme, že nás při maximalizaci lagrangiánu zajímají jenom některá trénovací data - ta, která leží na okraji (support vektory)
 - * je tam podmínka $a_i(t_i y(x_i) - 1) = 0$
 - * z toho vyplývá, že pro každé x_i platí $t_i y(x_i) = 1$ (bod je na marginu) nebo $a_i = 0$ (bod se nepoužívá k predikci)
 - predikce $\dots y(x) = \sum_i a_i t_i K(x, x_i) + b$
- klasifikátor pro lineárně neseparabilní třídy
 - soft-margin SVM
 - v mnohém se podobá hard-margin SVM
 - dovolíme některým bodům, aby porušily pravidlo (aby byly uvnitř marginu nebo na opačné straně decision boundary)
 - pořídíme si slack variables $\xi_i \geq 0$
 - pro body, které porušují $t_i y(x_i) \geq 1$ bude platit $\xi_i = |t_i - y(x_i)|$
 - jinak $\xi_i = 0$
 - takže podmínku $t_i y(x_i) \geq 1$ nahradíme $t_i y(x_i) \geq 1 - \xi_i$
 - úpravy lagrangiánu
 - musíme tam přidat další multiplikátory, které zajistí nezápornost ξ_i
 - výsledný lagrangián bude stejný jako minule, akorát a_i (odpovídají λ ům) budou navíc omezeny shora nějakým C
 - support vektory teďka nebudou jenom body na okrajích marginu, ale i všechny, které mají kladné ξ_i (jsou divné)
 - pozorování: $\xi_i = \max(0, 1 - t_i y(x_i))$
 - téhle funkci se říká hinge loss
 - soft-margin SVM lze vlastně formulovat jako $\operatorname{argmin}_{w,b} C \sum_i \mathcal{L}_{\text{hinge}}(t_i, y(x_i)) + \frac{1}{2} \|w\|^2$
 - to nám hodně připomíná klasický vzorec pro logistickou regresi apod.
 - akorát místo regularizační konstanty λ tady máme C , které hraje opačnou roli (čím větší C , tím slabší regularizace)
 - k trénování se používá sequential minimal optimization
 - bereme řádky po jednom, k tomu a_i , náhodně a_j a taky bias, postupně zlepšujeme lagrangián
- jádrové funkce
 - idea
 - uvažujeme duální formulaci modelu (místo vah máme koeficienty, kterými přispívají řádky trénovacích dat)
 - máme polynomiální features (pomocí zobrazení φ)
 - při predikci bychom museli N -krát počítat $\varphi(x_i)^T \varphi(z)$
 - když si ale například představíme features 0. až 3. řádu, tak si můžeme všimnout toho, že $\varphi(x)^T \varphi(z) = 1 + x^T z + (x^T z)^2 + (x^T z)^3$
 - zobecnění
 - budeme uvažovat kernelové funkce (kernely), které mají všechny následující vzorec (pro různé φ): $K(x, z) = \varphi(x)^T \varphi(z)$
 - kernel je tedy funkce, která dostane dva vektory a ona mi vrátí součin features těch vektorů

- výše jsme si jeden takový kernel ukázali
- polynomiální kernel stupně d (homogenní polynomiální kernel)
 - vyrábí kombinace d vstupních features
 - $K(x, z) = (\gamma x^T z)^d$
 - γ je hyperparametr, který škáluje features
- polynomiální kernel stupně nejvýše d (nehomogenní polynomiální kernel)
 - $K(x, z) = (\gamma x^T z + 1)^d$
- Gaussian Radial basis function (RBF) kernel
 - obsahuje polynomiální features všech řádů
 - $K(x, z) = e^{-\gamma \|x - z\|^2}$
 - je to funkce vzdálenosti
 - * pro stejné vektory vrací jedničku, pro hodně vzdálené vektory vrací nulu, pro ostatní něco mezi tím
 - * dá se to interpretovat jako rozšířenou verzi algoritmu k nejbližších sousedů (k -NN)
 - * γ ovlivňuje, co už je „moc daleko“
 - polynomiální features vysokých řádů mají nízké váhy
- kernely se typicky používají tam, kde potřebujeme lineárně separabilní data (například v SVM) - transformují nám původní data do více dimenzí, tam už je pak obvykle můžeme rozumně separovat
- klasifikace do více tříd
 - kombinujeme několik binárních klasifikátorů
 - první možný (nepoužívaný!) přístup: one-versus-rest (ovr) schéma
 - natrénuju K nezávislých binárních klasifikátorů (patří prvek konkrétní třídě? ano/ne)
 - při predikci vyberu třídu, které její klasifikátor přisuzuje největší pravděpodobnost
 - je potřeba klasifikátory kalibrovat, aby se ty pravděpodobnosti daly porovnávat!
 - takhle se to u SVM nedělá
 - alternativní přístup: one-versus-one schéma
 - klasifikátor na každou dvojici tříd, které existují
 - většinové hlasování (každý klasifikátor hlasuje pro jednu třídu)
 - klasifikátorů je víc, ale trénujeme je na méně datech

7.7 Kombinace více modelů

- bagging a boosting
 - jedná se o dva různé přístupy k trénování více modelů
 - bagging
 - modely se trénují nezávisle
 - pro každý model náhodně vybereme vzorek trénovacích dat, na nichž se učí (typicky samplujeme s opakováním = bootstrapping)
 - boosting
 - modely se trénují sekvenčně
 - snažíme se postupně opravovat chyby předchozích modelů
 - princip algoritmu AdaBoost
 - * postupně trénujeme slabé klasifikátory (např. mělké rozhodovací stromy/pařezy) na vážených trénovacích datech
 - * v každé iteraci upravujeme váhy trénovacích dat, aby špatně klasifikovaná data měla větší váhy

- * rovněž přiřazujeme váhy klasifikátorům podle jejich přesnosti (accuracy)
- metoda náhodných lesů
 - trénování
 - pro každý strom náhodně (s opakováním) vybereme sample trénovacích dat (= bagging)
 - pro každý vrchol náhodně vybereme podmnožinu features, které při dělení bereme v úvahu
 - predikce
 - u regrese průměrujeme hodnoty jednotlivých stromů
 - u klasifikace hlasujeme
 - les vs. strom
 - strom se dá rychle natrénovat, jeho predikci lze snadno interpretovat (jako sérii rozhodnutí)
 - les je obvykle přesnější a lépe generalizuje

7.8 Statistické testy

- studentův t-test (jednovýběrový a dvouvýběrový)
 - jednovýběrový t-test
 - způsob, jak získat intervalový odhad pro θ , neznáme-li rozptyl σ^2
 - uvažujeme statistiku $T = \frac{\bar{X}_n - \theta_0}{\bar{\sigma}/\sqrt{n}}$, která má t-rozdělení s $n - 1$ stupni volnosti, pokud H_0 platí
 - intervalový odhad
 - * najdeme bodový odhad $\hat{\theta}$ pro θ
 - třeba pokud nás zajímá μ , tak prostě použijeme výběrový průměr
 - * máme n hodnot
 - * $\delta = \frac{\bar{\sigma}}{\sqrt{n}} \cdot \Psi_{n-1}^{-1}(1 - \alpha/2)$
 - kde Ψ_{n-1}^{-1} je kvantilová funkce studentova t-rozdělení
 - * vrátíme $[\hat{\theta} - \delta, \hat{\theta} + \delta]$
 - v tomto intervalu bude pravděpodobně reálná hodnota θ pro populaci, z níž jsme data vybrali
 - testování hypotézy
 - * např. pokud ověřujeme, že se střední hodnota rovná nějaké konstantě, tak konstantu i výběrový průměr dosadíme do vzorce pro T
 - * $H_0 \dots T$ má t-rozdělení s $n - 1$ stupni volnosti
 - * H_0 zamítáme, když je spočítaná hodnota statistiky větší než kritická hodnota
 - poznámka: proč uvažujeme $n - 1$ stupňů volnosti, když máme n hodnot?
 - * z $n - 1$ hodnot můžu n -tou hodnotu dopočítat
 - dvouvýběrový t-test
 - můžeme testovat třeba to, jestli se rovnají střední hodnoty parametru θ u dvou různých populací (samlujeme nezávisle, vzorků může být různý počet)
 - např. $H_0 \dots \mu_X = \mu_Y$
 - opět uvažujeme testovou statistiku T s t-rozdělením
 - * $T = \frac{\bar{X}_n - \bar{Y}_m}{\text{se}}$
 - * přičemž se je směrodatná chyba
 - * $\text{se} = \sqrt{\frac{\sigma_X^2}{n} + \frac{\sigma_Y^2}{m}}$
 - případně opět $\delta = \text{se} \cdot \Phi^{-1}(1 - \alpha/2)$
 - bonus: párový test

- data jsou ve dvojicích, dvojice dat spolu nějak souvisí
- např. $H_0 \dots \mu_X = \mu_Y$
- uvažuju $R_i = Y_i - X_i$ (rozdíl dvojice), použiju jednovýběrový test
- chí-kvadrát test (test dobré shody)
 - Pearsonův chí-kvadrát test dobré shody
 - $O_i \dots$ pozorovaná četnost kategorie i
 - $E_i \dots$ očekávaná četnost kategorie i
 - testová statistika: $\sum_i \frac{(E_i - O_i)^2}{E_i}$
 - typická úloha: je kostka spravedlivá?
 - použijeme χ^2 distribuci pro 5 stupňů volnosti
 - * pokud χ^2 pro naši konkrétní kostku náleží kritickému oboru W , prohlásíme, že kostka není spravedlivá
 - $H_0 \dots$ pozorované četnosti jednotlivých kategorií odpovídají jejich očekávaným četnostem
 - pokud H_0 platí, má testová statistika rozdělení χ^2 rozdělení s $n - 1$ stupni volnosti, přičemž n je počet kategorií
 - H_0 zamítáme, když je spočítaná hodnota statistiky větší než kritická hodnota

7.9 Učení bez učitele

- shlukování (algoritmus k-means)
 - algoritmus
 - na vstupu máme body $x_1, \dots, x_N$, počet clusterů K
 - inicializujeme středy clusterů $\mu_1, \dots, \mu_K$
 - * buď náhodně (ze vstupních bodů)
 - * nebo pomocí `kmeans++`
 - první střed se vybere náhodně (ze vstupních bodů)
 - každý další střed vybereme z nepoužitých bodů tak, že pravděpodobnost výběru každého bodu úměrně roste s druhou mocninou vzdálenosti jeho nejbližšího středu (clusteru)
 - opakujeme, dokud to nezkonverguje nebo nám nedojde trpělivost
 - * body přiřadíme clusterům (každý bod přiřadíme tomu clusteru, k jehož středu má nejblíže)
 - * aktualizujeme středy clusterů (vždy zprůměrujeme body, které jsme danému clusteru přiřadili)
 - použití
 - K -means se používá ke clusteringu, tedy k dělení dat do skupin, clusterů
 - je to technika učení bez učitele (unsupervised), neznáme targety
 - konkrétní příklady: seskupení pixelů podobné barvy, určení skupin zákazníků
 - ztrátová funkce, kterou algoritmus optimalizuje
 - $J = \sum_{i=1}^N \sum_{k=1}^K z_{i,k} \|x_i - \mu_k\|^2$
 - kde $z_{i,k}$ je indikátor přiřazení bodu x_i do clusteru k
 - konvergence
 - aktualizace přiřazení bodů ke clusterům nezvyšuje loss
 - aktualizace středů clusterů nezvyšuje loss
 - takže K -means konverguje k lokálnímu optimu
- hierarchické shlukování

- dva přístupy: postupné spojování menších shluků (aglomerativní shlukování) nebo naopak dělení velkých shluků na menší (divisivní shlukování) podle zadaných pravidel
- algoritmus spojování menších clusterů/shluků
 - na začátku je každý cluster singleton (patří tam jeden bod)
 - slučujeme nejpodobnější cluster, dokud nezískáme cílový počet clusterů
- dají se používat různé metriky podobnosti shluků (např. vzdálenost mezi centroidy shluků nebo přímo mezi jejich body, průměry vzdáleností, ...)
- příklady algoritmů divisivního shlukování
 - DIANA (divisive analysis)
 - * na začátku jsou všechny body v jednom clusteru
 - * v něm najdeme bod, který se nejvíc liší od všech ostatních a oddělíme ho do samostatného clusteru
 - * do téhož (nového) clusteru po jednom přidáváme i další body z původního clusteru, pokud se více hodí do nového clusteru (jsou mu blíže než původnímu)
 - * v každé fázi algoritmu se jeden cluster rozdělí na dva (ty jsou jakoby vnořené do původního clusteru)
 - * algoritmus končí, jakmile je každý bod ve vlastním clusteru
 - * můžeme získat dendrogram - graf, jak se clusterly postupně dělily
 - když ho vhodně zařídíme, tak získáme použitelné dělení (aby clusterů nebylo ani málo, ani moc)
 - principal directions divisive partitioning (PDDP)
 - * používá princip PCA - dělí clusterly pomocí projekce na hlavní komponenty
- většinou se jedná o hladové algoritmy → nelze zaručit, že je řešení optimální
- redukce dimenzionality (analýza hlavních komponent)
 - singulární rozklad (SVD, singular value decomposition)
 - mějme matici $X \in \mathbb{R}^{m \times n}$ s hodnotí r
 - $X = U \Sigma V^T$
 - * $U \in \mathbb{R}^{m \times m}$ ortonormální
 - * $\Sigma \in \mathbb{R}^{m \times n}$ diagonální matice s nezápornými hodnotami (tzv. singulárními čísly) zvolenými tak, aby byly uspořádány sestupně
 - * $V \in \mathbb{R}^{n \times n}$ ortonormální
 - možný pohled na dekompozici
 - * $X = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T$
 - * $\sigma_i \dots$ i -té singulární číslo
 - * $u_i \dots$ i -tý sloupec matice U
 - * $v_i^T \dots$ i -tý řádek matice V^T
 - redukovaná verze SVD
 - * σ označme vektor singulárních čísel
 - * pro matici X s hodnotí r bude v tom vektoru nejvýše r nenulových hodnot
 - * tedy matici Σ můžeme redukovat na rozměry $r \times r$, podobně U můžeme redukovat na $m \times r$ a V^T na $r \times n$, touto kompresí nepřijdeme o žádné informace
 - dokonce můžeme Σ zmenšit ještě víc
 - * zvolíme $k < r$
 - * necháme jenom k největších singulárních čísel, všechny ostatní zahodíme
 - * tím už se nějaké informace ztratí
 - * budeme mít aproximaci původní matice X , ta aproximace bude mít hodnot k
- algoritmus určení PCA (principal component analysis) M -té dimenze na základě datové

matice X

- spočteme SVD matice $(X - \bar{x})$
 - * kde $\bar{x}$ je průměr řádků matice (vektor průměrných hodnot jednotlivých features)
 - * k určení SVD (respektive nalezení V) lze použít algoritmus power iteration
 - do proměnné S uložíme kovarianční matici $\text{cov}(X) = \frac{1}{N}(X - \bar{x})^T(X - \bar{x})$
 - pro každou komponentu hledáme vektor v_i (sloupec matice V) tak, že začneme s náhodným vektorem a poté ho zleva násobíme maticí S a normalizujeme (dělíme vlastní normou), dokud to nekonverguje
 - od matice S na konci každého kroku odečteme $\lambda_i v_i v_i^T$, kde λ_i je norma vektoru v_i před poslední normalizací
 - necháme prvních M řádků V^T , ostatní zahodíme (takže V bude mít M sloupců)
 - pokud chceme získat hodnoty nových M komponent, tak stačí vynásobit XV , tak dostaneme matici $N \times M$
 - funguje to díky tomu, že pomocí SVD matice $(X - \bar{x})$ získáváme vlastní vektory kovarianční matice $\text{cov}(X) = \frac{1}{N}(X - \bar{x})^T(X - \bar{x})$
- aplikace PCA
 - aproximace matic
 - snížení šumu v datech
 - redukce dimenzí dat, extrakce features
 - vizualizace dat
 - praktické použití: doporučovací systémy
- příklad použití PCA
 - [\[obr.: příklad\]](#)
 - PCA postupně hledá osy s největším rozptylem